Software Quality Assurance, Verification, and Testing

Onlinevarsity
your e-way to learning

Software Quality Assurance, Verification, and Testing

© 2014 Aptech Limited

All rights reserved.

No part of this book may be reproduced or copied in any form or by any means – graphic, electronic or mechanical, including photocopying, recording, taping, or storing in information retrieval system or sent or transferred without the prior written permission of copyright owner Aptech Limited.

All trademarks acknowledged.

APTECH LIMITED

Contact E-mail: ov-support@onlinevarsity.com

First Edition - 2014

Preface

A high quality software product satisfies user needs, confirms to its requirements and design specifications, and exhibits an absence of errors. We know that good software would be of the best quality if it were cost effective and easily managed. In order to add these special features to our project, we need to be acquainted with the concepts of Software Assurance, Verification, and Testing.

Software quality assurance is an activity that is applied at each step in the software process. Quality assurance is a planned and systematic pattern of all the actions necessary to provide adequate confidence that the item or product confirms to established technical requirements. This leads to systematically developed software systems, which are of good quality and economical.

This book covers the Evolution of Quality concepts, Software Quality, principles for evaluating the Quality Attributes of a Software Architecture, Recommended Best Industrial Practice for Software Architecture Evaluation, Quality Metrics, Quality Standards and the related case studies.

Verification is the process of determining whether products of a given phase of the software development process fulfill the requirements established during the previous phase. **Validation** is the process of evaluating software at the end of the software development process to ensure compliance to software requirements. In general, validation involves actual testing and it happens after verifications are completed. **Testing** involves operation of a system or application under controlled conditions, and evaluation of the results. The controlled conditions should include both normal and abnormal conditions. Testing should intentionally attempt to make things go wrong to determine if things happen when they should not, or things do not happen when they should. This whole process is necessary for good quality software. The book covers the Software Verification and Validation, Software Testing, Significance of Testing, Designing Unit Test cases, and Organizational Approaches for Unit Testing.

This book is meant for programmers, software project mangaers and other people who make a software development team.

The only pre-requisite for reading this book is that students must be familiar with software development in any programming language.

This book is the result of a concentrated effort of the Design Team, which is continuously striving to bring you the best and the latest in Information Technology. The process of design has been a part of the ISO 9001 certification for Aptech-IT Division, Education Support Services. As part of Aptech's quality drive, this team does intensive research and curriculum enrichment to keep it in line with industry trends.

We will be glad to receive your suggestions.

Design Team, H.O.

Onlinevarsity
your e-way to learning

WRITE-UPS BY

EXPERTS AND **LEARNERS**

TO PROMOTE NEW AVENUES AND ENHANCE THE LEARNING EXPERIENCE

FOR FURTHER READING, LOGIN TO

www.onlinevarsity.com

Table of Contents

Sessions

Software Quality Assurance, Verification, and Testing

Software Quality Assurance

1. Evolution of Quality Concepts
2. Software Quality
3. Principles for Evaluating the Quality Attributes of a Software Architecture
4. Recommended Best Industrial Practice for Software Architecture Evaluation
5. Quality Metrics
6. Quality Standards
7. Case Study

Software Verification, Validation, and Testing

1. Introduction to Verification and Validation
2. Verification and Validation
3. Significance of Testing
4. Software Testing
5. Organizational Approaches for Unit Testing
6. Designing Unit Test Cases

ASK to LEARN

Onlinevarsity
your e-way to learning

Questions
in your
mind?

are here to HELP

Post your queries in ASK to LEARN @

www.onlinevarsity.com

Chapter

1 Evolution of Quality Concepts

Objectives

At the end of this chapter, you will be able to:

- *Describe the evolution of Quality concepts*
- *Describe various Quality measures*
- *Describe Total Quality Management*
- *Explain various frameworks that use TQM*

1.1 Introduction

It is a misleading notion that quality is commonly associated as being pricey. Any product that is claimed as having quality is supposed to be costly. The popular misconceptions and vagueness of the term 'quality' do not help the quality improvement effort in the software industries. The term must have a precise definition before a product can achieve even a semblance of quality. Crosby (1979) defines quality as "*conformance to requirements*" and Juran (1970) defines it as "*fitness for use*". These two definitions are interrelated with each other.

"*Conformance to requirements*" suggests that requirements must be clearly stated in such a manner that these cannot be misunderstood later. Next, the development and the production process measurements are taken continually to determine the conformance to those stated requirements. A nonconformance is regarded as a '*defect*' or the absence of quality. The definition "*fitness for use*" takes the customer's requirements and expectations into account. A customer's requirements and expectations resolve whether the product suits his purpose. The role of a customer can never be undermined.

The two definitions are actually two different perspectives of quality. The first one is known as the **producer's view of quality**, and the second is the **customer's view of quality**.

1.2 The evolution of quality concepts

As mentioned before, there are two perspectives on quality: customer satisfaction as the ultimate validation and producer's adherence to meeting the required specifications to achieve quality. Accordingly, two stages of qualities exist. The first is the **intrinsic** product quality, limited only to defect rate and reliability. This is a narrow definition of quality, also called "**small q**". The second stage is a broader definition of quality, which includes product quality, process quality and customer satisfaction, and is known as "**big Q**". These two levels of quality form a closed loop cycle. This cycle begins with the customer stating his requirements.

The second phase of this cycle is to lay down the product specifications. Designing and development constitutes the third phase of the cycle.

This is illustrated in Figure 1.1.

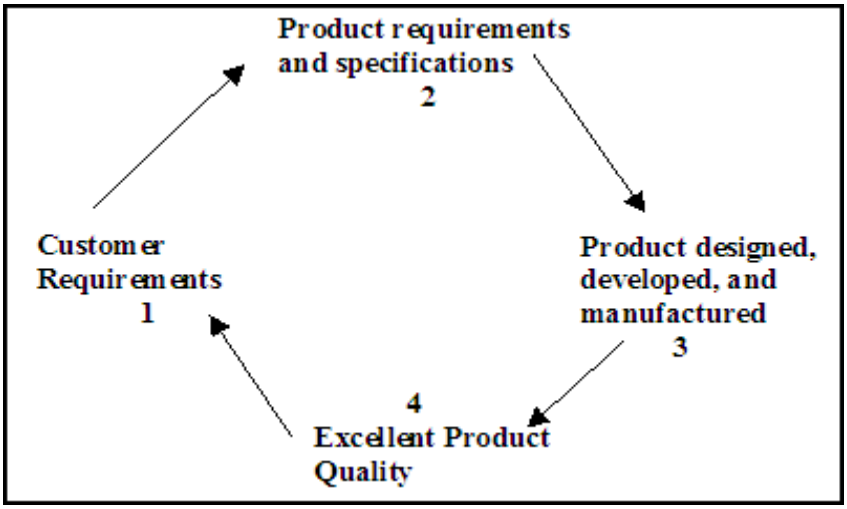


Figure 1.1: The quality loop

This loop is being followed in many industries including automobile, computer hardware and software, especially after 1980s when the quality concepts started gaining importance and wide acceptance. Product requirements before 1980s often used to be generated without customer input, and customer satisfaction was not a factor in the business –decision-making.

In software, the product quality is commonly referred to as the lack of "**bugs**" or functional defects in its narrowest sense of the definition of quality. This definition is usually expressed in two different ways:

- **Defect rate**: The number of defects per thousand lines (KLOC) of source code, per function point, or any other unit of the program size.

- **Reliability**: The number of failures per 'n' hours of operation, mean time to failure, which indicates how often the system fails, or the probability of failure-free operation in a specified time.

Customer satisfaction is measured in percent satisfied or non-satisfied by means of customer satisfaction surveys. We can reduce the bias employing techniques such as **blind surveys** where the interviewer does not have any prior knowledge of the customer and the customer does not know the company the interviewer represents. In addition to the overall customer satisfaction with the software product, satisfaction towards specific quality attributes is measured. For example, IBM monitors the **CUPRIMDSO** (Capability, Usability, Performance, Reliability, Install ability, Maintainability, Documentation and Serviceability and Overall) satisfaction levels of its software products. Hewlett-Packard focuses on the **FURPS** (Functionality, Usability, Reliability, Performance and Serviceability) for its products. Similar methodologies are being followed by

other organizations. Quality attributes, such as these are called **quality parameters** or **quality measures** or **parameters of fitness for use**.

A **quality measure** is a software feature or characteristic used to assess the quality of a system or component. Various quality measures are Correctness, Maintainability, User-friendliness, Scalability, Availability, Interoperability, Productivity, Verifiability, Visibility and Timeliness.

- A software program is called '**functionally correct**' if it behaves according to the specifications of the functions it is expected to provide. These functionalities are listed in the '**functional requirements specifications**'.
- The measure "**maintainability**" commonly refers to the modifications that are made to a software system after its initial release. Typically, it is viewed as a '**bug-fixing**' activity.
- A software product is **user-friendly** if a user finds it easy to use.
- The measure '**scalability**' refers to the ability of the system to perform in the same way, irrespective of the number of users using the system, with up gradations made to the hardware only.
- **Availability** is a measure of how often the software application is operational versus how often it is down for the maintenance, repairs, or unscheduled problems.
- **Interoperability** measure is defined as the ability of two or more systems or components to exchange information and to use the information that has been exchanged. It refers to the ability of a system to coexist and cooperate with other systems.
- **Productivity** measures the **efficiency** of the process. An efficient process results in faster delivery of the product.
- A software product is verifiable if its properties can be verified easily. **Verifiability** is defined by Evans as "the relative effort to verify the specified software operation and performance".
- A software development process is **visible** if all its steps and its current status are documented clearly.
- **Timeliness** is a process related quality that refers to the ability to deliver a product on time.

Another view of quality is that of the in-process quality versus the end-product quality. Starting from the customer requirements to the delivery of software products, the development process is complex and involves a series of stages, each with its own feedback paths. In each stage, an intermediate deliverable is produced for an intermediate user, which is the next stage. Each stage also receives an intermediate deliverable from the previous stage. Each intermediate deliverable will have its own quality attributes that affect the quality of the end product. If each stage of the development process meets the requirements of its next stage, the end product of the process will meet all the specified requirements. Therefore, to improve quality we require models during development process, and within the process it will be necessary to select and deploy specific methods and approaches, and use proper tools and technologies. We shall also require various metrics. This will help ensure that the development process is under control to meet the product's quality objectives.

1.3 Total Quality Management (TQM)

Since the 1980s, many companies worldwide have started adopting the **TQM** approaches to quality. Total Quality Management (TQM) represents a management style, the aim of which is the achievement of the long-term success by linking the concept of quality with the customer satisfaction. The aim of TQM is to create a culture in which all the members of the organization participate in the improvement of processes, products and services. Specific methods for the implementation of the TQM were put forward by Crosby (1979), Deming (1986), Freigenbaum (1961, 1991), Ishikawa (1985) and Juran (1970).

The **major elements** of any TQM system are:

- **Customer focus**: The objective is to achieve total customer satisfaction. The customer focus includes studying the customer's wants and needs, gathering customer requirements, and measuring and managing customer satisfaction.

- **Process**: The objective is to reduce the process variations and to achieve continuous process improvements. Processes include both the business process and the product development process. Product quality will be improved through the process improvement.

- **Human side of quality**: The objective is to create a company wide quality culture. Focus areas include leadership, management commitment, total participation, employee improvement, and other social, psychological and human factors.

- **Measurement and analysis**: The objective is to achieve improvement continuously in all quality parameters by the goal-oriented measurement system.

Moreover, an organization that practices TQM must focus on infrastructure, training, and education, and must do strategic quality planning. The diagram in Figure 1.2 illustrates the various parts of a TQM system in an organization.

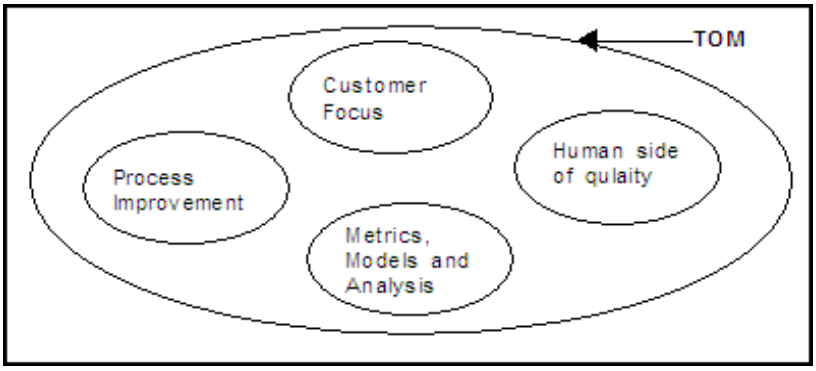


Figure 1.2: Key elements of Total Quality Measurements

1.4 Various frameworks using the foundation of the TQM philosophy

In the computer and electronic industries, examples of some of the successful implementations of TQM are Hewlett-Packard's **Total Quality Control** (TQC), Motorola's **Six Sigma Strategy**, and IBM's **Market Driven Quality**. Hewlett-Packard's TQC focuses on key areas, such as the management commitment, leadership, customer focus, total participation, and systematic analysis. There are strategies and plans in each area to improve quality, efficiency, and responsiveness with the final objective being to achieve success through customer satisfaction (Shores 1989).

Motorola's **Six Sigma** strategy focuses on achieving the stringent quality level to obtain total customer satisfaction. Cycle time reduction and participative management are among the key initiatives of the strategy (Smith 1989). The Six Sigma strategy is not just a measure of the quality level, but the concept includes the production design improvement and reduction in the process variations (Harry and Lawson, 1992). It represents a specific defect rate of 3.4 defects per million lines of code of a software product over its life cycle.

Various frameworks have gone behind the foundation of the TQM philosophy. Some examples are Plan-Do-Check-Act (Shewhart, 1931, Deming, 1986), Quality Improvement paradigms/ Experience Factory Organization (Basili, 1985, 1989; Basili and Rombach, 1987, 1988), Software Engineering Institute (SEI) Capability Maturity Model (Radice, 1985, Humphrey, 1989), and Lean Enterprise Management (Womack, 1990).

(*Source: Metrics and Models in Software Quality Engineering by Stephen H. Kahn*)

1.4.1 Plan-Do-Check-Act

Plan-Do-Check-Act (PDCA) cycle is a quality improvement process based on a feedback cycle for optimizing a single process or production line. It uses techniques such as feedback loops and statistical quality control to experiment with methods for improvement and build predictive models of the product. The basic assumption is that a process is repeated several times so that data models that allow one to predict results of the process can be built.The PDCA cycle is shown in Figure 1.3.

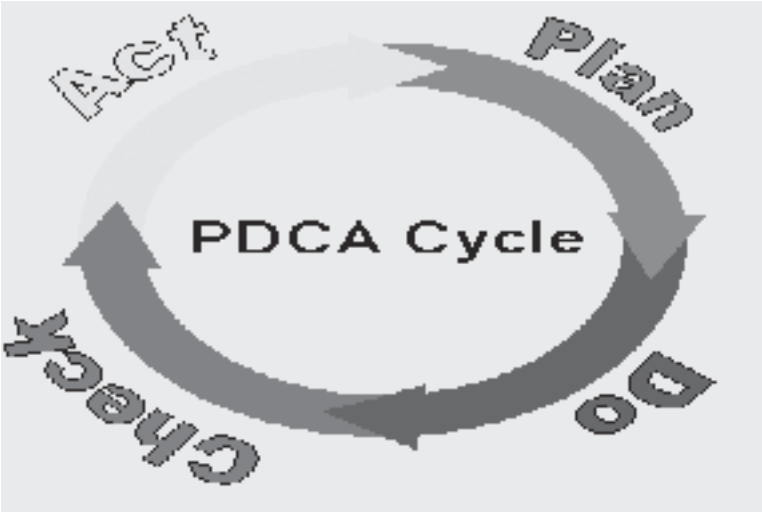


Figure 1.3: The PDCA cycle (Source: http://www.dartmouth.edu)

Various activities in the PDCA cycle include:

- **Plan:** A change or a test, aimed at improvement. In this phase, it is required to look for potential areas that need improvement. The first step includes areas that offer the most in return for your investment.
- **Do:** Carry out the change or test incrementally. Implement the change that was planned.
- **Check**: Check for any discrepancy that might have crept in while carrying out the planned change.
- **Act:** Adopt the change, abandon the change or run through the cycle once again.

After planning a change, while implementing and monitoring the change if we observe that it is worth continuing with the change, we may consider expanding the trial to a different area or the same area with slightly higher complexity. This sends us back to the Plan phase and it can be the beginning of the **ramp of improvement**, as illustrated below in Figure 1.4.

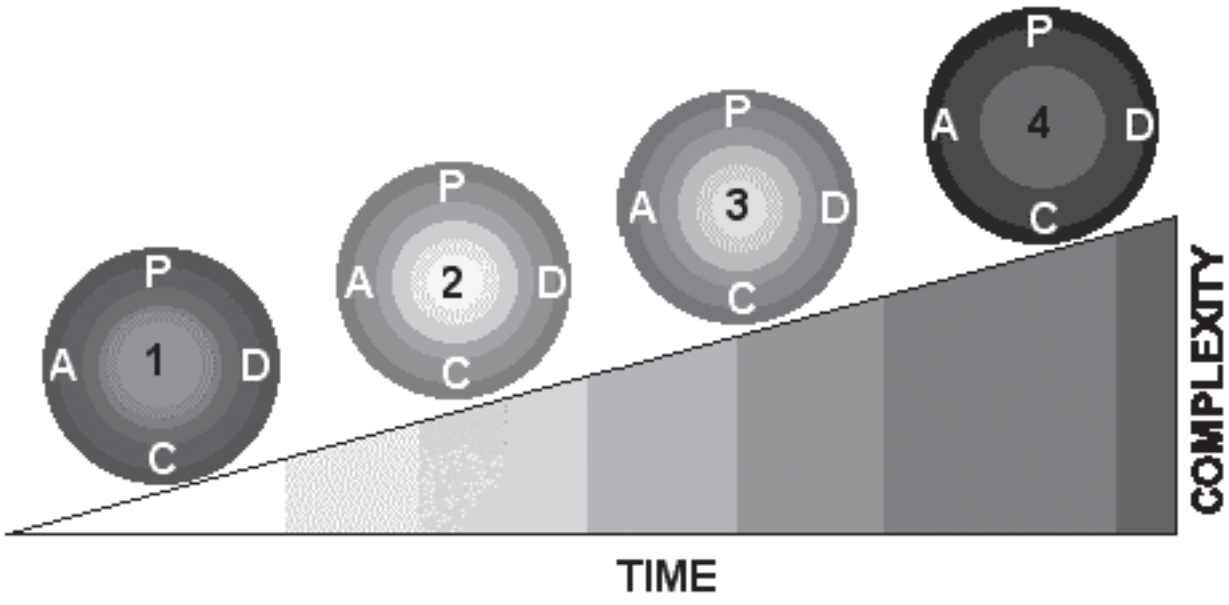


Figure 1.4: The ramp of improvement (Source: http://www.dartmouth.edu)

The ramp of improvement goes in a phase-by- phase manner, as follows:

P=Plan:

1. Define a problem or opportunity.
2. Analyze the situation. Study and define the problem and think continuously for causes and best possible corrective actions.
3. Develop the implementation plan.

D=Do:

1. Implement the corrective action.
2. Document the procedure and observation.
3. Use data-gathering tools to collect information.

C=Check:

1. Analyze information.
2. Monitor trends.
3. Compare the obtained results against the expected results from the plan.

A=Act:

1. If the results are as expected, do nothing.
2. If the results are not as expected, repeat the PDCA cycle.
3. Document the process, and the revised plan.

1.4.2 Quality Improvement Paradigm (QIP)

The **Quality Improvement Paradigm** (QIP)/ **Experience Factory Organization** attempts to build a continually improving organization, based on its evolving goals and an assessment of its status relative to those goals. This approach uses internal assessments against an organization's own goals and status rather than the process areas, and employs such techniques as **Goal/ Question/ Metric** (GQM), model building, and quantitative/ qualitative analysis to improve the product through the process. The GQM paradigm suggests a general model or framework for collecting data that can be useful for a specific purpose from projects. The type and extent of data collection depends on the goal of the organization. This indicates that an organization must initially specify its goals. These goals will then be translated to specific questions

along with a framework for the evaluation of the collection of data. An example of a goal can be to reduce the total number of defects found in the products released by an organization. The question can be "what currently is the quality of the released product"? The next step is to define metrics that need to be collected to answer these questions that have been framed for each goal. For example, one metric can be the number of field defects per KLOC found within a year of release of the product. The six fundamental steps of the QIP are:

- **Characterize the project and its environment**: Comprehend the current processes and the environment it operates in.
- **Set goals**: Based on the understanding of the process and the environment and objectives of the organization, set the quantifiable goals for performance improvement. The goals must be reasonable.
- **Choose processes**: Based on the characterization and goals, choose the processes that should be modified to meet the goals.
- **Execute processes**: Execute projects using the processes and provide the feedback data.
- **Analyze the data**: Analyze the data at the end of each project. Using the analysis, determine problems and make recommendations accordingly for improvements to be applied on future projects.
- **Package the experience**: Based on the experience gained from many projects, define and formalize the changes made to the processes and expectations from the new processes.

Figure 1.5 shows various steps in Quality Improvement Program (QIP) and Figure 1.6 shows with the feedback loop that is part of the program.

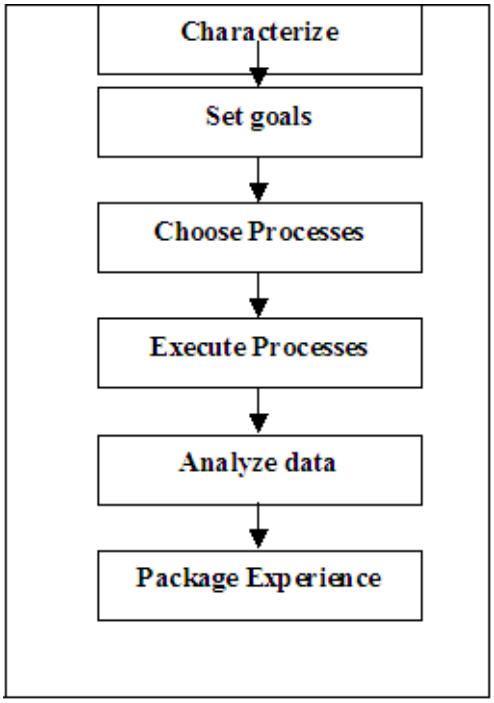


Figure 1.5: The Quality Improvement Program (QIP)

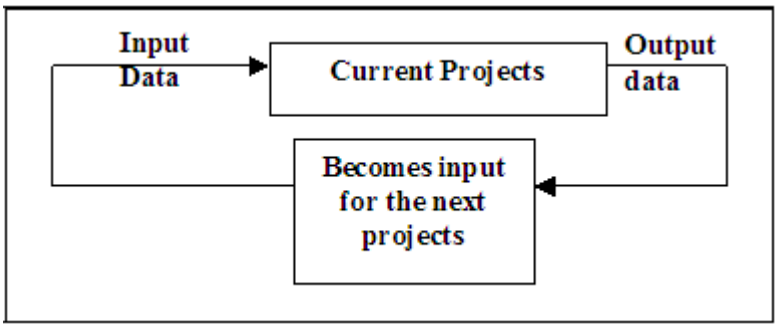


Figure 1.6: The Feedback Loop

An important part of the QIP is the feedback from the previous projects. Collecting the appropriate data provides the basic input data for the process improvement about the existing projects.

The **Experience Factory Organizations** separate the product development from the experience packaging activities. The basic objective of the **QIP/Experience Factory organizations** is to gain the learning experience from multiple project developments.

1.4.3 Software Engineering Institute (SEI)

The **Software Engineering Institute (SEI) Capability Maturity Model** is a leveled (level 1 to level 5) process improvement based on assessment with regard to a set of key process areas, until level 5 is reached. This level represents continuous process improvements. This approach is based on organizational and quality management maturity models developed by Lickert (1967) and Crosby (1979). The goal of this model is to achieve continuous process improvement through defect prevention, technology innovation and the process change management.

As a part of this method, a **five-level process maturity model** is defined based on the repeated assessments of an organization's capability in the key process areas. Process improvement is achieved by action plans for the poorly scoring areas. The basic assumption of this approach is that there are key process areas that will improve the software development. The levels and the process model have been discussed further in the later chapters.

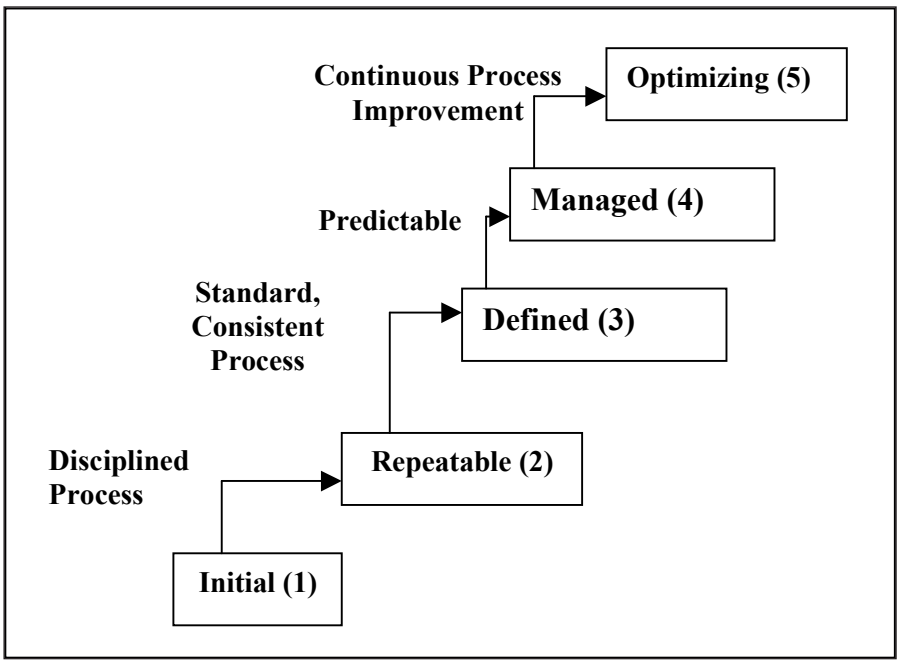


Figure 1.7: Capability Maturity Model

Let us discuss various levels of SEI-CMM as shown in Figure 1.7.

"Level 1 - The Initial Level

At the Initial Level, the organization typically does not provide a stable environment for developing and maintaining software. Such organizations frequently have difficulty in making commitments that the staff can meet with an orderly engineering process, resulting in a series of crisis. During a crisis, projects typically abandon planned procedures and revert to coding and testing. Success depends entirely on having an exceptional manager and a seasoned and effective software team. Thus, at Level 1, capability is a characteristic of the individuals, not of the organization.

Level 2 - The Repeatable Level

At the Repeatable Level, policies for managing a software project and procedures to implement it are established. Planning and managing new projects is based on the experience with similar projects. The process capability is enhanced by establishing the basic process management discipline on a project-by-project basis. An effective process can be characterized as one, which is practiced, documented, enforced, trained, measured, and able to improve.

Projects in Level 2 organizations have installed basic software management controls. Realistic project commitments are based on the results observed on previous projects and on the requirements of the current project.

Level 3 - The Defined Level

At the Defined Level, the standard process for developing and maintaining software across the organization is documented, including both software engineering and management processes, and these processes are integrated into a coherent whole. This standard process is referred throughout the CMM as the organization's standard software process. Processes established at Level 3 are used to help the software managers and technical staff performs more effectively.

The software process capability of Level 3 organizations can be summarized as standard and consistent because both software engineering and management activities are stable and repeatable. Within established product lines, cost, schedule and functionality are under control, and software quality is tracked. This process capability is based on a common, organization-wide understanding of the activities, roles, and responsibilities in a defined software process.

Level 4 - The Managed Level

At the Managed Level, the organization sets quantitative quality goals for both software products and processes. Productivity and quality are measured for important software process activities across all projects as part of an organizational measurement program. An organization-wide software process database is used to collect and analyze the data available from the projects' defined software processes. Software processes are instrumented with well-defined and consistent measurements at Level 4. These measurements establish the quantitative foundation for evaluating the projects' software processes and products.

Projects achieve control over their products and processes by narrowing the variation in their process performance to fall within acceptable quantitative boundaries. The risks involved in moving up the learning curve of a new application domain are known and carefully managed.

The software process capability of Level 4 organizations can be summarized as being quantifiable and predictable because the process is measured and operates within measurable limits. This level of process capability allows an organization to predict trends in process and product quality within the quantitative bounds of these limits. As the process is both stable and measured, when some exceptional circumstance occurs, the "special cause" of the variation can be identified and addressed. When the known limits of the

process are exceeded, action is taken to correct the situation. Software products are of predictably high quality.

Level 5 - The Optimizing Level

At the Optimizing Level, the entire organization is focused on continuous process improvement. The organization has the means to identify weaknesses and strengthen the process proactively, with the goal of preventing the occurrence of defects. Data on the effectiveness of the software process is used to perform cost benefit analysis of new technologies and proposed changes to the organization's software process. Innovations that exploit the best software engineering practices are identified and transferred throughout the organization.Software project teams in Level 5 organizations analyze defects to determine their causes. Software processes are evaluated to prevent known types of defects from recurring, and lessons learned are disseminated to other projects.

(**Source: Software Engineering Institute (SEI) Web site**)

SUMMARY

- A software product must meet the customer's stated requirements
- Software is one of the most difficult products that defy quantification
- Various Quality Measures are required to quantify software as a product to meet exactly a customer's stated requirements
- Total Quality Management (TQM) represents a management style, the aim of which is the achievement of the long-term success by linking the concept of quality with the customer satisfaction
- Plan-Do-Check-Act (PDCA) cycle is a quality improvement process based on a feedback cycle for optimizing a single process or production line
- The Quality Improvement Paradigm (QIP) attempts to build a continually improving organization, based on its evolving goals and an assessment of its status relative to those goals
- The Software Engineering Institute (SEI) Capability Maturity Model is a leveled (level 1 to level 5) process improvement based on assessment with regard to a set of key process areas, until level 5 is reached

CHECK YOUR PROGRESS

1. The definition "*fitness for use*" takes the customer's requirements and expectations into account.

   [True/False]

2. In software, the product quality is commonly referred to as the lack of "bugs" or _____.

3. Customer satisfaction is measured in percent satisfied or non-satisfied by means of customer satisfaction surveys.

   [True/False]

4. Plan-Do-Check-Act (PDCA) cycle does not use techniques such as feedback loops.

   [True/False]

5. The basic objective of the QIP/ Experience Factory organizations is to gain the learning experience from multiple project developments.

   [True/False]

6. The Software Engineering Institute (SEI) Capability Maturity Model has 6 levels.

   [True/False]

Chapter

2 Software Quality

Objectives

At the end of this chapter, you will be able to:

- *Describe what is Software Quality*
- *Explain Software quality assurance*
- *Explain goals of SQA at various stages of Software development*
- *Implement Software quality assurance*

2.1 Introduction

Most programmers are very casual about controlling the quality of the software they write. They develop some code, run it through some fairly obvious ad hoc tests, and if it seems okay, it is assumed to be fine. While this approach may work well for small, personal programs, it does not work for professional software development. Modern software engineering practices include considerable effort directed towards software quality assurance and testing. The idea, of course, is to produce completed software systems that have a high probability of satisfying the customer's needs.

There are two ways to deliver software free of errors. The first is to prevent the introduction of errors in the first place. And the second is to identify the bugs lurking in the code, seek them out, and destroy them. Obviously, the first method is superior. A big part of software quality comes from doing a good job of defining the requirements for the system and designing a software solution that will satisfy those requirements. Testing concentrates on detecting those errors that creep in, despite the efforts put in to keep them out.

2.2 Why Worry About Software Quality?

When a computer hobbyist develops software, he does not think much about software quality. The "quality" of one-user, short-lived programs like these does not really matter much, since they are not for public consumption.

The professional software engineer developing commercial products or systems for use by his employer has a more serious problem. Besides the initial effort of writing the program, software maintenance is a serious issue. "Maintenance" is everything that happens to a program after one assumes it is done. In the real world, software maintenance is a major issue. Industry estimates

indicate that maintenance can consume up to 80 percent of a software organization's time and energy.

The job of maintenance is greatly facilitated if the software being changed is well structured, well documented, and well behaved. More often, however, code is written rather sloppily and is badly structured. Over time, such code becomes such a tangle of patches and fixes that one is better off re-writing the whole module, rather than trying to fix it once again. High-quality software is designed to survive a lifetime of changes.

2.3 What is Software Quality?

Roger Pressman, a noted software engineering author and consultant, defines software quality as:

Conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software.

A shorter definition is a high-quality software system, which is delivered to the users on time, costs no more than was projected, and most importantly, works properly. "Working properly" implies that the software must be as nearly bug-free as possible.

While these are workable definitions, they are not all-inclusive. For example, by building a software system that conforms precisely to a set of really bad specifications, one does not have a high-quality product. Additionally, software developer's help to ensure that the system specifications are of high quality (that the specifications properly address the user's needs) as well as builds an application that conforms to this specification.

A couple of other important points are implied in this definition. It indicates that one has specifications for the programs being written. Too often, one works from a fuzzy notion of what one is trying to do. This fuzzy image becomes refined over time, but if one has been writing code during that time, one finds that much of it has to be changed or thrown out.

Another implication is that "software" includes more than the executable code. The "deliverables" from a software development project also include the written specifications, system designs, test plans, source code documentation, and user manuals. Specifications and designs might include narrative descriptions of the program requirements and structure, graphical models of the system (such as data flow diagrams), and process specifications for the modules in the system.

Software quality impacts other system deliverables just as it affects the source code. The quality of documentation is particularly important. Very often, people try to change someone else's code without being able to understand the mindset at the time it was written. Detailed documentation about the parts of the software system, the logic behind them, and how they fit together is extremely important. However, erroneous documentation is worse since it can lead down a blind alley.

There's a compelling economic incentive for building quality into software. The true cost of a software development project is the base cost (what we spend to build the system initially) plus the rework cost (what we spend to fix the errors in the system). The rework cost rarely is figured into either the time or money budgets, with the consequence that many projects cost much more to complete than expected and soak up still more money as work is done to make the system truly conform to the specifications.

2.4 Software Quality Assurance (SQA)

Software quality assurance is the sub field of software engineering devoted to seeing that the deliverables from a development project meets the acceptable standards of completeness and quality. The overall goal of SQA is to lower the cost of fixing problems by detecting errors early in the development cycle. SQA has a watchdog function of looking over the other activities involved in software development.

First, one cannot test quality into a product; it has to be built in. Testing can only reveal the presence of defects in the product. Second, software quality assurance is not a task that is performed at one particular stage of the development life cycle, and most emphatically not at the very end. Rather, SQA permeates the entire development process. Third, people not directly involved in the development effort best perform SQA. The responsibility of the SQA effort is to the customer, to ensure that the best possible product is delivered, rather than to the software developers or their management.

2.5 Goals of SQA at various stages of software development

The goals of SQA are universally applicable at the various stages of structured software development including requirements analysis, system specification, system design, code implementation, testing, and maintenance.

2.5.1 Requirements Analysis

- Ensure that the system requested by the customer is feasible (many large projects have a separate feasibility study phase even before gathering formal requirements).
- Ensure that the requirements specified by the customer will in fact satisfy his real needs, by recognizing requirements that are mutually incompatible, inconsistent, ambiguous, or unnecessary. Sometimes needs can be addressed in better ways than those the user is requesting.
- Furnish the customer with the appropriate software system that will actually be built to address his stated requirements. Simple prototypes often are useful for this.
- Avoid misunderstandings between developers and customers, which can lead to untold grief and hassles farther, down the road.

2.5.2 Software Specifications

- Ensure that the specifications are consistent with the system requirements, by setting up a requirement traceability document. This document lists the various requirements, and then tracks how they are addressed in the written specifications, the system design (which data flow process addresses the requirement), and the code (which function or subroutine satisfies the requirement).
- Ensure that specifications have been supplied for system flexibility, maintainability, and performance where appropriate.
- Ensure that a testing strategy has been established.
- Ensure that a realistic development schedule has been established, including scheduled reviews.
- Ensure that a formal change procedure has been devised for the system. Uncontrolled changes, resulting in many sequential (or even concurrent) versions of a system can really contribute to quality degradation.

2.5.3 Design

- Ensure that standards have been established for depicting designs (such as data flow diagram models for process-oriented systems, or entity-relationship models for data-oriented systems), and also that the standards are being followed.
- Ensure that changes made to the designs are properly controlled and documented.
- Ensure that coding does not begin until the system design components have been approved according to agreed-upon criteria. Of course, we all do some coding before we really “should”; so long as we think of it as “prototyping”, it is fine. We should not get carried away prematurely.
- Ensure that design reviews proceed as scheduled.

2.5.4 Coding

- Ensure that the code follows established standards of style, structure, and documentation. Even though languages like C allows us to be super-compact and hence super-obscure in our coding, we must not forget that a human being (maybe even we) may have to again work with that code. Clarity of code is usually preferred over conciseness.
- Ensure that the code is being properly tested and integrated, and also that revisions made in coded modules are properly identified.

- See that code writing is following the stated schedule. The customer is not entitled to know the schedule but he should know its impact on the delivery time.
- Ensure that code reviews are being held as scheduled.

2.5.5 Testing

- Ensure that test plans are created and they are being followed. This includes a library of test data, driver programs to run through the test data, and documentation of the results of each formal test that has been performed.
- Ensure that the test plans that are created, addresses all the system specifications.
- Ensure that after testing and reworking, the software conforms to the specifications.

2.5.6 Maintenance

- Ensure the consistency of code and documentation. This is quite difficult; we tend not to update documentation when the program is changed. However, such an oversight can create nightmares the next time a change has to be made.
- Ensure that the established change control process is being observed, including procedures for integrating changes into the production version of the software (configuration control).
- Ensure that changes made in the code follow the coding standard, are reviewed, and do not cause the overall code structure to deteriorate.

Most professional software engineers do not follow this stringent as a quality plan. Unfortunately, the result of having no SQA plan deteriorates the quality of the software.

2.6 Making SQA Happen

The first step towards implementing a quality improvement program in a software organization is to define and document the procedures for software development. Unfortunately, there is no one procedure for software development, which can be used by all organizations. Each organization has its own climate and addresses different customer segments.

2.6.1 Quality management systems

The starting point of defining the processes should be to have a QUALITY POLICY, WHICH STATES THE AMBITION or the desire regarding where the management would like the organization to reach over a period of time. If the policy can be stated in measurable terms, it is better so that we can keep measuring the parameter to see if we are moving towards the target.

Typically, organization quality policies talk of defect rates, productivity figures or a combination of both. Once the quality policy has been finalized, work should commence on documenting the procedures, standards, methodologies and others. In order to ensure that the documentation process is done on a time bound basis, it is important that it assigns a full time responsibility to a set of people. Hence, the company must have a full time quality assurance group or software engineering process group (SEPG), which will define monitor and fine tune processes based on feedback, on a continuous basis.

Getting the processes documented by people having lien responsibilities on a part time basis is a sure prescription for failure. The QA team will consult the relevant people while documenting the processes and find out that the people doing the job, review the documents before implementing the same.

a) Processes

Processes are a series of steps, which must be taken in order to achieve the end goal of developing software. Processes of similar nature are clubbed together into key process areas (KPA), which are essential for systematic development of software. Topical KPAs in software development environment will be project initiation, planning, tracking, development, implementation, recruitment, training, configuration management, quality management, vendor management and metrics analysis.

An organization must try to implement all key process areas simultaneously as they may run into implementation problems. Each process must have an input and an output, and may have some tools to perform the function.

b) Standards

Standards are rules, which give details of forma and content of the input and the output processes. People are generally familiar with coding or programming standards. They list out a set of rules, which state the ‘dos’, and ‘don’ts ‘ of programming in a particular language.

The most important feature of a standard points out that it must be verifiable. A third person must be able to check out the input or output to a process and come to a conclusion. As technology is changing rapidly, it is better that standards are checked through automated tools.

c) Techniques

An organization might want to standardize on the use of some industry techniques - a technique may be defined as a set of specialized sub-processes to achieve a very specialized function.

In today’s GUI world, many software personnel are not aware of writing good efficient batch processing programs, which are required for end of the day activities. IBM had developed a technique called B-Line, which could aid in writing efficient batch programs whereby a record is read only once, processed and updated after processing.

This also resulted in structured and easy to maintain code. There are many other techniques, which the organization could ask all software personnel to follow. Some typical techniques, which organizations standardize on are Sorting, Searching, Printing, and Selection.

d) Guidelines

Guidelines are the same as standards with one exception. Organizations expect full compliance to standards while the software personnel depending on the requirements and environments of a project could adapt guidelines. This implies the difference between a rule and a norm.

e) Templates

Templates are an aid to implementing standards. The best way to describe a template is that it gives us “Fill in the blanks”. There can be templates for all documentation; it can define a standard table of contents and expect people to fill in the details for each project. Similarly, there can be template programs for maintenance of tables, passing financial entries into general ledger, printing of reports and others. These template programs can be used and filled in with details, which are required for the project.

f) Checklists

A shopping checklist is a list of items to be purchased and items are knocked off the list as and when they are bought. A checklist of the software development environment has a similar function. Checklists are aids to ensure that a person does not miss an item or activity by oversight. Typically, checklists are made where a person has to think of a number of items and is likely to miss out if a checklist does not aid him. For example, listing of resource requirements for a project.

g) Forms

Most organizations are going in for electronic computer based forms instead of paper based forms. Today, most organizations have electronic signature or approvals on forms, whenever required. Paper based forms are cumbersome to implement and maintain. A quality-oriented organization will promote an electronic culture within their organization.

h) Document structure

The workflow document should be separate into two separate volumes for ease of reference. One volume should list all the processes and its descriptions while the second volume can have all the other items- standards, techniques, guidelines, templates, checklists and forms. This makes referencing easy. There should be a cross-reference across the two volumes, as appropriate. It is a good practice to keep these documents under a Document Management System for easy access to all personnel. Hardcopies may be maintained as additional copies, if required.

i) Document Update

The workflow document must be updated periodically. Otherwise, it is not possible to have continuous

improvement in the organization. There must be a procedure to monitor implementation of the process. This could be done through process audits, which will give feedback on problems in implementing some of the processes. These problems must be received by a committee, which should recommend changes in the process definitions, if required.

After getting the required approval, the process should be updated in the workflow document. This procedure must be done periodically. If it is done too frequently then people will not have time to understand and implement the process. If not done frequently enough then the benefit of continuous improvement is lost. It should be balanced depending on the organizational needs.

2.6.2 Quality Control Techniques

Let us discuss various techniques of quality control, which can be applied at various stages or milestones in the SDLC.

a) Reviews

Many software-engineering authorities believe that the single most effective SQA activity is to hold structured reviews or walkthroughs at key stages of the development process. The review process involves collecting several pairs of skilled eyes, besides those of the developer, into a room and scrutinizing the deliverables from a particular phase of system development; the purpose of the review is to find errors. It has the additional benefit of educating our associates about our project, which can be valuable if the members of a software development group ever have to do maintenance on systems they did not build (the usual real-world case).

It is better to schedule reviews or walkthroughs at these points during development - after the requirements specification has been completed; after overview design is complete (usually a data flow model); after detail design is complete (process specifications for individual modules); after the test plan is written; and after coding has been done. It also helps to always include the primary customer representative (“project champion”) in the requirements and overview design reviews.

The specific topics covered during an SQA review depend on the nature of the deliverables produced. Reviews of specifications and overview designs should examine data flow diagrams and data models for consistency, proper balancing between levels, accuracy, and completeness of the data dictionary. During the detail design review, structure charts and process specifications are examined for errors and inconsistencies.

A code review checks for the presence of all the features described in the process specification for the module being studied. Again, we should look for inconsistencies with the data dictionary and other design documents. The code should be checked for quality of structure, documentation, completeness, complexity, and clarity. Look for the characteristics of well-structured modules, with

high cohesion (performing a single task) and low coupling to other modules.

Each step of the review should check all deliverables against the formal standards we agreed upon at the outset of the project. This includes system and code-level documentation. Consistency in development of the products is more important than the standards selected.

b) Walkthroughs

Walkthroughs are one-on-one reviews and more detailed. The objective in addition to finding weakness is to confirm that the reader has properly understood the document or item. In a walkthrough, the author gives the item to the person who is going to do the work for reading and understanding. After the person has read the document, the reader states an understanding to the author doing the walkthrough. This will serve as a dry run through the item and also confirm that what has been stated is what has been understood. If any corrections arise out of the discussion, the author updates the item accordingly.

c) Inspection

Inspection involves examination of an activity by a person who is not the creator for the adherence to the documents. At the end of the inspection, the inspector must give a formal note listing the deviations from set standards and procedures. These must be corrected by the creator and a re-inspection may be required if the number of deviations are high.

2.6.3 SDLC Stages

a) Customer requirement specifications

This is normally the starting point of a SDLC. Since the customer writes it, it cannot be checked against any organizational standard unless the customer happens to be an in-house customer. The normal check or verification done on customer requirements is to see if it is complete. If there are some points, which are not clear then a list of clarifications can be made and discussed with the customer. The verification for completion may have to be done till all points are clarified. It might be a good practice to update the specifications given by the customer with the clarifications so that it is complete. A copy of the updated specifications can be given to the customer.

b) Estimation

Once customer requirements are clear, the project manager must evaluate the various options for executing the project. It is a good practice to review the options with peers to ensure all options have been considered and the best option put forward to the customer. Once the options have been

narrowed down, a detailed estimation is worked out for all options. A good way to ensure proper estimation is to ensure that more than one individual does the estimation and checks to see the reasons for the variation of the estimation. It is important to note that all estimated must be reviewed before submission to the customer.

c) Functional specifications

The first stage of the SDLC after the estimates have been approved is to write the functional specifications. After this, a panel of experts reviews the specifications. The people who should review the specifications should include functionality experts, data security experts, and personnel with expertise on systems with which the proposed system should interface and not other area in which the expertise is required. It must be noted that the customers should not be a part of the internal review. After this, a review should be done with the clients who are end user of the product. After incorporating the changes in the review, a sign off on the functional specifications must be obtained before the commencement of the system design.

d) Design

Design specifications translate the functional requirements into the computer system, which will meet the customer needs. The design specifications should be reviewed by a panel of experts in various areas of technology – database, security, telecommunications, development methodology performance tuners and application experts.

The objective of this phase is to ensure that all requirements have been taken care of in the design and there is no control or security weakness in the design. Depending on the gravity of the situation, a decision on whether or not a second review is required will be taken.

After the review is completed, the project team does a detailed analysis of all the points. Based on the results of the analysis, the review comments are incorporated into the design specifications. The customer is not a part of this review.

e) Verification

The best technique to verify a program specification is a walkthrough. The specification is given to the programmer who is actually going to do the coding of the program. After reading and understanding the specifications, the programmer explains his understanding of the specifications. Any missed out point or incorrect understanding of the specification is noted. After the walkthrough, the specification is updated (if required) before coding is started.

f) Unit test plans

This follows the same quality control technique as program specifications. Normally, if a programmer analyst has written the unit test plans, the module leader will do the walkthrough of the UTP. The main emphasis on the review must be to ensure coverage through a variety of transactions and not volume testing using a large number of transactions. A good UTP must try to cover as many lines of the code as possible. Many organizations have standards, which state that the minimum

percentage of lines that must be executed for satisfactory completion of unit testing.

g) Coding

The major emphasis of quality control on coding is adherence to programming standards. This is verified through the inspection of code. Inspection can be done manually through a second person or through automated standard checkers. If automated checkers are available then tolerance limits can be set – if code is not adhering to standards by less than X% then rewrite the code, or the non-adherence percentage is greater than y% then the code has to be modified to meet acceptable percentages before unit testing can commence.

h) Unit Testing

Unit testing involves testing of individual components of the system to see if they are functioning properly. Unit testing must be done based on the documented Unit Test Plan. The author of the program must first do unit testing. The testing can then be verified by a second person to ensure that it has been completed as per the UTP. Use of automated testing tools with a replay facility will save time on re-entry of data and will expedite the process.

i) System Testing

Integrated testing of the system with emphasis on interfaces between configuration items and modules is the essence of System Testing. The emphasis is on testing the functionality of the system, its security, recovery and restart procedures and interfaces with external systems, if any.

j) Acceptance Testing

Acceptance testing is similar to system testing except that users or customers do it. Acceptance testing is done based on an Acceptance Test Plan, which is written and reviewed by users. The only emphasis in Acceptance Testing is to ensure that business needs of the users are addressed by the new system.

j) Regression Testing

When errors are found during testing, they are corrected and tested and testing proceeds as per plan. The key question is how one ensures that a new error has not been introduced during correction in an area that has already been tested. The most logical decision is to restart testing always from the beginning but it is not the most practical solution. Testing done to ensure that corrections to software during testing phase do not introduce new errors in Regression Testing.

k) System Documentation

When developing a system, a number of documents have to be produced for use and operations of the system. Typically, a system will have a user manual for users, operations manual for operations personnel, and technical documentation for maintenance personnel. The best person to review the documentation is the target audience of the concerned document.

User manual should be reviewed by users and verified by using it during acceptance test of the

system. Operations personnel should review operations manual while peer reviews should be done on technical documentation.

l) Implementation

This is the stage when the users or customers see the real benefit of using the new system.

Typically the steps involved are:

- Implementation Planning and Testing
- Installation of Software
- Training of users and operations personnel
- Data Conversion
- Parallel Run
- Live operations

The best form of verification for all the above activities is either a peer review or a panel review. Sign Off's are important on completion of data conversion and parallel run.

m) Project Management

This is the most important and often ignored activity. For projects of reasonable size, it is common to appoint a steering committee, which will assist the project leader to resolve project management issues. The project leader makes a presentation on the status of the project against plan and the reasons for deviation, if any.

A slippage analysis is done and the required corrective action is initiated. By conducting these meetings periodically, problems are known in a timely manner and not after they have reached a crisis point where corrective actions become impractical. If there are delays, then the customer is informed of the revised schedule.

2.6.4 Testing

a) Test Case Design

Historically, testing has occupied the greatest fraction of the time and effort associated with software development. A big reason for this is the lack of attention traditionally paid to system specification and design. Of course, one can continuously test any program and still not be completely sure that it will work perfectly 100% of the time. However, the usage of thorough, structured test procedurescan help build confidence in the quality of the product being developed.

The testing process begins with the development of a testing plan. In principle, either an independent testing person/ group or the software developer can do this.

b) Testing Strategies

There are two different aspects to testing a piece of software. First, does software properly perform its intended functions? Second, is the structure of the code free of syntax and logic errors? This is the dichotomy of “functional” versus “structural” testing. The customer is concerned with the first case; he does not care what the code looks like, so long as it does the job. He thinks of the program as a black box: he supplies the inputs, and by some magical process, he gets the desired outputs back. The developer is concerned about the structure and flow of his code; to test it properly, he must study the code and devise tests in accordance with the way the program is written. These two aspects of software testing are called “black box” and “white box” testing, respectively.

c) System Integration and Integration Testing

Generally one works on a software project with a few dozen separate modules. The unit-level testing has convinced that the units are well behaved and properly coded. The next problem is to assemble all these modules together into the final system, in accordance with the program architecture devised during system design.

There are basically two ways to approach the process of system integration and integration testing. One-way is to pull the modules all together and hope the compiler succeeds in compilation. This is called “big-bang” integration, and it’s almost guaranteed to fail. Alternatively, one could use an incremental approach of joining modules into small clusters and testing the clusters as we go. This technique is superior, and it will cost much less. These topics will be discussed in detail in the later chapters.

SUMMARY

- High quality software is the one that is delivered on time.
- SQA goal is to lower the cost of fixing problems by detecting errors early in the development cycle.
- SQA is applied to every stage of software development.
- Different techniques are available to implement SQA.
- The starting point of defining the processes in Quality Management Systems is to have a QUALITY POLICY, WHICH STATES THE AMBITION.
- Reviews, walkthroughs and Inspection techniques of quality control, are applied at various stages of SDLC.
- Testing is essential to produce a good quality Software.

CHECK YOUR PROGRESS

1. Maintenance can consume 80 percent of a software organization's time and energy. **[True/False]**

2. SQA is not applicable to coding stage of development **[True/False]**

3. Developers should give the customer a good idea about the system that will be built as per his requirements. **[True/False]**

4. Changes made to the designs need not be documented. **[True/False]**

5. There should be consistency of code and documentation. **[True/False]**

6. The goal of walkthrough is to discover and make note of problem areas. **[True/False]**

7. Inspections can be used throughout the software life cycle to access and improve the quality of work products. **[True/False]**

Onlinevarsity
your e-way to learning

To enhance your knowledge,

visit REFERENCES

@

www.onlinevarsity.com

Chapter

3 Principles for Evaluating the Quality Attributes of a Software Architecture

Objectives

At the end of this chapter, you will be able to:

- *Explain the principles for evaluating Quality Attributes of a Software Architecture*

3.1 Introduction

Software quality is the degree to which software possesses a desired combination of attributes (examples: reliability and interoperability). The three user-oriented product characteristics include quality, cost, and schedule. Cost and schedule can be predicted and controlled to some extent by mature organizational processes. However, process maturity does not translate automatically into product quality. Software quality requires mature technology to predict and control quality attributes. If the technology is lacking, even a mature organization will have difficulty producing products with predictable performance, dependability, or other attributes.

Quality, cost, and schedule are not independent. Poor quality eventually affects the cost and schedule because software requires tuning, recoding, or even redesign to meet original requirements. Cost and schedule overruns are common because serious problems are often not discovered until the system integration phase.

Few would disagree that it is more cost effective to detect potential software quality problems earlier rather than later in software development. Recently, the software architecture has received attention as being the right focal point for the detection of aberrations in software.

Horror stories that illustrate such problems are common. “On Monday, September 2, 1991, at 9 a.m., a $6 million manufacturing support systems/network integration program went live, the largest computer project the company had undertaken. By 10:30 a.m., ‘The system had miserably failed,’ reported the program manager of a chemical company. ‘We had not anticipated needing so much memory, consequently, the system froze in less than two hours, stopping all work at the site’ ”. The quickness of the disaster suggests that the designers were flying blind throughout the development of the system.

Let us discuss few principles for analyzing software architecture to determine if it exhibits certain quality attributes. Analysis techniques indigenous to the various quality attribute communities can provide a foundation for performing software architecture evaluation. Principles provide a context for existing evaluation approaches such as scenarios, questionnaires, checklists, and measurements.

There is no agreement on the definition of architecture. For example, in some communities, the artifact that we refer to as the “architecture” is referred to as the “design” and architecture is a higher-level abstraction from which many designs could be derived.

3.2 Overview of the Principles

In this section, we provide a brief description of the principles. This involves identification of the contract between the system and the environment, Identification of the software architecture, Identification of the hardware resources allocated to the software components and Analysis of the information gathered by application of the previous principles.

3.2.1 Information-Gathering Techniques

Quality is relative to the intended use of the system. An evaluation of quality must take into consideration the environment surrounding the system in addition to the system itself. The system can be decomposed and any of its parts can be subject to the analysis. Everything outside the current focus of interest constitutes its “environment.” Therefore, the environment is strictly relative to the system being evaluated at the moment and will change as the system changes.

Figure 3.1(a) illustrates the relationships between the types of information needed for the analysis and the recursive nature of this analysis.

Figure 3.1(b) illustrates the principles underlying an attribute based architecture evaluation. In this system, three components process input data from the environment and pass their results to a fourth component. The last component in turn sends results back to the environment.

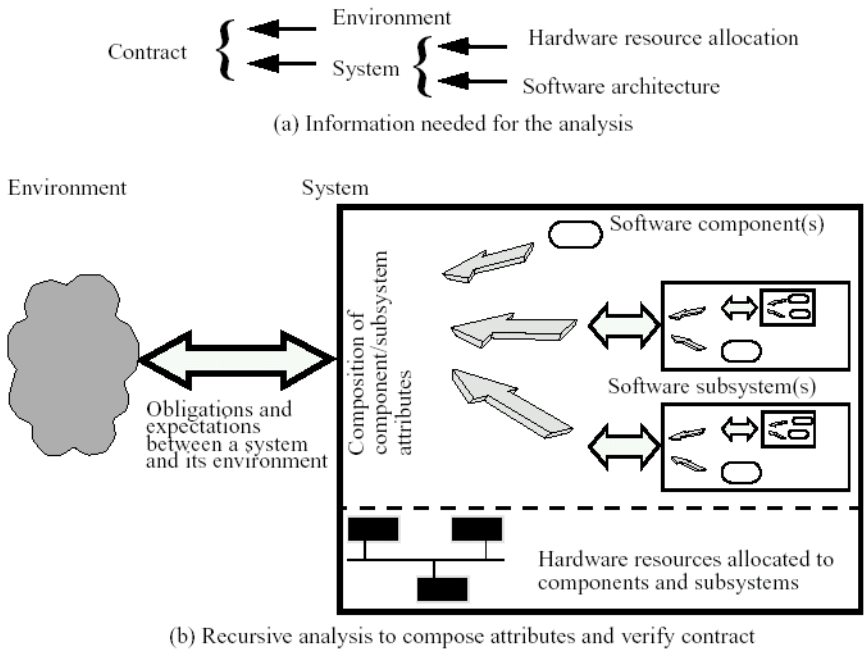


Figure 3.1: Information Gathering and Analysis

The system typically consists of software and hardware components. As per our convenience, we differentiate between these two types of components to focus on the architecture of the software components; the software architecture. We use the terms subsystem and component to refer to the building blocks of a system.

A subsystem is made up of smaller subsystems or components; a component cannot be decomposed. The system and its environment are partners in a “contract” where the system and the environment both have expectations of each other and where both have obligations to meet these expectations. The expectations of each party must be consistent with the obligations of the other party. The principle of “design by contract” is described, as the principle that interfaces between modules of software system, which should be governed by precise specifications. The contracts will cover mutual obligations (preconditions), benefits (post conditions), and consistency constraints (invariants).

Subsystem/components are relatively neutral terms to describe the structure of a system. The term “process” might be loaded with implications about the number of threads of control or processor allocation. A subsystem/ component can have any number of threads and can use any number of processors.

Scenarios, checklists, and questionnaires are qualitative techniques applicable to the identification of the contract, the hardware resource allocation, and the software architecture. When conducting an evaluation, one or more of these techniques might be applied, depending on the environment and the system.

The fundamental design concepts are the operational principle and normal configuration. The operational principle defines how the device works; it defines the device and provides criteria for (technical) success. The normal configuration includes the shapes or arrangements that are commonly agreed to best embody the operational principle. “Every device possesses an operational principle, and, once it becomes an object of normal every day design, a normal configuration. Engineers doing normal design bring these concepts to their task usually without thinking about them”.

Although the focus of the article is on object-oriented analysis and synthesis, the first three activities are applicable to a broader class of software engineering situations, which are discussed below:

- Characterize the system — “This step elicits those characteristics of the systems that should shape the architectural design”. A questionnaire helps to focus the activity and ensure that all issues are addressed. This questionnaire emphasizes fundamental design considerations regarding size, memory usage, data access time, throughput, identification of critical threads, response time, and the like.

- Define conceptual entities — “This step defines and describes precisely the conceptual entities of the architecture and the relationships that must hold among them”. “Which objects appear on an OIM [Object Information Model] depend on the concepts inherent in the architecture under construction”.

- Define theory of operation — “This step describes precisely how the system works as a whole. We have found that an informal but comprehensive theory-of-operation document or technical note works well to develop the appropriate concepts. This document should describe the threads of control that run through the architecture domain, covering all modes in which the system operates, such as normal system operation; cold-start and initialization procedures; warm-start, restart, and fail over operation, as required for the delivered system; and shutdown”.

3.2.2 The Identification of the Contract

This principle requires that the evaluators identify the expectations and obligations of the system. This type of analysis is not inconsistent with the notion of a discovery evaluation in which the incipient architecture specifications are checked against requirements.

There are different types of attributes. Some are measured by system activities (examples: latency and availability), some are measured by inspection activities (examples: coupling and cohesion), and some are measured by user activities (example: time to complete a task). In addition, depending on the attributes of interest, the environment might be an operational environment (example: networks, and users), a development environment (example: life-cycle organizations), or a policy environment (examples: laws and institutional regulations). These cover a lot of ground. Expectations and obligations must be observable and measurable to produce meaningful evaluation.

System obligations and expectations are written in the form of scenarios. These scenarios are short descriptions of a requirement, an operational situation, a modification to the system, and the like.

The following are illustrative examples of scenarios identified from a contract:

- The environment depends on data from the system and expects no more than 1 hour of system down time in a year (minimum availability requirement).
- The system failure rate is less than one failure/month (minimum reliability requirement).
- The environment expects the response time or latency of the system to be less than 100 milliseconds (worst-case latency requirement).
- The system must process up to 20 input events per minute (throughput requirement).
- The system can resist over flooding by excessive rate of input events (security against a type of attack).
- The environment expects the system to allow processor and network upgrades (modifiability of resources).
- The environment provides inputs with exponentially-distributed arrival times with arrival rate.

These scenarios define what needs to be confirmed by the analysis.

3.2.3 The Identification of the Software Architecture

Software architecture is characterized by a particular combination of software components and connections. This principle requires that the evaluators identify the components and connections within the system.

There are different kinds of connections between components including structure, the component connections showing the flow of data, behavior, the underlying semantics of the system and the components including the flow of control. Knowledge of the operational principle and normal configuration [Vincenti 90] are essential to the identification of the architecture.

3.2.4 The Identification of the Hardware Resource Allocation

This principle requires that the evaluators collect information about the underlying computation, storage, and communication resources. These are necessary to the analysis because they are consumed by or shared between software components. They will have an impact on the overall quality of the system to the extent that hardware resources are finite or fallible.

The system under consideration is implemented with standard processors and local area networks. In order to simplify things, assume that we are not using shared memory multiprocessors; memories belong to a processor. Variations include:

- Independent — Each software component executes on a separate processor.

- Shared — All software components execute in the same processor as schedulable units of concurrency (i.e. process).

Although the number of resource allocation schemes is potentially unbounded, in reality, resources are likely to be chosen from a small collection of commercial, off-the-shelf options. The identification of the hardware resources and their allocation to software components can be carried out through questionnaires and checklists that ask what resources (examples: processors, memory and storage devices, buses, networks) are used by each software component. It is particularly important to identify shared resources, which might not be apparent from the descriptions of the structure and behavior.

3.2.5 The Analysis

This principle requires that the evaluators use the information gathered so far to determine if the system will be able to fulfill its obligations. The goal of this analysis is to ensure that components (hardware and software) cooperate in a manner that ensures that the system's obligations are fulfilled. Analysis is performed from the point of view of the system. Part of the analysis takes place during the contract identification, to ensure that the system expectations and system obligations are stated in a manner that is unambiguous and consistent. Of course, this assumes that the system will meet its obligations; such a detailed analysis is necessary to verify that the system can fulfill each of its obligations.

Realizing that an architectural design is still a relatively high-level design, accurate prediction might not be possible. However, the evaluation process will be useful in gaining insights into the system and should continue to be used throughout the development. In the absence of quantitative data, quantitative assumptions (or budgets) can be established and modified as development progresses.

3.2.6 System Profiles and Software Architecture Trade-offs

A system can be subject to various types of attribute-specific analysis. Hence, for each quality attribute of interest one can apply a specific process to analyze the system from that attribute's point of view. In a given contract, applying the analysis process to two different systems (systems that differ in software architecture or resource allocation) is likely to yield different results along some attribute specific metric.

These analysis processes are not necessarily formal or quantitative; the process depends on the attribute. Calculating and understanding the value of a single overall metric for software quality may be more troublesome than it is worth. The major problem is that many of the individual characteristics of quality are in conflict; added efficiency is often purchased at the price of portability, accuracy, understandability, and maintainability; added accuracy often conflicts with portability via dependence on word size; and conciseness can conflict with legibility.

Hence, we can conclude that the analysis techniques associated with each attribute provide the basis for generating scenarios, questions, and checklists. The only information that needs to be gathered is the information that is needed to perform the analysis. Thus, one practice that might emerge from this work is the development of precise statements about the following:

(1) The type of information one could expect to see in a component's "attribute specification"

(2) The type of attribute obligations a component should be able to make

(3) The type of attribute compositions a component could be engaged in

SUMMARY

- Software quality is one of the three user-oriented product characteristics (quality, cost, and schedule).
- An evaluation of quality must take into consideration the environment surrounding the system in addition to the system itself.
- The fundamental design concepts are the operational principle and normal configuration.
- The evaluators must identify the expectations and obligations of the system.
- The evaluators must identify the components and connections within the system.
- The evaluators must collect information about the underlying computation, storage, and communication resources.
- The evaluators must use the information gathered so far to determine if the system will be able to fulfill its obligations.
- A system can be subject to various types of attribute-specific analysis.

CHECK YOUR PROGRESS

1. Poor quality does not affect the cost and schedule.
   [True/False]

2. A system consists of _____and hardware components.

3. The operational principle defines how the device works.
   [True/False]

4. System obligations and expectations are written in the form of _____.

5. The goal of analysis is to ensure that hardware and software components cooperate in a manner that ensures that the system's obligations are fulfilled.
   [True/False]

6. Software architecture is characterized by a particular combination of software _____ and connections.

Onlinevarsity

your e-way to learning

Technowise

Are you a

TECHNO GEEK

looking for updates?

Login to

www.onlinevarsity.com

Chapter

4 Recommended Best Industrial Practice for Software Architecture Evaluation

Objectives

At the end of this chapter, you will be able to:

- *Explain software architecture evaluation in terms of cost and benefits*
- *Describe various categories of architecture evaluation techniques*

4.1 Introduction

It is fitting that the architectural decisions receive close scrutiny because of their importance. Few will argue against the cost-effectiveness of quality evaluation as early as possible in the software development life cycle. If problems are found early in the life cycle, they are trivial to correct. The necessary corrections will be a change to a requirement, or specification, or design. Software quality cannot be appended late; it must be inherent from the beginning, built-in by design.

The definition of software architecture is the earliest point in the life cycle of a system where an artifact exists that can be checked to determine how well the system will meet its requirements—both explicit and implicit. It is in an organization's best interest to evaluate (and reject, if necessary) prospective design candidates during the design phase, before long-term institutionalization occurs. Therefore, an effective technique to evaluate candidate architecture before it becomes the accepted blueprint is of great economic value.

Furthermore, when acquiring a large software system that will have a long lifetime within the acquiring organization, it is important that the organization develops an understanding of the underlying architecture of candidate systems. This understanding allows an assessment of the suitability of the candidates with respect to qualities of importance, particularly with respect to how the candidates will support the future needs of the organization.

It should be noted that architecture could not guarantee the functionality or quality required of a system. Poor downstream design, implementation, testing, or management decisions can always undermine an acceptable architectural framework. Decisions at all stages of the life cycle—from high-level design to coding and maintenance—affect quality. A software architecture evaluation assesses the ability of the architecture to support the desired qualities. Refinements of the architecture into implementation that preserve the qualities are necessary for the final product to actually achieve those qualities.

An architecture evaluation should be held as early as possible in the life cycle of the project, and that the evaluation should be conducted as a formal review. Ideally, such a review is held early enough so that decisions that have been made can be revoked without extensive cost, but late enough so that bindings to

hardware have been made to analyze the performance characteristics of the system. This places the evaluation somewhere in the early phases of high-level system design.

4.2 Costs and Benefits

There are a number of motivations for performing an architecture evaluation, but there is an associated cost in terms of time and committed resources. An organizational issue is to decide whether the benefits are worth the cost.

4.2.1 Costs

Some of the workshop participants maintain records to help determine the cost of a full architecture evaluation. For example, AT&T, having performed approximately 300 full-scale architecture evaluations on projects requiring a minimum of 700 staff days, reported that the average cost of an architecture evaluation review was based upon estimates from individual project managers, 70 staff days. In addition, two of the consultants who performed architecture evaluations reported cost estimates. Rational Software Corporation has performed around 30 evaluations and charges an average cost of $50K for projects of at least 500 KLOC in size.

Most of the organizations represented (AT&T, Siemens, Nortel, Motorola, and Lockheed Martin) have established corporate units responsible for defining and conducting architecture evaluations. One of the main reasons for this centralization is that each company is interested in maximizing the amount of corporate reuse at the architectural level. All these companies reported that the individual start-up costs for such an organization were non-trivial.

Architecture evaluation teams should not include members of the development project. Instead, individuals are selected from within their company to serve on architecture evaluation teams based on their past performance and demonstrated skills with large-scale system organization and design. This membership issue surfaces two cost-related concerns.

First, there is the worry of reduced organization-wide development productivity because superior designers are removed from active involvement. Making a non-permanent membership on the architecture evaluation board can mitigate this cost. The second concern stems from this temporary engagement on the architectural evaluation team. Each new evaluator needs to be trained in the evaluation techniques and gain experience in evaluation before being a productive member of the evaluation team. There is a cost associated with this learning curve for architecture evaluators.

4.2.2 Benefits

The benefits of an architectural evaluation can be categorized as follows.

4.2.2.1 Financial

At AT&T, each project manager reports perceived savings from an architecture evaluation. On average, over the past eight years, projects receiving a full architecture evaluation have reported a 10% reduction in project costs. Given the cost estimate of 70 staff days, this reported 10% cost reduction illustrates that on projects of 700 staff days or longer, the evaluation pays for itself.

Several consultants report that over 80% of their business was repeat business. Their customers recognized sufficient value in architecture evaluation to be willing to pay for additional evaluations. Workshop participants also reported several anecdotes of estimated cost savings for the customers of their evaluations. One consultant reported that as a result of an architecture evaluation, a large company avoided a multimillion dollar purchase when the architecture of the global information system they were procuring was, upon evaluation, not capable of providing the desired system attributes necessary to support a product line. Another reported that early analysis of an electronic funds transfer system showed a fifty billion dollar transfer capability per night: half the desired capacity. An architecture evaluation of a retail merchandise system revealed early that there would be peak order performance problems that no amount of hardware could fix; a major business failure was prevented. In the architecture evaluation of a revision control system, a number of severe limitations in achieving system portability and modifiability were uncovered. A major redesign of the system was recommended.

It is also useful to point out cases where architecture reviews did not occur. One participant described how a rewrite of a customer accounting system was estimated to take two years. After seven years, the system had been re-implemented three times. Performance goals have never been met, despite the fact that the latest version uses sixty times the CPU power of the original prototype version. In a large engineering relational database system, performance problems that were largely attributable to design decisions made integration testing impossible. The project was cancelled after twenty million dollars had been spent.

4.2.2.2 Increased Understanding and Documentation of the System

One of the benefits of any formal review is that it forces the reviewer to prepare for the review. By giving the reviewers an indication of the focus of the architecture evaluation and the requirement for a representation of the architecture before the review is held, the reviewers are required to document the system's architecture. Many systems do not have a top-level architecture that is understandable to all the developers. The architecture is either too brief or, more commonly, too long, perhaps thousands of pages. Furthermore, there are often misunderstandings among developers about some of the assumptions for their components. The process of preparing for the review will reveal many of these problems.

Furthermore, the architecture evaluation focuses on a few specific areas with specific questions to be answered. Answering these questions usually involves giving an explanation of the design choices and their rationales. Having a documented design rationale is important later in the life cycle so that the implications of modifications can be assessed. Capturing design rationales after the fact is one of the more difficult tasks in software development. By capturing the design rationale as presented in the architecture review (even by low-cost methods such as videotaping), invaluable information is available for later use.

4.2.2.3 Detection of Problems with the Existing Architecture

The third benefit of an architecture evaluation is early detection of problems with the existing architecture. The earlier in the life cycle that problems are detected, the cheaper it is to fix them. The problems that can be found by an architectural-level inspection include unreasonable (or expensive to meet) requirements, performance problems, and problems associated with potential downstream modifications. For example, an architecture evaluation, which exercises scenarios of typical system activity, can manifest rough performance specifications. Exercising system modification scenarios can reveal portability and extensibility problems that will be especially critical if the architecture is to support a product line, rather than just a single product. Architecture evaluation, therefore, provides early insight into product capabilities and limitations.

4.2.2.4 Clarification and Prioritization of Requirements

The fourth significant benefit is validation of requirements. Discussion and examination of how well the architecture meets requirements also opens requirements up for discussion. What results is a much clearer understanding of the requirements and usually, a prioritization of the requirements. Requirements creation, when performed in isolation from a high-level design, usually results in specification of conflicting system properties. High performance, security, fault-tolerance, low cost are all easy to demand but difficult to achieve and often impossible to achieve. Architecture evaluations uncover the conflicts and tradeoffs, and provide a forum for their negotiated resolution.

4.2.2.5 Organizational Learning

Organizations that practice architecture evaluation as a standard part of their development process report, anecdotally, an improvement in the quality of the architectures that are reviewed. As development organizations learn to anticipate the kinds of questions that will be asked, the kinds of issues that will be raised, and the kinds of documentation that will be required for evaluations, they naturally pre-position themselves to maximize their performance in the reviews. Architecture evaluations not only result in better architectures after the fact, but before the fact as well. Over time, an organization develops a culture that promotes good architectural design.

In summary, architecture evaluations tend to increase quality, control cost, and decrease budget risk. Architecture is the framework for all technical decisions and, as such has a tremendous impact on cost and quality of the product. An architecture review does not guarantee high quality or low cost, but it points out areas of risk in a design. Other factors such as testing or quality of documentation and coding will contribute to the eventual cost and quality of the system. Benefits other than the immediate ones of

helping to facilitate the construction of the system also accrue. Architectures are manifested in organizational entities (for example, work breakdown structures are based on system decompositions of functionality) but they also reflect existing organizational structures. For example, if subcontractors are developing portions of a system then recognizing commonalities across the portions of the system being developed by the two different subcontractors is difficult. An architecture evaluation that has maintenance as one of its focus areas will detect commonalities regardless of organizational considerations.

4.3 Categorization of Evaluation Techniques

A variety of techniques are used to perform an architecture evaluation. Each of the techniques has a different cost and can be used to elicit different information. There are two basic categories of these techniques: those that generate qualitative questions to ask about architecture and those that suggest quantitative measurements to be made on architecture. Questioning techniques can be applied to evaluate architecture for any given quality. In fact, it is consideration of the quality that drives the development of questions (in the form of checklists or scenarios, as described below).

Questioning techniques, however, do not directly provide a means for answering those questions. Measuring techniques, on the other hand, are used to answer specific questions. In that sense, they address specific software qualities (for example, performance or scalability) but are not as broadly applicable as questioning techniques.

Each of these techniques will be described briefly below, as background for its recommended application.

4.3.1 Questioning Techniques

There are three questioning techniques:

- Scenarios
- Questionnaires
- Checklists

These techniques differ from each other in applicability, but they are all used to elicit discussion about the architecture and increase understanding of the architecture's fitness with respect to its requirements.

4.3.1.1 Scenario

People often want to analyze software architectures with respect to quality attributes expressed using words like maintainability, security, performance, reliability, and so forth. These words provide convenient ways for describing and communicating a host of common, recurring problems in software. However, most software quality attributes are too complex and amorphous to be evaluated on a simple scale. Furthermore, quality attributes do not exist in isolation, but rather only have meaning within a context. A system is modifiable (or not) with respect to certain classes of changes, secure (or not) with respect to

specific threats, usable (or not) with respect to specific user classes, efficient (or not) with respect to its utilization of specific resources, and so forth. This notion of context-based evaluation of quality attributes has led to the adoption of scenarios as the descriptive means of specifying and evaluating quality attributes within a context. A scenario is a specified sequence of steps involving the use or modification of the system. Scenarios provide a means to characterize how well a particular architecture responds to the demands placed on it by those scenarios. Scenarios test what we normally call modifiability (by proposing a set of specific changes to be made to the system), security (by proposing a specific set of threat actions), performance (by proposing a specific set of usage profiles that tax resources), and so on.

4.3.1.2 Questionnaire

A questionnaire is a list of general and relatively open questions that apply to all architectures. Some questions might apply to the way the architecture was generated and documented (by asking if there is a designated project architect or if a standard description language was used). Other questions focus on the details of the architecture description itself (by asking if all user interface aspects of the system are separated from functional aspects). The evaluation team looks for a favorable response and will probe a single question to a level of detail that is necessary to satisfy their concern. An example of a questionnaire-based evaluation process (that happens to go beyond architectural investigation) is the software risk evaluation process [Joseph 94] developed by the Software Engineering Institute. The utility of a questionnaire is related to the ease with which the domain of interest can be characterized and circumscribed.

4.3.1.3 Checklist

"Just as a building inspector uses a checklist to ensure that a structure is in compliance with the building codes, software architects use a checklist to help them keep a balanced focus on all areas of the system." A checklist is a detailed set of questions that is developed after much experience evaluating a common (usually domain-specific) set of systems. Checklists tend to be much more focused on particular qualities of the system than questionnaires. For example, performance questions in a real-time information system will ask whether the system is writing the same data multiple times to disk or whether consideration has been given to handling peak as well as average loads.

4.3.1.4 Questioning Techniques Summary

There is a natural relationship between these three questioning techniques. Scenarios are intended to be system-specific questions. Experience evaluating a family of related systems can result in generalizing a set of commonly used scenarios, turning them into either domain-specific entries in a checklist or more general items in a questionnaire. Checklists and questionnaires reflect more mature evaluation practices. Scenarios can reflect less mature evaluation practices. Another difference is in scenarios, since they are system specific, and are developed as part of the evaluation process. Checklists and questionnaires are assumed to exist before a project begins.

4.3.2 Measuring Techniques

Measuring techniques result in quantitative results. Rather than provide ways to generate the questions

that will be asked about architecture, these techniques provide answers to questions an evaluation team might already have about particular qualities of architecture. Since the questions almost always precede the answers, it is obvious that measuring techniques are more mature than the questioning techniques.

4.3.2.1 Metrics

Metrics are quantitative interpretations placed on particular observable measurements on the architecture, such as fan in/fan out of components. The most well researched measuring techniques provide answers on overall complexity that can suggest places of likely change. With measuring techniques, the evaluation needs to focus not only on the results of the measurement/metric, but also on the assumptions under which the technique was used. For example, a calculation of performance characteristics makes assumptions about patterns of resource utilization. How valid are these assumptions? Coupling and cohesion metrics make assumptions about the types of functionalities embodied in the components being examined. How valid are these assumptions?

4.3.2.2 Simulations, Prototypes, and Experiments

Building a prototype or a simulation of the system may help to create and to clarify the architecture. A prototype whose components consist of functionless stubs is a model of the architecture. Performance models are an example of a simulation. The creation of a detailed simulation or prototype just for review purposes is typically expensive. On the other hand, these artifacts often exist as a portion of the normal development process. In this case, using these artifacts during a review or to answer questions that come up during the review becomes a normal and natural procedure.

An existing simulation or prototype may be an answer to an issue raised by a questioning technique. For example, if the evaluation team asks “What evidence do we have to support this assertion?,” one valid answer would be the results of a simulation.

SUMMARY

- An architecture evolution should be held as early as possible in the life cycle of the project.
- The cost of the architecture evolution must be reduced.
- The benefits of an architectural evaluation are: Financial, Increased Understanding and Documentation of the System, Detection of problems with the existing Architecture, Clarification and prioritization of Requirements and Organizational learning.
- Better understanding and documentation is done for the system during architecture evolution.
- Another benefit of an architecture evaluation is early detection of problems with the existing architecture.
- Architecture evaluations tend to increase quality, control cost, and decrease budget risk.
- Questioning techniques can be applied to evaluate architecture for any given quality.
- Measuring techniques result in quantitative results.

CHECK YOUR PROGRESS

1. If problems are found early in the life cycle, they are difficult to correct. **[True/False]**

2. Architecture evaluation teams should not include members of the development project. **[True/False]**

3. Architecture evaluations tend to increase quality, control cost, and decrease budget risk. **[True/False]**

4. Measuring techniques are used to answer specific _____.

5. A questionnaire is a list of general and relatively open questions that apply to all architectures. **[True/False]**

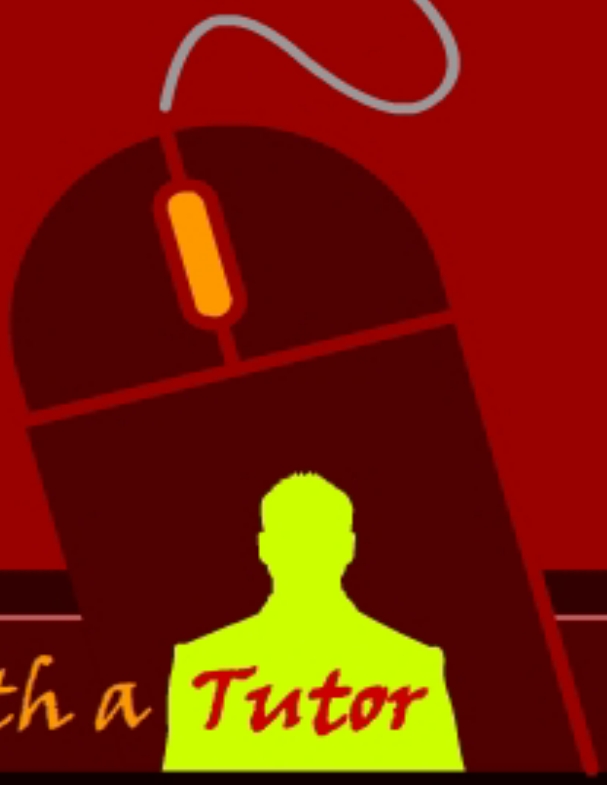
6. Building a prototype or a _____of the system may help to create and to clarify the architecture.

7. Questioning techniques are more mature than the measuring techniques. **[True/False]**

Tutor Chat

Onlinevarsity
your e-way to learning

are just a click away



To chat with a Tutor

login to www.onlinevarsity.com

Chapter

5 Quality Metrics

Objectives

At the end of this chapter, you will be able to:

- *Describe the various end-process quality metrics*
- *Describe the various In-process quality metrics*
- *Analyze Case Study*

5.1 Introduction

Software metrics can be categorized into three classes of product metrics, process metrics, and project metrics. In this chapter, we will describe various software quality metrics, which are a subset of software product, process and project metrics. These metrics focus specifically on the quality aspects.

Software quality metrics can be further sub-divided into end-product quality metrics and in-process quality metrics. The software quality engineering views quality from the entire software life cycle perspective. So, we should include metrics while discussing quality from the quality engineering perspective, which are concerned with both software products and process.

5.2 The end-process quality metrics

The description of software quality for a software product consists of two levels:

- Intrinsic product quality
- Customer satisfaction

We will study the following **quality metrics** in relation to the two levels of quality as described above:

- Mean Time To Failure (MTTF)
- Defect density
- Customer problems
- Customer satisfaction

5.2.1 Mean Time to Failure Metrics (MTTF)

The intrinsic product quality is measured in terms of the number of "**bugs**", also known as **functional defects**, in the software product or in terms of how long the software can run before encountering a "**crash**". The two required metrics for these are **defect density** (rate) and **mean time to failure** (MTTF). The MTTF metric is most often used in non-commercial fields, such as avionics, weapon systems, and so on. For example, the US government mandates that its new air traffic controller system cannot be unavailable for more than three seconds a year. The defect density metric is used in many commercial software systems.

The MTTF measures the time between failures, whereas the defect density measures the **defects** relative to the software size. The software size is expressed in Lines of code, function points, or any other standard methods. The terms **errors**, **failures**, **defects**, and **faults** are defined as:

- An **error** is a human mistake that results in incorrect software.
- The resulting **fault** is an accidental condition that causes a unit of the system to fail to function as required.
- A **defect** is an anomaly in a product.
- A **failure** occurs when a functional unit of a software-related system can no longer perform its function or cannot perform it within the specified limits.

(Reference: IEEE/ ANSI standard 982.2)

Let us discuss a few examples of how researchers and practitioners define the usage of the terminologies.

To many organizations, **errors** often mean **faults**. There is also a separate notion of "*processing error*", which defines the state of the system between the triggering of the fault and the actual failure of the system. This particular notion of error is highly relevant for software fault tolerance, which is concerned with how to prevent failures in the presence of processing errors.

Anomalies usually mean a class of faults that are unlikely to cause failures in themselves but may eventually cause failures indirectly. In this sense, an anomaly is a deviation from the usual, but it is not necessarily wrong. For example, deviations from accepted standards of good programming practice, such as use of non-meaningful names are often regarded as anomalies.

Defects normally refer collectively to faults and failures. However, sometimes a defect is a particular class of fault. For example, Mellor uses **defect** to refer to faults introduced prior to coding. [Mellor 1986] In the quality perspective, the term *defect* can be defined as deviations from the specifications or expectations, which might lead to failures in operations.

Bugs refer to faults occurring in the code.

Crashes are a special type of incident, where the system ceases to function.

Figure 5.1 illustrates how a human error finally leads to failure.

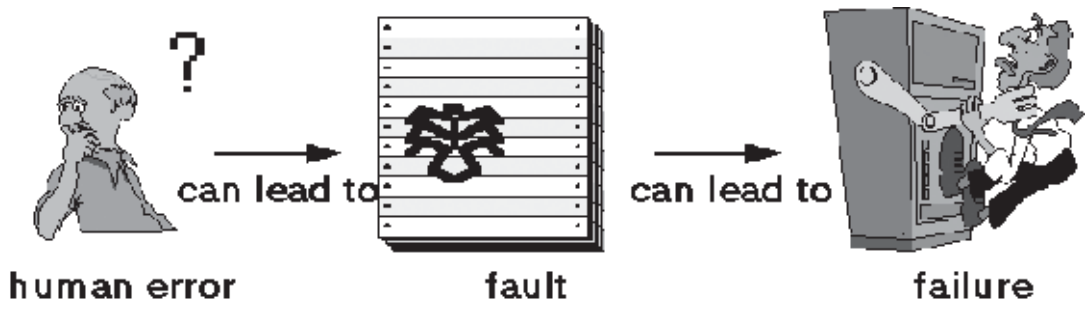


Figure 5.1: Illustrating how a human error finally leads to failure

When an error occurs, a fault or a defect is generated in the product, which ultimately leads to failure. Sometimes, a fault may cause more than one failure situations, and on the other hand, some faults may not materialize even after a long time; so, these remain latent. The defects that may cause higher failure rates are generally, found first and removed.

The “mean time between failures” (MTTF) data is very difficult to collect because it requires failure time for each recorded software failure. It is very difficult to record the time for all the failures observed during test or operation. In a useful metric, the time between failures has to be accurately recorded. This is perhaps the reason the MTTF metric is not widely used in the commercial organizations. The defect rate, which is the defect density, on the other hand has an appeal for the commercial developers. The defect rate of a product or the expected number of defects over a certain time period is important for the estimation of cost and resource in the maintenance phase of the software life cycle.

5.2.2 The defect density metrics

In software, the **defect rate** is usually defined as the number of defects per KLOC in a given time interval. The time interval is typically one year after the shipping of the product to the market or the entire life of the product. Studies have shown that usually more than 95% of the defects are found in the first four years of the life of the product (LOP). Hence, the IBM Rochester has defined the LOP as four years.

The defect rate is very difficult to measure because of the problem in precisely defining the Lines of Code (LOC) metric. There is no standard method for counting the LOC. The problem comes from the ambiguity of defining the LOC. In *Software Engineering Metrics and Models by Conte et al (1986)*, the LOC is defined as:

“A line of code is any line of program text that is not a comment or blank line, regardless of the number of statements or fragments of statements on the line. This specifically includes all lines containing program headers, declarations, and executable and non-executable statements”.

In Boem's book, *Software Engineering Economies (1981),* the LOC method counts the lines as physical count and includes executable lines, data definitions, and comments. In fact, there are several variations in counting the lines of code, according to Jones (1986):

- Count only executable lines
- Count executable lines plus data definitions
- Count executable lines, data definitions, and comments
- Count executables lines, data definitions, comments, and job control language
- Count lines as physical lines on an input screen
- Count lines as terminated by logical delimiters

The method that the IBM uses for LOC is to count source instructions including executable lines and data definitions but excluding comments and program prologues. So, it can be easily seen that there are various LOC counting methods.

When a software product is released for the first time, and a certain LOC counting method is followed, it is relatively easy to state its quality level. However, when another version of the same product with enhancements and bug fixes comes to the market, the situation becomes more complicated. Now, one will need to measure the quality of the entire product as well as the portion of the product that is new and changed. Following **tasks** needs to be carried out to calculate the defect rate for the new and changed code:

- **LOC count**: The entire software product, which is the previous version as well as the new and changed code, must be counted for each new release of the software.
- **Defect Tracking**: Defects must pinpoint to the release origin, the specific portion of the code that contains the defect as well the release number.

The defect rate of the entire product will always improve after each release due to bug fixes, but that of the new and changed code will not improve unless there is real improvement in the development process.

These tasks, are enabled by the practice of the '**modified flagging'**. When a new function is introduced or an enhancement is made, the new and changed lines of code are flagged with a specific **Identification Code** (ID) number through the use of comments. The ID is mapped with the requirements number, and generally placed in the program prologue. Therefore, any changes in the program can be linked to a certain requirement. Consider an example for better understanding. Suppose, new user functionality such as keyboard shortcuts is to be introduced after shipping of the product. The new requirement can be numbered as, say, 'Kbnew01'. The code required for making the new changes will have an ID number, say, 'Kbnew01'. The ID number should appear before the actual code in the comments section.

The new and changed LOC counts can also be obtained through the use of the '**delta-library**' method.

By comparing program modules in the original library with the new versions in the current release library, the LOC count can determine the amount of the new and changed code for the new release.

5.2.3 Customer Problems Metrics

From the customer's point of view, all problems they encounter while using the software product are the problems of the software product. The actual problems cannot only be in the product itself or product defects; it can also be usability problems, unclear documentation or information. The problems can also be duplicates of valid defects that indicate the defects for which fixes are available, but the customer is unaware of or even can be **user errors**. User errors generally occur when a user does not use the proper procedure of executing the software. These "**non-defect-oriented**" problems, together with the actual product defects, constitute the total problem domain of the software from the customer's perspective.

The problems metric is usually expressed in terms of **problems per user month (PUM)**:

PUM = Total problems that customers reported (true defects and non-defect-oriented problems) for a time-period/ Total number of license-months of the software during the period

where,

Number of license-months = Number of install licenses of the software X Number of months in the calculation period

Several approaches can be taken to achieve a low PUM:

- Improve the development process and reduce the product defects
- Reduce the non-defect-oriented problems by improving all aspects of the products, such as usability, documentation, customer education, and support
- Increase the sale, which denotes the number of installed licenses of the product

The first two approaches serve to reduce the numerator of the PUM metric, and the third increases the denominator. The result of any one of these courses of action is that the PUM metric will have a lower value.

Therefore, two metrics represent two perspectives of product quality. The customer problems metric can be regarded as an intermediate measurement between defects measurement and customer satisfaction. To reduce customer problems, one will have to reduce the functional defects in the products, and other factors such as usability, documentation, and **problem rediscovery**. The term 'problem rediscovery' means that the customer is not aware that the problem has been detected earlier, and the solution of which is probably already available. Similarly, to improve customer satisfaction, one will have to reduce defects and overall problems in the products, and manage other factors of broader scope such as timing and availability of the product, company image, services and total customer solutions. The scope of the

three metrics can be represented through Figure 5.2.

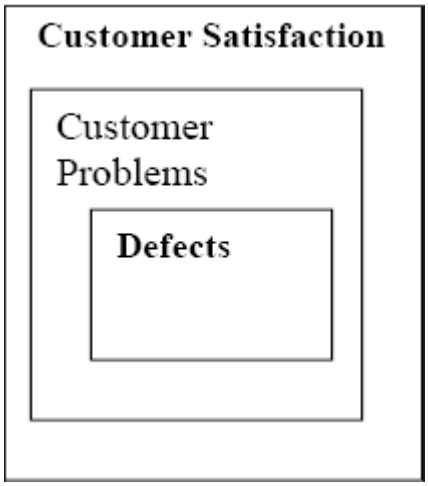


Figure 5.2: Scopes of the three software quality metrics

5.2.4 Customer Satisfaction Metrics

A customer survey generally takes care of this metric. The survey is done based on the following five-point scale:

- Very satisfied
- Satisfied
- Neutral
- Dissatisfied
- Very dissatisfied

Based on the five-point scale data, several metrics can be created:

- Percent of completely satisfied customers
- Percent of satisfied customers (satisfied and completely satisfied)
- Percent of dissatisfied customers (dissatisfied and completely dissatisfied)
- Percent of dissatisfied customers (neutral, dissatisfied, and completely dissatisfied)

Generally the second metric, **percent satisfaction** is used. Only in some cases where reducing the customer dissatisfaction is the goal, the metric number 4 is used.

The purpose of this survey is to gauge the level of customer satisfaction with the product. There are various methods of customer surveys based on criteria. For example, IBM follows CUPRIMDSO criteria (capability, usability, performance, reliability, installability, maintainability, documentation/ information, service, and overall). Similarly, Hewlett-Packard use FURPS (functionality, usability, reliability, performance, and supportability).

Let us discuss FURPS in detail. Hewlett-Packard developed a set of software quality factors that has been given the acronym FURPS – functionality, usability, reliability, performance, and supportability. The FURPS quality factors are described below:

Functionality is assessed by evaluating the feature set and capabilities of the program, the generality of the functions that are delivered, and the security of the overall system.

Usability is assessed by considering human factors, overall aesthetics, consistency, and documentation.

Reliability is evaluated by measuring the frequency and severity of failure, the accuracy of results, the Mean Time Between Failures (MTBF), the ability to recover from failure, and the predictability of the program.

Performance is measured by processing speed, response time, resource consumption, throughput, and efficiency.

Supportability is a combination of:

- The ability to extend the program includes **extensibility**, **adaptability**, and **serviceability**. These three terms are also associated with **maintainability**.
- **Testability**, **compatibility, configurability:** The ability to organize and control elements of the software configuration.
- The ease with which a system can be installed is **installability**
- The ease with which problems can be localized is **localizability**.

The FURPS quality factors and attributes, as described above, are useful to establish the quality metrics for each activity in the software process.

5.3 In-process quality metrics

Since, our goal is to comprehend and quantify the programming process and to learn how to engineer quality into the process, the in-process quality metrics play an important part. However, the in-process quality metrics are less formally defined compared to the end-product quality metrics. In-process quality metrics are defined as the tracking of defect arrival during the formal machine testing for organizations.

5.3.1 Defect Density during the formal machine testing

Defect rate during the **formal machine testing**, which refers to testing after code is integrated into the system library, has usually **positive correlation** with the defect rate experienced in the **field**. The positive correlation is a relation where the values of two variables increase or decrease together, as illustrated in Figure 5.3.

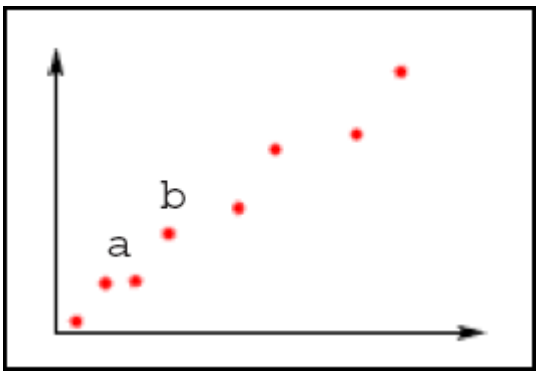


Figure 5.3: The positive correlation of values of two variables 'a' and 'b'

In case of higher defect rates, it can be concluded that the software product has experienced higher error injection during its development phase, unless of course, these are due to additional testing or a new and more effective testing approach.

The metric of **defects per KLOC** is a good indicator of quality while the software is still under testing. Following scenarios can be used to assess the release quality:

- If the defect rate during testing is same or lower than the previous release, then there can be two possibilities:
  - The quality perspective is positive.
  - Additional testing such as customer testing and stress testing will be necessary.
- If the defect rate during testing is higher than that of the previous release, then there can be the following possibilities:
  - The quality perspective is negative. In this case, more testing or a new and improved testing approach will be required as a remedial measure.
  - The quality perspective is positive or same.

5.3.2 Defect Arrival Pattern during the formal machine testing

The overall defect density during the testing phase is an overall indicator. The pattern of **defect arrivals** provides more information. The defect arrivals are actually the **reported problems**. Even with the same overall defect rate during test, different patterns of defect arrivals indicate different quality levels. Figure 5.4 shows a few samples of defect arrival patterns.

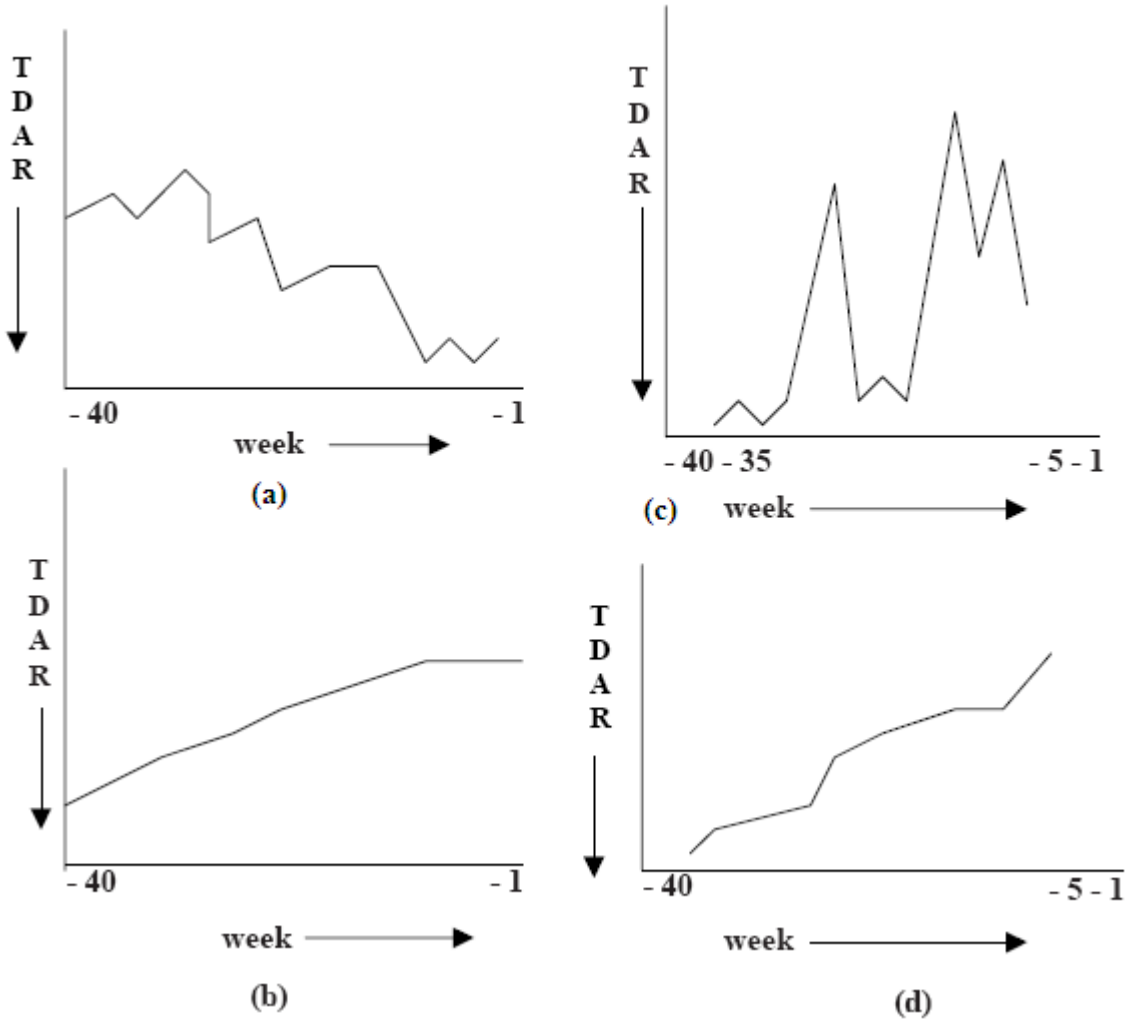


Figure 5.4: Various Test Defect Arrival Rate (TDAR) patterns during test

[Source: "Metrics and Models in Software Quality Engineering" by Dr. Stephen H. Kahn.]

Figure 5.4 shows two contrasting patterns for both the defect arrival rates during a sample test. Sample data were plotted from two contrasting patterns for both the defect arrival rates ((a) and (c)), and the cumulative defect rate ((b) and (d)). The left hand side of the patterns of Figure 5.4 (a) and (b) shows that the data were plotted after some weeks before the code-freeze (40 weeks, shown as -40) until the week before code-freeze (shown as –1). The second pattern of Figure 5.4 (c) and (d) indicates that the testing started late (35 weeks before the code freeze approximately), the test suite was not sufficient (the number of defects are substantial in the second case compared to the first one) and that the testing ended prematurely (5 weeks before the code freeze). These are only sample and imaginary data used to explain the concept of defect arrival patterns.

The main objective of checking the defect arrival pattern is to look for defect arrivals that are stabilized at a very low level, or the time interval between failures becoming very large, before ending the testing and releasing the product. The time unit for observing the arrival pattern is usually in weeks and sometimes in months.

There are three **metrics** useful for the defect arrival pattern during test:

- The **defect arrivals** - The defects reported during the testing phase by time interval. The time interval is usually in week. Out of all the defect arrivals, not all defects are valid.

- The pattern of **valid defect intervals -** The problem determination is done on the reported problems. This is the true defect pattern.

- The pattern of **defect backlog over time -** This metric is necessary because development organizations cannot investigate and correct all the reported problems instantaneously. The metric is indicative of the workload as well as quality. If the defect backlog is large at the end of the development cycle and lot of corrections have to be integrated into the system, the stability of the system and as a result its quality will be affected in the long run. Retesting, which refers to the regression testing, will be necessary to ensure that product quality levels are reached.

5.3.3 Phase-based Defect Removal Pattern

This pattern is basically an extension of the test defect density metric. In addition to testing for the defect rate pattern over a time period, it requires the tracking of defects at all phases of the development cycle including design reviews, code inspections or formal verifications before testing. It is found that a large proportion of programming defects are related to design problems. Conducting formal reviews or functional verifications to enhance the defect removal capability of the process at the previous phase help reduce the error injection to the next phase. This pattern reflects the overall defect removal ability of the development process.

Many organizations also employ several other metrics such as inspection coverage and inspection effort for the in-process quality management.

5.3.4 Defect Removal effectiveness

The **defect removal effectiveness** (DRE) can be defined as follows:

$$DRE = \frac{\text{Defects removed during a development phase}}{\text{Latent defects in the product}} \times 100\%$$

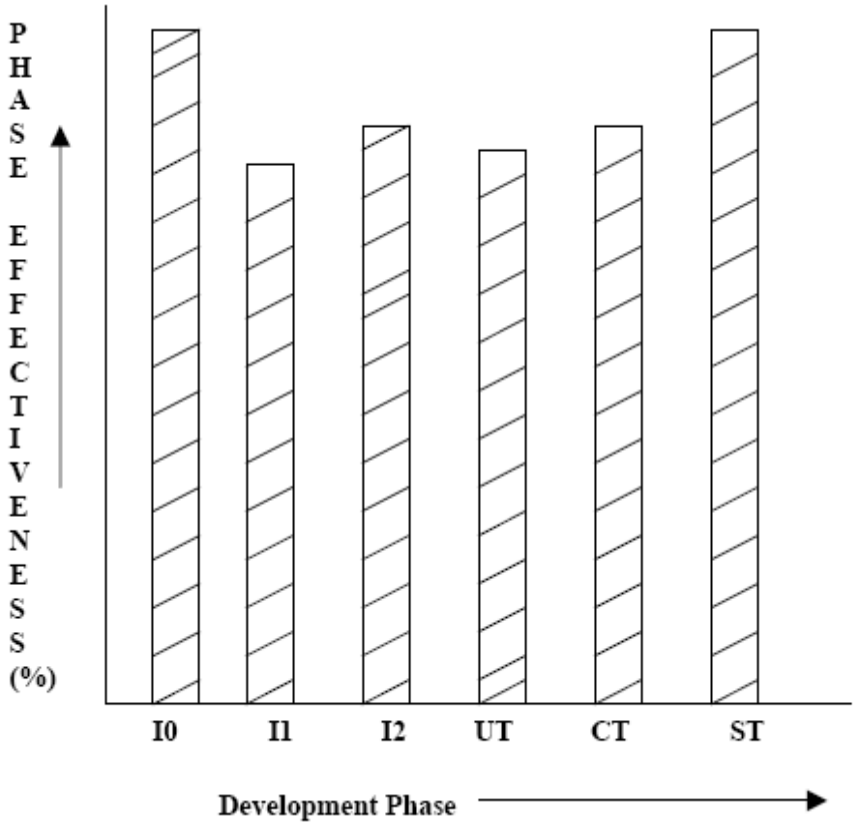


Figure 5.5: The phase effectiveness of a software process

[Source: "Metrics and Models in Software Quality Engineering" by Dr. Stephen H. Kahn.]

Since the total number of latent defects in the product at any given point of time is not known, the denominator of the metric can only be approximated. This is typically estimated by:

Defects removed during a phase + defects found later

This metric can be calculated for the entire development process, for the front-end, which is before the code integration, and for each phase. These are called **early defect removal** and **phase effectiveness** respectively. The higher the value of the metric, the more effective is the development process and as a result, fewer defects are injected to the next phase. This metric is a key concept of the defect removal model for software development. Figure 5.5 shows various phases of defect removal, which are depicted

as high-level design review (I0), low-level design review (I1), code inspections (I2), unit test (UT), component test (CT), and system test (ST). The figure shows the DRE by phase for a software project. The weakest phases are unit test (UT), low-level design review (I1), code inspections (I2), and component test (CT). Based on this metric, action plans can be drawn to improve the effectiveness of these phases.

The data presented above for each phase are only sample data, and presented for the explanatory purpose only.

5.4 Case study – Motorola's QPSD Program

The example given below shows how various metrics programs are carried out in Motorola.

[*Source: Daskalantonakis, M.K., "A Practical View of Software Measurement and Implementation Experiences Within Motorola", as appeared in "Metrics and Models in Software Quality Engineering" by Dr. Stephen H. Kahn.*]

After following the **Goal/ Question/ Metric model** of Basili and Weiss (1984), goals were identified, questions were formulated in quantifiable terms and metrics were established. The goals and measurement areas identified by the **Motorola Quality Policy for Software Development** (QPSD) are listed below:

Goals:

- Improve project planning
- Increase **defect containment** (defect containment has been explained as follows)
- Increase software reliability
- Decrease software defect density
- Improve customer service
- Reduce the cost of nonconformance
- Increase software productivity

Measurement areas:

- Delivered defects and delivered defects per size
- Total effectiveness throughout the process
- Adherence to schedule
- Estimation accuracy
- Number of open customer problems
- Time that problems remain open

- Cost of nonconformance
- Software reliability

For each goal, the questions to be asked and the corresponding metrics were also formulated.

Goal 1: Improve Project Planning

Question 1.1: What was the accuracy of estimating the actual value of project schedule?

Metric 1.1: Schedule Estimation Accuracy (SEA)

$$SEA = \frac{\text{Actual project duration}}{\text{Estimated project duration}}$$

Question 1.2: What was the accuracy of estimating the actual value of project effort?

Metric 1.2: Effort Estimation Accuracy (EEA)

$$EEA = \frac{\text{Actual project effort}}{\text{Estimated project effort}}$$

Goal 2: Increase Defect Containment

An explanation of the term '**defect containment**' is being included here before proceeding with the questions. Due to the large size and high complexity of most software systems in use today, the defect reduction activities can greatly reduce the number of faults but it cannot completely eliminate these. Hence, some residual faults will usually remain. For software systems where failure impact is substantial, such as many real-time control software used in medical, nuclear, transportation, and other embedded systems, this low defect level and failure risk may still not be adequate. Some additional QA alternatives are needed.

On the other hand, these few remaining faults may be triggered under rare conditions or unusual dynamic scenarios, making it unrealistic to try to generate the huge number of test cases to cover all these conditions or to perform exhaustive inspection or analysis based on all possible scenarios. Instead, some other means must be used to prevent failures by breaking the causal relations between these faults and the resulting failures, thus "tolerating" these faults, or to **contain** the failures to reduce the resulting damage.

Question 2.1: What is the currently known effectiveness of the defect detection process prior to the release?

Metric 2.1: Total Defect Containment Effectiveness (TDCE)

TDCE = (Number of prerelease defects)/(Number of prerelease defects + Number of post-release defects)

Question 2.2: What is the currently known containment effectiveness of faults introduced during each constructive phase of software development phase for a particular software product?

Metric 2.2: Phase Containment Effectiveness for the ith phase (PCE_i)

PCE_i = (Number of phase i errors) / (Number of phase i errors + Number of phase i defects)

where, the ith phase denotes any phase between 1 to n, 'n' being any number.

Goal 3: Increase Software Reliability

Question 3.1: What is the rate of software failures, and how does it change over time?

Metric 3.1: Failure Rate (FR)

FR = (Number of failures) / (Execution time)

The **execution time** is the duration the software is in use.

Goal 4: Decrease Software Defect Density

Question 4.1: What is the normalized number of in-process faults, and how does it compare with the number of in-process defects?

Metric 4.1a: In-process Faults (IPF)

IPF = (In-process faults caused by incremental software development) / (Assembly-equivalent delta source size)

Typically, the Lines of Code (LOC) is expressed as the number of lines written in a high-level language, such as Fortran. However, each high-level instruction, in turn, generates several equivalent assembly language instructions. Therefore, for example, for 5 lines of code (expressed as 'delta') out of the total 100 lines, the generated assembly instructions can be 20, which is the assembly-equivalent delta source size.

Metric 4.1b: In-process Defects (IPD)

IPF = (In-process defects caused by incremental software development) / (Assembly-equivalent delta source size)

Question 4.2: What is the currently known defect content of software delivered to customers, normalized by assembly-equivalent size?

Metric 4.2a: Total Released Defects total (TRD total)

TRD total = (Number of released defects) / (Assembly-equivalent Total source size)

Metric 4.2b: Total Released Defects delta (TRD delta)

TRD delta = (Number of released defects caused by incremental software development) / (Assembly-equivalent total source size)

Question 4.3: What is the currently known customer-found defect content of software delivered to customers, normalized by assembly-equivalent source size?

Metric 4.3a: Customer-Found Defects total (CFD total)

CFD total = (Number of customer-found defects) / (Assembly - Equivalent total source size)

Metric 4.3b: Customer-Found Defects delta (CFD delta)

CFD delta = (Number of released defects caused by incremental software development)/(Assembly-equivalent total source size)

Goal 5: Improve customer service

Question 5.1: What is the number of new problems that were opened during the month?

Metrics 5.1: New Open Problems (NOP)

NOP = Total new post-release problems opened during the month

Question 5.2: What is the total number of open problems at the end of the month?

Metric 5.2: Total Open problems (TOP)

TOP = Total new post-release problems that remain open at the end of the month

Question 5.3: What is the mean age of open problems at the end of the month?

Metric 5.3: Mean Age of Open Problems (AOP)

AOP = (Total time the post-release problems remain open at the end of the month) / (Number of open post-release problems remaining open at the end of the month)

Question 5.4: What is the mean age of the problems that were closed during the month?

Metric 5.4: Mean Age of Closed problems (ACP)

ACP = (Total time post-release problems closed within the month these were open)/(Number of open post-release problems closed within the month)

Goal 6: Reduce the cost of Nonconformance

Question 6.1: What was the cost to fix post-release problems during the month?

Metric 6.1: Cost of Fixing Problems (CFP)

CFP = Dollar cost associated with fixing post-release problems within the month

Goal 7: Increase Software Productivity

Question 7.1: What was the productivity of software development projects (based on the source size)?

Metric 7.1a: Software Productivity total (SP total)

SP Total = (Assembly-equivalent total source size) / (Software development effort)

Metric 7.1b: Software Productivity delta (SP delta)

SP delta = (Assembly-equivalent delta source size) / (Software development effort)

SP Total is the productivity for entire source code size, whereas the term '**SP delta**' is for a part of the entire source code.

In reference to the preceding discussions, we can observe that metrics 3.1, 4.2a, 4.2b, 4.3a, and 4.3b are end-product quality metrics; metrics 5.1 to 5.4 are software maintenance metrics, and metrics 2.1, 2.2, 4.1a and 4.2b are the in-process quality metrics. Other metrics are for the scheduling, estimation, and productivity. Items 1 to 4 are for project status/ control, and 5 to 7 are actually in-process quality metrics that can provide valuable information about the status of the project and for the possible actions that can be taken for further quality improvements.

SUMMARY

- Software metrics can be classified into three categories of product, process and project.
- Software quality metrics can be further categorized into end-product and in-process quality metrics.
- The MTTF measures the time between failures, whereas the defect density measures the defects relative to the software size.
- Defect rate is usually defined as the number of defects per KLOC in a given time interval.
- User errors generally occur when a user does not use the proper procedure of executing the software.
- The purpose of Customer survey is to gauge the level of customer satisfaction with the product.
- The metric of defects per KLOC is a good indicator of quality while the software is still under testing.
- The main objective of checking the defect arrival pattern is to look for defect arrivals that are stabilized at a very low level, or the time interval between failures becoming very large, before ending the testing and releasing the product.
- The **defect removal effectiveness** (DRE) can be defined as follows:

$$DRE = \frac{\text{Defects removed during a development phase}}{\text{Latent defects in the product}} \times 100\%$$

CHECK YOUR PROGRESS

1. MTTF is an in-process metric.
   [True/False]

2. Defect density is used in commercial organizations.
   [True/False]

3. Customer problems is one of the end-process metrics.
   [True/False]

4. The task of finding defect rates is enabled by 'change flagging' practices.
   [True/False]

5. The customer problems metric is typically expressed in PUM.
   [True/False]

6. FURPS is a measure of the customer satisfaction.
   [True/False]

7. Functionality is assessed by evaluating the feature set and capabilities of a program.
   [True/False]

8. The term ___ is measured by processing speed, response time, resource consumption, throughput and efficiency.

9. The metric of defects per KLOC is a good measure of quality while the product is still being tested.
   [True/False]

10. The ____ property determines the ease with which problems can be localized.
    [True/False]

Chapter 6

Quality Standards

Objectives

At the end of this chapter, you will be able to:

- *Describe various quality standards*
  - *ISO 9000*
  - *SEI-CMM*
  - *Malcolm Baldrige Assessment*

6.1 Introduction

Any organization implements its own set of process models or frameworks for producing products. The degree (that is, to what extent) to which this process will be implemented varies from one organization to another, sometimes even from one project to another. The success of a project primarily depends on the implementation maturity of a process. In addition to the process model, there are issues related to the overall quality management system of an organization, which is one of the most important parameters that determine the fate of the projects.

In this Session, we shall discuss some of the popular process frameworks that are used to assess the process maturity of an organization or a project. We shall cover Software Engineering Institute (SEI) process maturity assessment methods, the Malcolm Baldrige assessment method, and ISO 9000 registration process. The first framework is especially concerned with software, the other two are actually quality process and management standards that apply to all industries.

6.2 ISO 9000

ISO 9000 is a set of standards and guidelines for a quality assurance management system, and represents a body of quality standards, *International Organization for Standardization*. There are **twenty** elements in ISO guidelines. While obtaining the necessary registration, a formal audit, called '**external audit**', is involved and an organization appearing for the ISO 9000 certification needs to qualify in that audit. The necessary guidelines for the implementation of the twenty elements towards the development, supply, and maintenance of software are also specified in ISO 9000-3. ISO 9003 is a set of guidelines for implementing an ISO 9001 compliant set of processes.

The twenty elements are:

1. Management responsibility
2. Quality system
3. Contract review
4. Design control
5. Document control
6. Purchasing
7. Purchase-supplier product
8. Product identification and traceability
9. Process control
10. Inspection and testing
11. Inspection, measuring and test equipment
12. Inspection and test status
13. Control of non-conforming product
14. Corrective action
15. Handling, storage, packaging, and delivery
16. Quality records
17. Internal quality audits
18. Training
19. Servicing
20. Statistical techniques

It has been observed that most common occurrences of non-conformances are in the area of corrective actions and the documentation control. The implementation of the defect prevention process is an appropriate mechanism to address the corrective actions. It is important that the mechanism be applied throughout the entire organization. If an organization does not implement the defect prevention process, at least a process for corrective action must be established.

The ISO 9000 series of standards is comprised of individual, yet related, standards for quality management and quality assurance. Only ISO 9001 and 9000-3 have been discussed in this Session.

6.2.1 ISO 9001

ISO 9001's focus is directed at an organization's internal processes and methods and at managing at the external level, which refers to controlling, assuring, and the quality of products and services delivered.

When viewing the major factors affecting the outcome of the software development, ISO 9001's focus is on all the factors except "**technology**".

- **Scope of ISO 9001**

  ISO 9001 is a generic international standard, and written for use by the widest possible audience. As a result, the standard provides *requirements* such as *what* needs to be done and does not issue specific *prescriptive* solutions such as *how* to do it.

  Being so broadly focused, the ISO 9001 standard does not offer details about its application to specific domains of expertise. While providing help in the application of the standard for specific domains, a series of guidelines are available, for example, ISO 9000-3 is a guideline for the software development industry.

6.2.2 ISO 9000-3

ISO 9000-3 provides the "**guidance**" on implementing an ISO 9001 compliant set of processes, collectively referred to as a "**quality system**" or as a "**quality management system**".

"ISO 9000-3 is a guidance for the software development, supply and maintenance environments. The guideline is primarily written for the contract driven software markets. It can easily be adapted for other market needs such as commercial-off-the-shelf (COTS), internal software development, and so on.

ISO 9000-3 mirrors the provision of ISO 9001- it does not add to, or otherwise change, the requirements of ISO 9001.

ISO 9000-3 is not to be used as an internal/external audit tool. The purpose of this is to guide software organizations with their ISO 9001 implementation and process change efforts. In short, software organizations are audited against ISO 9001 (not ISO 9000-3)".

(Source: http://www.scc.ca/iso9000)

6.3 The SEI process Capability Maturity Model (SEI-CMM)

The Software Engineering Institute (SEI) of Carnegie-Melon University developed a process framework for the software development. This framework (SEI SW-CMM) includes **five levels** of the process maturity.

Level 1: Initial

The characteristic of this level is:

Chaotic: Unpredictable cost, schedule, and quality performance.

Level 2: Repeatable

The characteristics at this level are:

Intuitive: The cost and quality highly variable, reasonable control of schedule and *ad hoc* methods and procedures. The key elements for achieving level 2 maturity are:

- Requirements management
- Software project planning and oversight
- Software sub-contractor management
- Software quality assurance
- Software configuration management

Level 3: Defined

The characteristic of this level is:

Qualitative: The reliable costs and schedules, improving but unpredictable performance. The key elements to achieve this level are:

- Organizational process improvements
- Organizational process definition
- Training program
- Integrated software management
- Software product management
- Software product engineering
- Inter-group coordination
- Peer reviews

Level 4: Managed

The characteristic of this level is:

Quantitative: The reasonable statistical control over product quality.

The key elements for this level are:

- Process measurement
- Quality management

Level 5: Optimizing

The characteristic of this level is:

Quantitative: The basis for continued capital investment in process automation and improvement. The key elements for this level are:

- Defect prevention
- Technology innovation
- Process change management

"The SEI maturity assessment framework is being used by various agencies in the software industry. The framework is to be used with an **assessment methodology**. The assessment methodology relies on an 85-item questionnaire. In support of each question, the SEI maturity level is indicated. There are also specially designated questions as key to each specific maturity level. When qualified for a certain level, 90% of the key questions and 80% of all questions for that level must be answered as '**yes**'. Moreover, maturity levels are *hierarchical* in nature. For example, Level 2 must be attained before calculation for Level 3. If an organization has more than one projects, its ranking is determined by answering the questionnaire. Moreover, the answer to each of these questions must be substantially true across the whole organization.

It is found that at the level 4, there is an extensive use of metrics and models. At level 5, the element of defect prevention is the key. The SEI maturity assessment model has been used as an evaluation mechanism for contract software by the US department of Defense. According to Humphrey (*American Programmer, 1990*), among the assessment conducted by the SEI, about 75% of the projects have been observed to be at level 1. There are only a few level 4 and 5 projects".

(*Source: Metrics and Models in Software Quality Engineering by Dr. Philips Kahn.*)

6.4 The Malcolm Baldrige Assessment

The Malcolm Baldrige National Quality Award (MBNQA) is one of the most prestigious quality awards in the United States. It was established in 1988 by the US Department of Commerce, and named after Malcolm Baldrige, the then Secretary of that department. The award is given annually to recognize the US companies that excel in the quality management and the quality achievement. The assessment consists of **seven** categories:

- Leadership
- Information and analysis
- Strategic quality planning
- Human resource utilization
- Quality assurance of products and services
- Quality results
- Customer satisfaction

The system for scoring the assessment items is based on three evaluation dimensions of **approach**, **deployment** and **results**. **Approach** refers to the methods the company is using to achieve the purposes addressed in the examination item. **Deployment** refers to the extent to which the approach is applied. **Results** refer to the outcomes and effects of achieving the purposes addressed and applied.

"There are **thousand points** available for the 'Baldrige criteria for performance excellence in education, business and health'. The criteria also form the basis for the Malcolm Baldrige National Quality Award process. Each assessment item is given a percentage score (ranging from 0% to 100%). A candidate for the Baldrige award should score in the 70% range. A candidate organization going for this assessment should score in the 70% range. This award is achieved in the following way:

- For an approach assessment item, continuous refinement of approaches is in place and a majority of them are linked to each other.
- For a deployment assessment item, deployment has reached all the company's major business areas as well as many support areas.
- For a results assessment item, the company's results in many of its major areas are among the highest in the industry. There should also be evidence that the results are caused by the approach.

Score is important, but equally important is the **feedback**. The feedback consists of the observed strengths and most importantly, the areas for improvement. By eliminating the high priority weaknesses, an organization can be assured of continuous improvement.

Four elements of the framework of the award criteria, which are evident to make a MBNQA winner include:

- **Driver**: Depicted by the leadership of the senior executive management team. The *Leadership* element of MBNQA drives the system by:

- Setting direction
- Creating values, goals and systems
- Guiding the pursuit of customer value and company performance improvement

➢ **System**: The set of well-defined and well-designed processes for meeting the company's quality and performance requirements. Composed of processes for meeting the company's customer and performance requirements. Process elements include:

- Information and Analysis
- Strategic Planning
- Human Resource Development and Management
- Process Management

➢ **Measure of progress**: The results of the company's in-process quality measurements. These measures provide a results-orient basis for channeling actions towards delivering ever-improving customer value and company performance. These measures include:

- Product and service quality
- Productivity improvement
- Waste Reduction/ Elimination
- Supplier Performance
- Financial Results

➢ **Goal**: The basic aim of the quality process is the delivery of continuously improving value to customers. These goals include:

- Customer satisfaction
- Customer satisfaction relative to competitors
- Customer retention
- Market Share Gain

(Source: http://www.tantara.ab.ca)

The **purpose** of this assessment approach are given as follows:

- Help elevate the quality standards in an organization
- Facilitate the communication and sharing among and within organizations of all types based on a common understanding of key quality requirement.
- Serve as a working tool for planning, training, assessment, and other uses
- Provide the basis for winning the award
- Provide feedback to the applicants

IBM has taken the MBNQA assessment discipline as a key component of its total quality management program, the **'Market Driven Quality Strategy and Implementation'**.

The Malcolm Baldrige assessment is not just limited to the software organizations, projects and processes; it concerns itself with the overall quality management, regardless of industries. The MBNQA covers three broad categories of **manufacturing**, **service** and **small business**.

The MBNQA criteria and the ISO 9000 quality assurance system can complement each other in an organization's endeavor for the achievement of quality. However, there are some differences between the two as well. The MBNQA assessment is a *non-prescriptive* assessment tool that focuses on process and results, while the ISO 9000 requires passing an audit. Moreover, while the ISO 9000 concerns itself with a quality management system and process control, it does not examine the quality results and customer satisfaction, towards which the MBNQA assessment is heavily inclined.

The two standards including the MBNQA criteria and the ISO 9000 form a *synergy* on an organization's quest for total quality management:

- The formal ISO documentation requirements such as quality records, which facilitate addressing the Baldrige examination items.
- The formal ISO validation requirements such as internal assessments, external audits, and periodic surveillance, which assist the completeness and the thoroughness of the processes.
- The heavy ISO emphasis on the corrective action, which contributes to the company's continuous improvement program.
- The audit process itself, which results in additional focus on many of the Baldrige examination areas.

6.4.1 Comparison among ISO 9001, SEI-CMM and MBNQA assessment

Each standard and model emphasizes on the different aspects of Total Quality Management. As shown in the chart below, all standards and models have an overwhelming focus on *process management*. Michael Tingey studied the standards and the three process models. He discovered that "the MBNQA

model puts a higher emphasis on *improving* and then on *planning*. In comparison, the ISO 9001 standard puts its higher emphasis on *managing* and then on *implementing*. The SEI SW-CMM model is the just reverse of ISO 9001-its largest emphasis is on *implementing* and then *managing"* ("*Comparing ISO 9000, Malcolm Baldridge and SEI/CMM for Software"* by M. Tingey).Figure 6.1 shows a comparison of the three process frameworks.

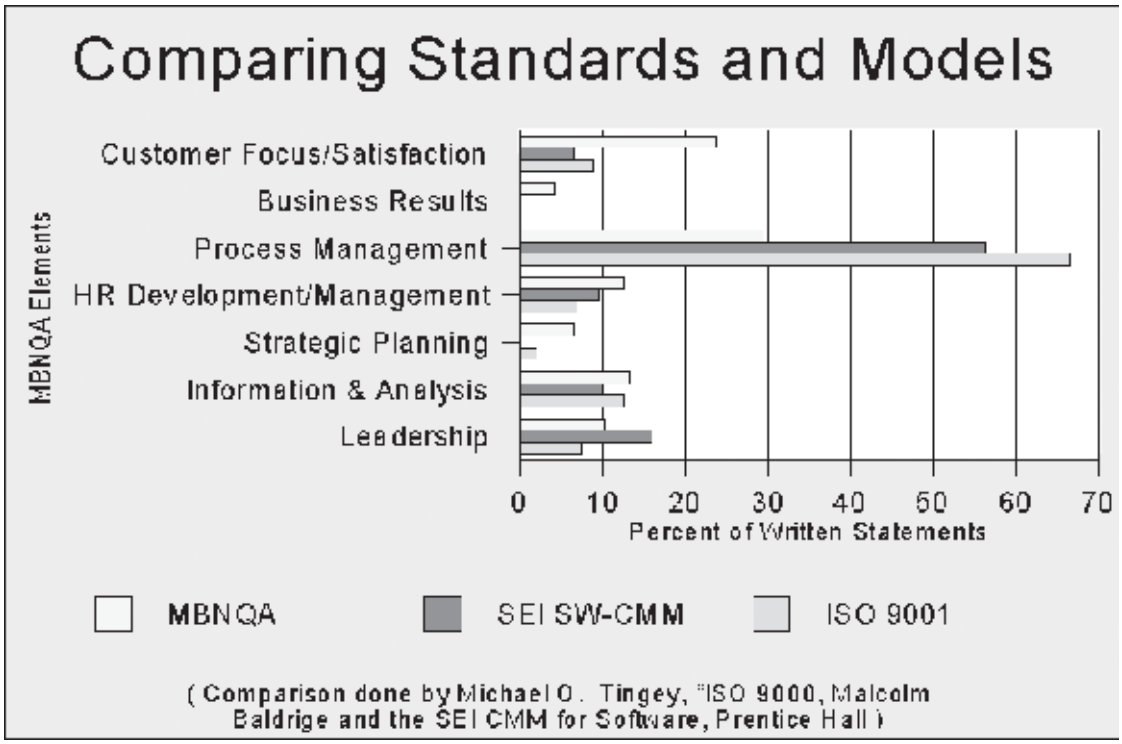


Figure 6.1: Comparison of the three process frameworks

SUMMARY

- Any organization implements its own set of process models or frameworks for developing its products.
- The success of a project or projects depends heavily on the implementation maturity of a process.
- ISO 9000 is a set of standards and guidelines for a quality assurance management system.
- ISO 9000-3 is a set of guidelines for implementation of ISO 9001.
- The framework SEI SW-CMM includes five levels of the process maturity.
- The award MBNQA is given annually to recognize the US companies that excel in the quality management and the quality achievement.
- Four elements, which are evident to make a MBNQA winner include driver, system, measure of progress and goal of the framework of the award criteria.

CHECK YOUR PROGRESS

1. The ISO 9000 is a series of standards. **[True/False]**

2. The success of a project or projects depends heavily on the implementation maturity of a process. **[True/False]**

3. ISO 9000 is a set of standards and guidelines for a quality assurance management system. **[True/False]**

4. ISO stands for _____.

5. ISO ____ provides "guidance" for implementing an ISO 9001 compliant set of processes.

6. The SEI SW-CMM framework includes four levels of the process maturity. **[True/False]**

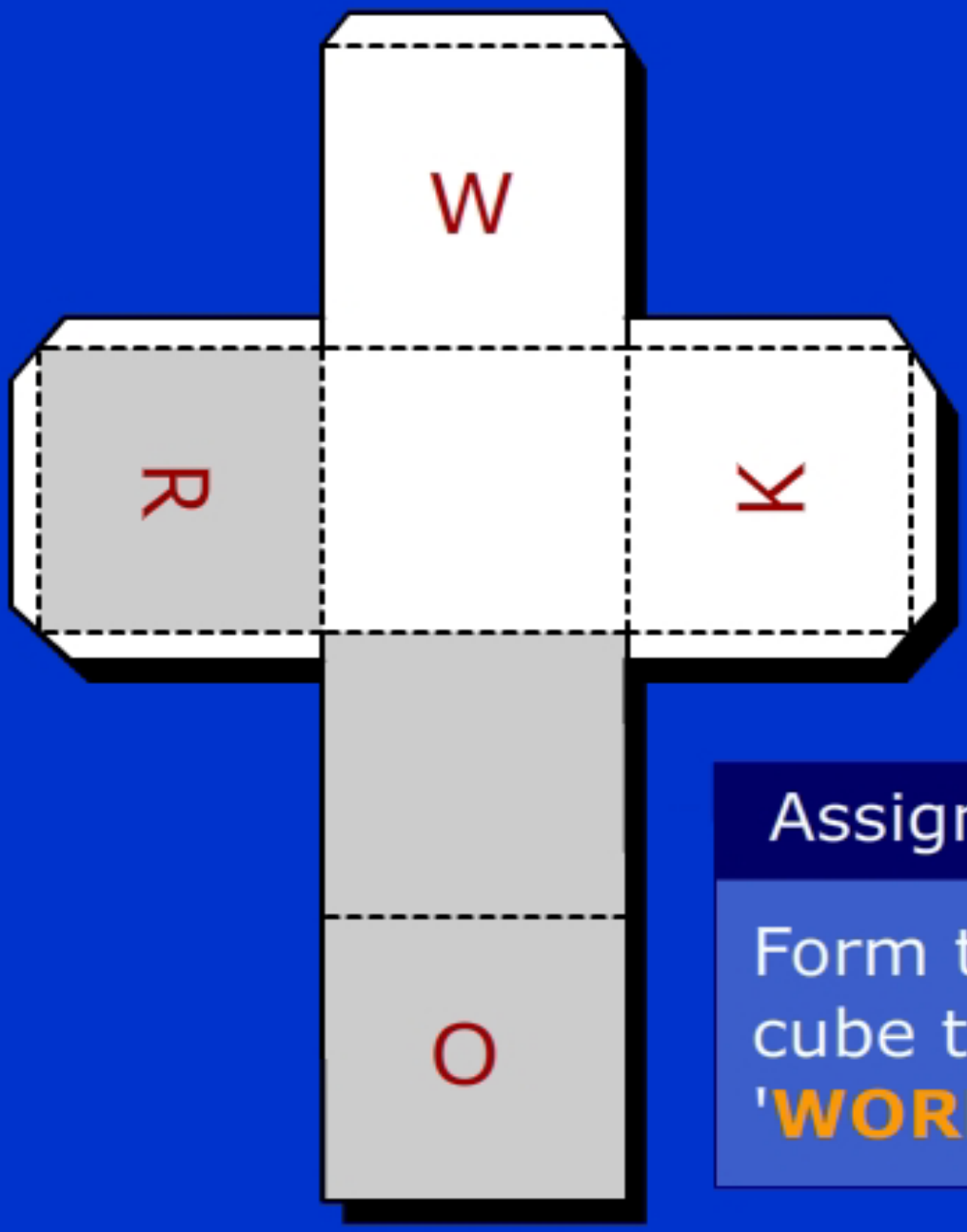
7. The MBNQA deals with the overall quality management. **[True/False]**

8. In MBNQA, the feedback system is one of the most important components. **[True/False]**

9. The MBNQA assessment is a *prescriptive* assessment tool. **[True/False]**

WORK ASSIGNMENT

Onlinevarsity
your e-way to learning



Assignment

Form the cube to read '**WORK**'.

"Practice does not make perfect. Only perfect practice makes perfect."
- Vince Lombardi

For perfection, solve assignments @

www.onlinevarsity.com

Chapter

7 Case Study

Objectives

At the end of this chapter, you will be able to:

- *Analyze different case studies*

7.1 Introduction

Many software organizations are working actively to improve their product and process quality. We can review their implicit and explicit management activities and ask:

- Which activities have had a positive impact on process and improvement on product quality?
- Which activities have had a negative impact?

We can then develop a list of typical problems and solutions to those problems.

7.2 Case Studies

One company (called as ObjectWorld) is a manufacturer of hardware and software designed to make object-oriented programming easier. Another company (PhoneHome) is a manufacturer of telecommunications equipment, designed for use in the commercial sector by the Regional Bell Operating Companies (RBOC). The last company (ViewIt) is a manufacturer of remote sensing equipment designed for the commercial sector. These companies have similar features:

- Demanding sophisticated customers
- Quick development cycle to make additional features promptly available
- The needs to publicly announce ship dates and then live up to those dates

7.2.1 Case study – I

ObjectWorld realized that continuing its strategy of proprietary hardware to sell its software was untenable for the long term. It started to remake itself into primarily a software company with less reliance on internally developed hardware. As this strategy evolved, the software products became even more complex, and were difficult to develop and debug. The company started spending

more money and time on supporting product already in the field. The company, thus, had a difficult time planning and implementing project plans for new products. As a result, the software engineering organization spent less focused time on developing new products. The software engineers recognized that the processes were not sufficient to deal with the problem, and that other ways must be determined to bring the problems under control (refer Figure 7.1).

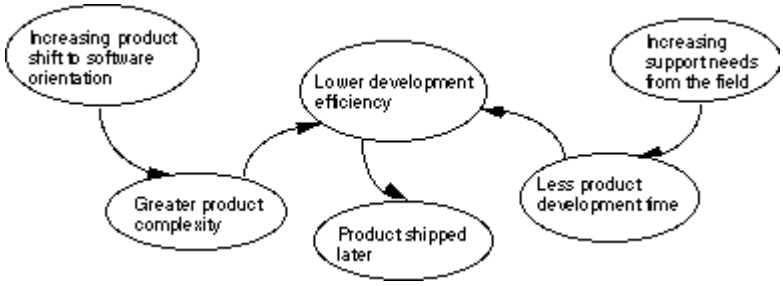


Figure 7.1: ObjectWorld Product Development Problems

There were three levels of “interested employees” at ObjectWorld: Senior management (President and VPs); middle management (Directors and Senior technical contributors); and the technical contributors (including first line managers). First line managers made technical contributions.

Senior Management took what they thought were reasonable steps; they put a number of managerial controls into place, told the development staff to develop better products, and started to refine a long-term competitive strategy. One of the managerial controls was a biweekly “Operating Committee” meeting. Every two weeks, all Director level managers would present the status reports to other directors and the senior management. Resources could be channeled to projects in trouble.

Unfortunately, a number of unintentional events occurred as follows::

1. The meetings took more time to prepare for and attend, so that the middle manager in charge was taking time away from the project and spending it on status generation and reporting.

2. The meetings were not well run, so there were discussions at the meetings that were inappropriate (too technical, not relevant to all attendees, and the like).

3. Too much detail at the senior level led to requests for more detail at the middle level, which of course, led to more requests at the technical contributor level.

After a short time, both middle and senior management realized what was happening and stopped it. Senior management specified tighter agendas for these meetings and middle managers started tracking their projects very closely and presenting only the project information at the Operating Committee meetings. This slowed the drain on product development resources. Middle management proposed reengineering the software development process. Senior management did not understand why the process should be decomposed and reengineered, but they trusted middle management’s recommendation. Therefore, senior management did not resist the reengineering proposal.

The software process definition was addressed in three parallel steps:

1. How to define what the product was supposed to be, and get the customer requirements?
2. How to develop software and do initial testing?
3. How to do final system test, beta test, and respond to bugs in the field?

Different middle managers took responsibility for each of the three steps.

There was a very active user group for the company, so there was substantial input by the current users for new features of the product. ObjectWorld did not understand the need or how to determine customer requirements from future potential customers; hence, part of the first process was not implemented.

The software engineering staff agreed on a number of milestones (feature freeze, code freeze, beta freeze), and a number of accepted technical practices (code review of all code generated after code freeze, code review of all code in particularly difficult parts of the system to implement). Middle management and the engineering staff also agreed on how to develop schedules (iteratively), and how to publish them (broadly). There was a formal beta test process defined and successfully used for a number of years. The software manufacturing process had extremely well defined entry and exit criteria. Readiness reviews, for reviewing the number and type of outstanding bugs in each area of the software were held before beta and general releases. System testing was highly automated, and some developers worked as hard on test code development as they had on product code development.

As a result of these processes, the number of major bugs declined before the software was released, and the number of people required in software support declined over time.

The process changes became part of the culture of the software engineering organization. The technical contributors became responsible for integrating the process changes into the culture. They began to refuse project plans that were not based on the process.

ObjectWorld increased its product reliability and decreased the number of defects in its products. Customers and the company agreed that the software improved in quality.

Activity for student: Explain the positive and negative practices followed by Object World

Solution to the activity:

ObjectWorld Positive practices

1. Management did not try to solve the problems of the engineering staff. Management defined the problems, as it perceived them, but not the solutions. The engineering staff had the responsibility and capability to find and implement appropriate solutions.

2. Project planning was more defined, with specific milestones that every project had to meet.

3. Dates for re-planning were built into the schedule (iterative scheduling).

4. Each project's schedule was published to the entire company, along with an integrated schedule highlighting all major milestones.

5. Instituting code reviews at particular milestones in the project enabled the engineers to find bugs as cheaply and quickly as possible.

6. The end game, the final test and release part of the process, was extremely well defined. If a project made it to the entry criteria for the end game, it was obvious how long it would take to ship.

7. The process was completely integrated into the company culture. It was integrated by choice (complete agreement by the technical staff), not by fiat from management.

ObjectWorld Negative practices

1. Insufficient requirements generation made for fuzzy project plans at the beginning. In turn, less capable developers who were not able to understand how their pieces fit into the whole project.

2. The notion of critical path planning was not well integrated into the process.

ObjectWorld Conclusions

1. Management did not delude them into thinking that everything was fine when it was not. Both technical contributors and management agreed that problems existed. Each set of people solved the problems they knew best how to solve.

2. Technical buy-in at the technical contributor level was the most useful component of process improvement changes made by ObjectWorld. Company culture can be a very powerful force, especially when used for process, project, and product improvement.

7.2.2 Case study – II

PhoneHome had a very successful product introduction, and by the early 1990's, they were in the midst of the negative feedback shown in Figure 7.2. Additional product features were required to satisfy the market PhoneHome. It was difficult to find adequate resources to verify the product features throughout the lifecycle and hence, the product was shipped prematurely.

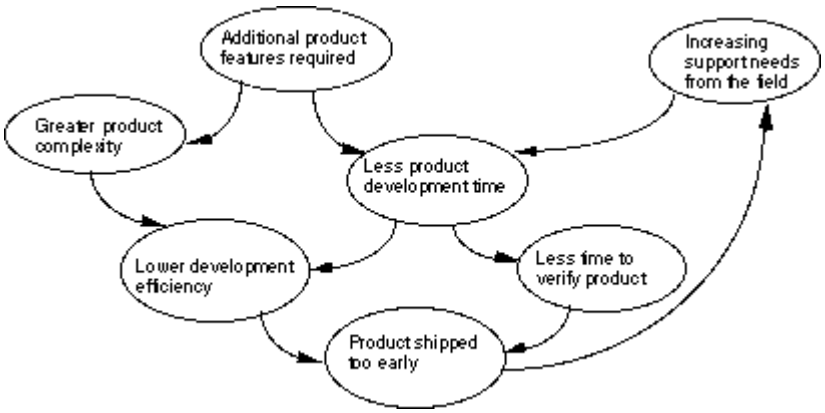


Figure 7.2: PhoneHome Product Development Problems

PhoneHome started spending more money and time on supporting product already in the field. There were insufficient field people to support the current products; so engineering staff was frequently called on to support the products. At one time, there was an estimate that 40% of all Software Engineering product development time (development, documentation, SQA activities) was spent on support of products in the field, in addition to the technical support staff.

In addition, PhoneHome had a customer perception problem. The major customers (RBOC representatives) were quite willing and eager to work with PhoneHome representatives to define features for the product. At the same time, a major customer who was demanding the new features and early delivery was also concerned with the quality of the delivered products. In particular, some products and features were quite reliable, but a substantial minority of products and features were not sufficiently reliable. This major customer had encountered this problem and adopted TQM practices to solve it. The customer demanded that PhoneHome adopt appropriate management practices to solve this problem. The customer would then start to audit PhoneHome's process to verify the changed product development process.

PhoneHome also had a difficult time planning and implementing project plans for new products. As originally happened with ObjectWorld, the software engineering organization spent substantially less focused time on developing new products (refer Figure 7.1). There were few management controls on the production of software products. PhoneHome had just undergone tremendous personnel growth in the Engineering area, increasing in staff from 15 people to 50 and then to 100 people. The previous culture including project management, was oriented around 5-15 people working on related components of a major project. For relatively small projects, the culture (informal reviews, informal planning, informal verification) worked quite well. This culture could not sustain a 50-100-person organization working on many major projects. There was insufficient communication among the technical contributors and middle management. One of the results for the lack of communication was inadequate risk analysis of product state.

There were the same three levels of "interested employees" as at ObjectWorld: senior management (President and VPs); middle management (Directors and Senior technical contributors); and the technical contributors and first line managers. Many of the first line managers also made technical contributions.

Case Study

Near the end of one product development cycle, the auditors requested a process audit. A process audit reviews the process documentation and project evidence that the process was followed. It does not specifically review a particular product's quality.

The first process audit had a number of non-conformances in the areas of managerial control, and system testing. Managerial control refers to project phase definition of a project, entry and exit criteria for each phase, and how these phases interact with each other. For example, an initial planning phase would have specific entry and exit criteria for how to enter and leave the phase, and list activities related to how to plan a project, verify the plan, and monitor the plan.

Engineering management decided to write a Process Manual for phase definition and control. The auditors recommended this step. There were a number of good reasons for writing the manual; the customer wanted the company to do so, and Engineering needed a way to unite all the new employees from different cultures into one culture. Some negative issues regarding company culture and practices surfaced during the writing of the manual:

1. The technical contributors wrote less than 10% of the manual. The technical contributors did not have an opportunity to discuss the process issues involved in the manual. The Software Engineering Directors wrote the contents of the manual. The amount of technical contributor involvement was directly proportional to the immediate culture change.

2. The process chosen by the Software Engineering directors reflected what the company said was the corporate goal, but did not reflect how the company actually made decisions. A slightly modified waterfall model was chosen as the one and only development process. In reality, a spiral model would have met the needs of the customers and Engineering more for large system development, and a concurrent engineering model would have met needs for patch and very small system development.

A few positive things came out of the manual writing:

1. The appropriate use of the software configuration management system was defined.

2. Instructions for how to create the product for shipment were defined.

3. Specific entry and exit criteria for customer acceptance testing were defined.

4. System testing became more creative and had better coverage than in previous releases. PhoneHome increased the number of bugs found during system test as compared to the number of bugs found at acceptance testing or customer sites.

As a result of these process changes,

1. The number of bug's known before the customer can accept, was higher. PhoneHome found the bugs, not the customer.

2. Engineering knew the count of open bugs, and undertook appropriate risk analysis.

3. The number of major bugs declined before the software was released. The software was more reliable.

PhoneHome did not undertake other high advantage activities. For example, PhoneHome did not use iterative scheduling techniques and broad schedule publication. Project schedules were extremely aggressive and difficult to achieve. Senior management had a concern specifying that broad internal publication of the schedule would be potentially damaging to the company in case of schedule slips. Damage would come in the form of “leaking” the internal schedule to the customers and having to reset the customers’ expectations.

During this time, PhoneHome’s senior management was promoting TQM ways of thinking and working, but did not internalize TQM ways of working. Instead of posing problems to project leaders or managers, senior management frequently imposed solutions. There was a substantial amount of short-term thinking and action. One piece of evidence of this short-term thinking was the use of voicemail for asking people to work on “critical” or “urgent” activities, only to discover after one had finished the work that the product of that work was neither critical nor urgent. There were also many poorly run, lengthy meetings that were unrelated to product development.

The original process changes at PhoneHome did not become part of the culture of how the software engineering organization worked. After more audits and technical contributor rewrites of the process manual, the process started to become integrated into the culture. For example, people state their problems with another group instead of a solution. The process is not seamlessly integrated into the company culture. The company is still working to balance short-term needs with long term needs in its daily activities.

Activity for student: Explain the positive and negative practices followed by PhoneHome

Solution to the activity:

PhoneHome Positive practices

Over time, PhoneHome increased its product reliability and decreased the real number of defects in its products. Customers and the company agreed that the software improved in quality. The positive key practices that PhoneHome implemented are as follows:

1. Defined a process that worked for large, customer-cooperative, complex system development. This process included entry and exit criteria for each phase, and code reviews for all new code. It also included bug tracking within each phase and within each project (bugs found per phase per product release).

2. The back end of the process, from customer acceptance through shipment, was extremely well defined. If a project made it to the entry criteria for customer acceptance, it was clear how long the acceptance testing and final debugging would take.

3. Major customers were consulted at many points during the software development process.

PhoneHome Negative practices

The negative key practices of PhoneHome:

1. Management stated solutions instead of stating the problems. This was a symptom of all management, with the most dramatic examples starting at the top of the company. This was partly due to lack of goal setting by senior management.

2. Not taking the current company culture into account when it was time to develop a new culture. The new process was defined by fiat, certainly not a way to guarantee acceptance into the organization.

3. Assuming that a strict definition of one software process was sufficient for all the Software Engineering responsibilities.

PhoneHome Conclusions

1. Management never asked: “Is there a problem with how we do things around here?” If middle or senior management had asked the question, the project and process problems would have surfaced earlier.

2. When a company is not focused on their business (product development and support), it is easy to let other interests take precedence over the business. For example, the supposed TQM practices involved many committees and extracurricular activities, both of which required substantial planning. Too often, managers who were supposed to be focused on products were focused on activities that were tangential to product development. Integration of TQM practices into a company starts with the definition of company and product goals, and then by acting on these goals. This has the double benefit of keeping the company focused on the products and showing all employees how to use the TQM practices in daily work.

3. Technical contributors had many valuable comments about process improvement. Technical contributors developed all the valuable components of the very first process manual. For process improvement to work, the technical contributors must be a part of the problem definition and solution.

7.2.3 Case Study – III

In the early-1990’s, ViewIt experienced the typical software engineering crisis. There were too many features to put into its legacy software, not enough time to do it, and there were too many bugs uncovered during system testing.

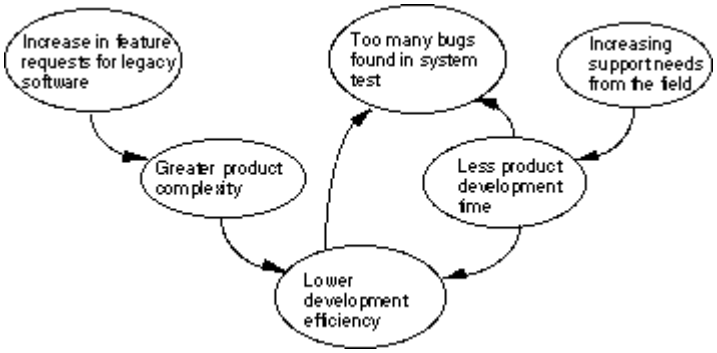


Figure 7.3: ViewIt Product Development Problems

ViewIt had already started spending more money and time on supporting product already in the field. There were insufficient field people to support the current products, so engineering staff were frequently called on to support the products (refer Figure 7.3). At one time, there was an estimate that 20% of all Software Engineering product development time (development, documentation, quality assurance) was spent in support of products in the field, in addition to the technical support staff.

The company also had a difficult time planning and implementing project plans for new products. The tendency was to develop project plans for the beginning of the project, and not update them as the project was in process. Product shipment decisions were based on how the software “felt” to the SQA personnel, rather than by analysis of objective metrics. As with ObjectWorld, the software engineering organization spent substantially less focused time on developing new products. Software Engineers changed roles frequently; either they were pulled off development projects to work on field-generated crises, or as projects changed importance in the minds of senior management, project resources (people and machines) were reallocated.

There were only two levels of “interested employee” at ViewIt: senior management (President and VPs), and the rest of the company. ViewIt has a very controlling, strictly hierarchical organization. Only the Vice Presidents or the President truly have the flexibility to make resource tradeoff decisions; all other decisions including those of project leaders, can be reversed by other senior management. Senior management has a very short-term vision; they believe that if they make all the correct decisions for the short term, the long term will take care of itself.

ViewIt has a technological approach to some of the problems of software engineering; they bought and implemented a reasonable bug-tracking tool and an excellent configuration management system. On some projects, the technical contributors insisted on writing unit tests and doing code reviews. When management insisted code review and unit test time be eliminated to make a particular schedule, the engineers agreed but received management commitment that these practices would be instituted again once the deliverable was complete.

As ObjectWorld and PhoneHome, ViewIt has not yet had a significant catalyst for change. There are a number of middle managers and individual contributors who would prefer to work differently. There are two tensions for employees: balance of short-term gains against long-term engineering investment; and the balance of work against other’s lives. ViewIt has technically challenging work, extremely aggressive schedules, and a young work force. In the past, people have worked whenever necessary to complete a

project. As more technical contributors take on added responsibility outside work, such as additional education or families, they are less likely to work 12-16 hours per day for long periods of time.

Technical contributors demanded changing practices. The effects of those demands are as follows:

1. Project management practices are changing. Project leaders now update their project plans on a regular basis.

2. The SQA group gathers metrics for readiness reviews to determine the ship risk based on the state of the code instead of how the product “feels”.

3. The Engineering staff adopted a concurrent engineering methodology to reduce product development time.

4. Most projects use code reviews to decrease the number of bugs found during system test.

5. Most projects develop unit tests along with the code to reduce the time required for system test.

The effect of these changes has been to increase development efficiency during product development, reduce product development time, and enable to predict when a product will be ready to ship. Quality and changing quality practices are defined by the software engineers and engineering management, and presented to senior management. As the company grew, senior management also required a process change: project managers are expected to be able to describe the complete state of their projects at any time. As a result of these process changes,

1. Projects tend to keep their resources for the entire duration of the project, unless the project is cancelled.

2. Product ship decisions are based on objective criteria.

3. The state of the software is understood (example: bugs).

4. For almost every project, the number of open major bugs has declined before the software was released.

Activity for student: Explain the positive and negative practices followed by ViewIt

Solution to the activity:

ViewIt Positive practices

Over time, ViewIt increased its product reliability and decreased the real number of defects in its products. Customers and the company agreed that the software improved in quality. The positive key practices that ViewIt implemented:

1. Defined an objective ship-review process based on objective criteria.

2. Defined a planning process for projects and informal reviews, which illuminate the effects of resource tradeoff decisions.

3. Defined and obtained sufficient technological tools to increase software engineering productivity.

4. Implemented code reviews and unit test development to discover defects earlier in the project.

ViewIt Negative practices

1. Management states solutions instead of stating the problems. Senior management still makes all the resource decisions. They do not always understand the data they have from the project leaders. Even though senior management claims to make decisions based on the short-term, frequently the short-term interests are not served by their decisions.

2. Management's lack of interest in a formal process undermines its stated desire to ship more products on time. There is substantial talk about "quality", but no definition of what it means. Management's definition appears to be "ship products that are good enough on time". The rest of the company's definition is "ship products that have very few bugs ASAP". The key difference is in the short-term versus long-term thinking. Senior management has a very short-term approach because they clearly see the immediate financial issues. The technical contributors have a very long-term approach because they see the support issues and longer term financial issues clearly. Either approach is fine, but the lack of consistency leads to strange and non-optimal product and ship decisions.

ViewIt Conclusions

1. Senior management is very focused on the business (product development and support). However, they have not fully articulated the goals of any of the products, specifically the quality goals. Senior management thinks the technical contributors are happy, but the technical contributors are frustrated with their jobs and the work they produce.

2. Technical contributors are driving the process improvement work. The practices are integrated into the company culture.

Onlinevarsity

your e-way to learning

BlOg

Balanced Learner-Oriented Guide

for enriched learning available @

www.onlinevarsity.com

Software Verification, Validation, and Testing

Chapter

1 Introduction to Verification and Validation

Objectives

At the end of this chapter, you will be able to:

- *Specify the software requirements*
- *Describe the formal Specification Techniques*

1.1 Introduction

The evolution of software that satisfies its user expectations is a necessary goal of a software development organization. In order to achieve this goal, various software engineering practices have to be applied throughout the different stages of software development. The objectives of these practices are to create and modify a software product in such a way so as to satisfy the customer expectations. These practices are collectively known as "*software verification and validation (V&V)*" practice.

The validation technique refers to the process of evaluating a software product at the end of its development stage to ensure that it is free from *failures* and complies with the requirements. A failure is defined as an incorrect product behavior. Most of the time the validation is achieved through various testing approaches. Other intermediate products can also be validated; for example, the *design specifications* can be evaluated.

The verification technique refers to the process of determining whether the products of a given phase of a software development process fulfill the requirements established. *Software technical reviews* represent one such method for verifying various products. A specification review will attempt to verify the specification requirements against the requirements description and the customer requirements specifications. *Proof of correctness* is another such technique for verifying programs to formal specifications. Verification techniques try to identify product *faults* or *errors*, which give rise to failures.

In this chapter, we will discuss about the software requirements specifications, the formal specification techniques, and reliability. Other techniques for verification and validation will be studied in the later chapters.

1.2 Software Requirements Specifications (SRS)

The *software requirements specification* is a technical specification of requirements for a software product. The goal of this specification is to completely and consistently specify the technical

requirements for a software product in an unambiguous and concise way by using formal notations. A document is created which gives details of the SRS.

The format of a sample SRS document has been presented in Table 1.1.

Section Number	Description
Section 1	Product Overview and Summary
Section 2	Development, Operating, and Maintenance Environments
Section 3	External Interfaces and Data flow
Section 4	Functional Requirements
Section 5	Performance Requirements
Section 6	Exception Handling
Section 7	Early Subsets and Implementations Priorities
Section 8	Foreseeable Modifications and Enhancements
Section 9	Acceptance Criteria
Section 10	Design Hints and Guidelines
Section 11	Cross-reference Index
Section 12	Glossary of terms used in the SRS

Table 1.1: A sample SRS document

Brief description of the various sections of the SRS document:

- **Sections 1** and **2** present an overview of product features and then summarize the processing environments for development, operation, and maintenance of the product.

- **Section 3** include the user displays and report formats, a summary of various user commands and reports options, data flow diagrams and a data dictionary.

- **Section 4** specifies the functional requirements for a software product.

- **Section 5** contains the performance criteria, for example, the response time for various activities, such as processing time for various processes, processor throughput (number of instructions executed per second), memory constraints, required bandwidth and security considerations.

- **Section 6** contains the various exception conditions, including the actions to be taken in exception conditions and the messages to be displayed. A table of exception conditions as well as reason for exception conditions is to be prepared.

- **Section 7** specifies the early subsets and implementations priorities for the system under development. This is required considering that a customer may require early delivery of subset capabilities, and the successive versions can provide increasing levels of capabilities.

- Expected modifications and enhancements that may be incorporated into the product following the earlier release are included in **section 8**. It is always important that already known probable changes be identified and incorporated into the product requirements.

- The software product acceptance criteria are specified in **section 9** of the requirements document. Acceptance criteria specify the functional and performance tests that must be carried out, and the standards to be applied to the source code, internal documentation, and external documentation (example: design specifications), the test plan, user manual, operational principles, and the installation and maintenance procedures.

- **Section 10** contains various design hints and guidelines for the product.

- **Section 11** maps the product requirements to the sources of information used in obtaining the requirements. A cross-referential directory should be provided as an index to the specific paragraphs in the Systems Definition and initial user's manual, and to any other sources of information.

- **Section 12** provides definitions of terms that might be unfamiliar to the customer and the product developers.

1.2.1 The expected properties of the Software Requirements Specifications

There are a number of expected properties that an SRS document must possess. An SRS document must be:

- Correct
- Complete
- Consistent
- Unambiguous
- Functional
- Verifiable
- Traceable
- Easily changeable

An incorrect or incomplete set of requirements can result in a software product that satisfies its requirements but does not satisfy customer needs. An inconsistent specification describes contradictory requirements in different parts of the document, while an unclear requirement is subject to different interpretations by different users of the document.

Requirements must be functional in nature. These should state *what* is required without implying *how* the system will meet its requirements. This in turn provides maximum flexibility to the developers.

Requirements must be verifiable from the two points: it must be possible to verify that the requirements satisfy the customer's need, and it must also be possible to verify that the subsequent products satisfy the requirements.

Lastly, the requirements should be indexed, segmented and cross-referenced to allow the easy use and modifications. Every software requirement should be traceable to specific customer statements and to the specific statements in the system definition. Changes occur in a project quite frequently, and the project success depends on the ability to incorporate the necessary changes. An SRS document must be modifiable to incorporate those frequent changes. Cross-referring can be achieved by referring to the appropriate paragraph numbers in the appropriate documents.

1.2.2 Verification of the specifications

The correctness of an application does not necessarily imply that the functions performed by the application exactly match the customer requirements. Even if the implementation ultimately satisfies the specification, we may still end up with a product, which does not satisfy the customer expectations. This happens when the customer requirements were not understood properly, and as a result the requirements specifications were drawn incorrectly.

There are two general methods for *verifying* a specification. The first one consists of observing the *dynamic behavior* of the specified system to check whether it matches the understanding we had of the behavior for the ideal system. The other consists of *analyzing the properties* of the specified system. These properties can be deduced from the specifications. The properties that have been deduced are then checked against the expected properties of the system.

The effectiveness of both the methods increases with the *degree of formality* of the specification. The observation of the dynamic system behavior can also be called '*simulation*'. It is obtained by executing the formal specification. This in turn leads to the prototype of the specified system, before any actual implementation has actually started. Similarly, property analysis can be carried out by *human inspection*, especially if the specifications are not totally formal. The formal specification has been discussed in the next topic.

1.3 The formal specifications techniques

During the requirement analysis specifying the functional characteristics of a software product is one of the most important activities to be carried out. Formal notations have the advantage of

being concise and unambiguous. These notations can be gainfully used to specify the user requirements.

Relational and **state-oriented** notations are used to specify the functional characteristics of a software product. Relational notations are based on the concepts of entities and attributes. Entities are *named elements* in a system; the names are chosen to represent the nature of elements, for example, a *stack*. These functions and relations are applied to the named entities specifying the attributes. Attributes specify permitted operations on entities, relationships among entities, and the data flowing between entities.

The state of a system is the information necessary to summarize the status of system entities at any given instant of time. The state-oriented notations are used for this purpose. Given the current state and the current stimulus, the next state can be determined.

Relational notations include **implicit equations**, **recurrence relations**, **algebraic axioms**, and **regular expressions**. State-oriented notations include **decision tables**, **event tables**, **transition tables**, **finite-state mechanisms**, and **Petri nets**. All these notations are formal because these are precise and unambiguous. Only some of these notations are discussed in this chapter.

1.3.1 Relational Notations

Different types of relational notations:

a) **Implicit equations:** Implicit equations describe the properties of a solution without describing how the solution will be obtained. For example, let us consider the implicit specification of a square root function, SQRT, which can be defined as:

```
(0<= A <=B)  [ABS(SQRT(A)**2-A)<E]
```

The above equation states that for all real values of A within the closed range between 0 and B as shown in the relation (0<= A <=B), the absolute value of the expression remains within the permissible error limit, E, as shown in the expression (ABS(SQRT(A)**2-A)<E).

b) **Recurrence Relations:** A recurrence relation contains an initial part called the **basis** and one or more recursive parts. The recursive part describes the desired value of a function in terms of other values of the function. For example, in a Fibonacci series successive terms can be obtained with two initial terms: first and second terms.

```
F1(0) = 0
F2(0) = 1

F1(T) = F1(T-1) + F1(T-2) for all T>1
```

Recurrence relations can easily be transformed into recursive programs, although it does not necessarily mean that all recurrence relations will have to be implemented as recursive programs.

c) **Algebraic axioms**: The axioms specify the fundamental properties of a system. These also provide a basis for deriving additional properties. These additional properties are known as **theorems**. In order to establish a valid mathematical system, the set of axioms must be complete and consistent, which means that it must be possible to prove desired results using the axioms, and it must be possible to prove contradictory results do not exist.

The axiomatic approach can be used to specify functional properties of software systems. The following example shows how this approach can be used to describe the Last In First Out (LIFO) principle in a stack.

The definitions that will be required to explain the LIFO principle of stack:

NEW	Creates a new stack
PUSH	Adds a new item to the top of a stack
TOP	Returns a copy of the top item
POP	Removes the top item
EMPTY	Tests for an empty stack
ITEM	Data element to be manipulated

SYNTAX:

OPERATION	DOMAIN	RANGE
NEW	( ) ->	STACK
PUSH	(STACK, ITEM)->	STACK
POP	(STACK)	STACK
TOP	(STACK)	ITEM
EMPTY	(STACK)	BOOLEAN

AXIOMS:

(**stk** is of type STACK, **itm** is of type ITEM)

1) EMPTY(NEW) = true
2) EMPTY(PUSH(stk,itm)) = false
3) POP(NEW) = error

4) TOP(NEW) = error

5) POP(PUSH(stk, itm)) = stk

6) TOP(PUSH(stk,itm)) = itm

In simple English, the axioms can be stated as follows:

1. A new stack is empty.
2. A stack is not empty immediately after pushing an item onto it.
3. Attempting to pop a new stack results in an error.
4. There is no top item on a new stack.
5. Pushing an item onto a stack and immediately popping it off leaves the stack unchanged.
6. Pushing an item onto a stack and immediately requesting the top item returns the item just pushed onto the stack.

So, it is obvious that the axioms convey the same meaning as in plain English, but are more concise and unambiguous.

1.3.2 State-Oriented Notations

Various state-oriented notations are as follows.

(a) **Decision tables**: Provide a mechanism for specifying complex decision logic. It is a formal, table-based notation, which can also be automatically processed to check for qualities, such as *completeness* and *clarity*. A decision table has two parts. The top part lists the different conditions and the bottom part specifies different actions. The table specifies under what combination of conditions what actions are to be taken. For example, let us consider the part of a banking system that is responsible for debiting from accounts. Considering this part, the relevant conditions are:

AC1: The account number is correct

AC2: The signature matches

AC3: There is enough money in the account.

The possible actions are:

A1: Give money

A2: Give statement that insufficient money is present

A3: Call the vigilance department to check for fraud.

The decision table is shown in Table 1.2:

	1	2	3	4	5
AC1	N	N	Y	Y	Y
AC2		N	N	Y	Y
AC3			N	Y	N
A1				X	
A2			X		X
A3		X			

Table 1.2: A sample decision table

The decision rules are the different combinations of conditions. For every condition, a 'Y' means 'yes' or 'true', and a 'N' means 'no' or 'false'. If an action is to be taken for a particular combination of the conditions, it is shown by an 'X' for that particular action. If there is no mark present in the table, it means that the action is not to be performed.

(b) **Transition table**: These are used to specify the changes in the state of a system as a function of changes. Depending on the current state and the current conditions, the next state results. Table 1.3 shows a transition table. It illustrates the format of a simple transition diagram, given that with the current state **S1** and current input '**y'**, the system will go to the state **S0**.

Current State	**Current Input**	
	x	**y**
S0	S0	S1
S1	S1	S0

Next State ⟶

Table 1.3: A transition table

The table illustrates that if the current state of the system is **S0** and the current input '**y**', then the state of the system changes to **S1**. Similarly, if the current state is **S1** and the current input is '**y**', then the state of the system changes to **S0**.

SUMMARY

- The validation technique refers to the process of evaluating a software product at the end of its development stage to ensure that it is free from *failures* and complies with the requirements.
- The verification technique refers to the process of determining whether the products of a given phase of a software development process fulfill the requirements established.
- The software requirements specification is a technical specification of requirements for a software product.
- An SRS document should be correct, complete, consistent, unambiguous, functional, verifiable, traceable and easily changeable.
- Formal notations have the advantage of being concise and unambiguous.
- Relational and state-oriented notations are used to specify the functional characteristics of a software product.

CHECK YOUR PROGRESS

1. The validation technique refers to the process of evaluating a software product at the end of its development stage. **[True/False]**

2. ___ is the probability of the operational success of the software.

3. _____ techniques try to identify product *faults* or *errors*, which give rise to failures.

4. _____ is a technique for verifying programs to formal specifications.

5. The _____ is a technical specification of requirements for a software product.

6. Section ____ of the SRS maps the product requirements to the sources of information used in obtaining the requirements.

7. The software _____ criteria are specified in section 9 of the Software Requirements Specifications.

8. State-oriented notations are used to specify the functional characteristics of a software product. **[True/False]**

9. Downtime is defined in terms of system failure. **[True/False]**

10. Implicit equations describe the properties of a solution without stating a solution method. **[True/False]**

Chapter

2 Verification and Validation

Objectives

At the end of this chapter, you will be able to:

- *Learn to prepare a test strategy*
- *Learn to prepare a test plan*
- *Study to carry out walkthroughs, inspections and reviews*
- *Perform the static analysis*
- *Perform validations by the symbolic execution*

2.1 Introduction

Testing is one of most important activities in verification and validation. However, before describing various testing procedures, it is necessary to reiterate the "*rules of the road*" or "*truisms*" about testing:

- The number of possible inputs is very large
- The number of possible outputs is very large
- The number of paths through the software is very large
- The software specification is subjective. We can also say that a bug is in the eye of the beholder.

These rules will be apparent as we go through the discussion on the testing procedures in detail.

2.2 Preparing a test strategy

Designing effective test cases is important, but so is the strategy that will be used to execute them. The testing strategy can be formulated based on the following considerations:

- Should we develop a formal plan for the tests?
- Should we test the entire program as a whole or run tests only on a small part of it?

- Should we rerun tests we have already conducted as we add new components to a large system?
- When should we involve the customer?

2.3 Preparing a test plan

The test plan is an important, but often overlooked product of the software design. A test plan is a prescription of activities that will be performed to demonstrate that a software product meets its requirements. The test plan specifies the following:

- **Objectives of testing**: To achieve error-free operation under given conditions for a given amount of time.
- **Test completion criteria**: To achieve a specified rate of error exposure and to achieve a specified percent of logical path coverage.
- **System integration plan**: It includes strategy, schedule and responsible individuals.
- **Methods to be used on particular modules**: These include walkthroughs, inspections, static analysis, dynamic tests and formal verification.
- **Particular test cases to be used**

The testing phase goes in a cycle that is shown in Figure 2.1.

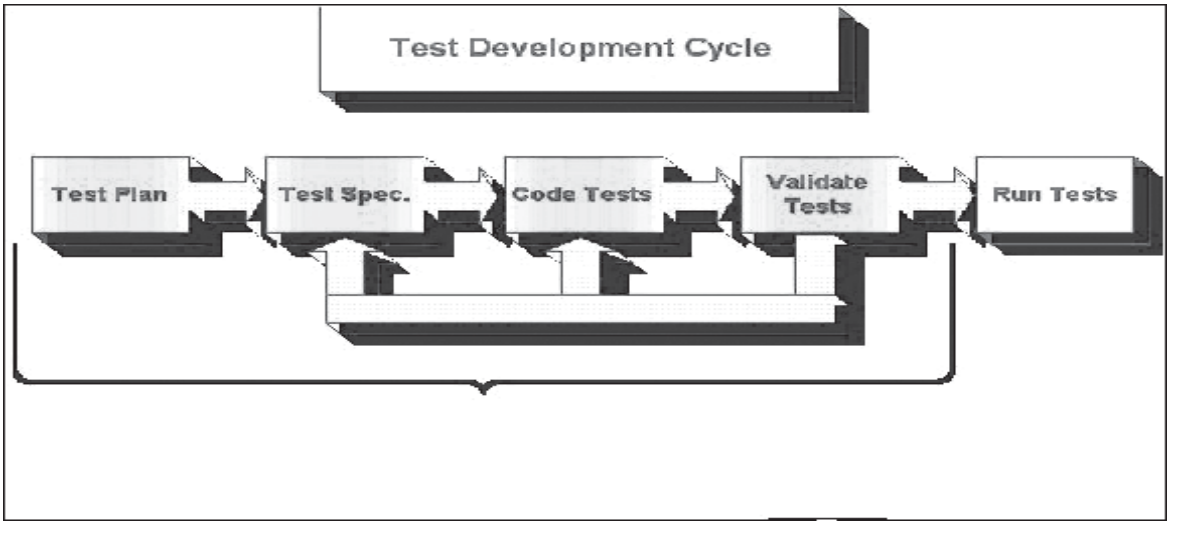


Figure 2.1: **The testing cycle**

The bugs or faults detected at the '**Validate Tests**' stage are fed back to the '**code test**' stage, any non-conformance to the specifications is sent back to the '**test specification',** and the cycle repeats itself.

2.4 Walkthroughs, inspections and reviews

The software technical review process is a part of **validations**. This process includes techniques, such as walkthroughs, inspections and reviews. Most of these activities involve a group meeting to assess the product.

Software technical reviews can be used to examine all the products and interim deliverables such as software requirements, design specifications, test plans, source code, principles of operation, user's manual, and maintenance procedures, of the software evolution process. A walkthrough is not a project review, but is an in-depth examination of selected products by individuals who are qualified to offer expert opinions. Inspection differs from a walkthrough in that a team of trained inspectors, working from checklists of items to be examined, performs inspections of products. Checklists contain a list of tasks to be examined. After each task is completed, it is ticked off.

2.4.1 Walkthroughs

A walkthrough team consists of a reviewer and three or four reviewers. Members of a walkthrough team consist of the project leader, a representative from the quality assurance group and a technical writer. Clients and users usually are included in the group during the requirements and preliminary design stage, but excluded from the subsequent walkthrough sessions. This is because they are generally required at the initial stages at the time of the freezing of the requirements. Several guidelines should be observed while carrying out walkthroughs:

- Everyone's work should be reviewed on a selected basis.
- Importance should be placed on detecting defects
- Major issues (such as code efficiency) should be discussed
- Walkthrough sessions should be of limited time

Sufficient time for walkthroughs must be allocated in the project schedule. Although it might seem as the wastage of time for the project teams, it offers the following benefits:

- Helps in detection of errors at the earliest possible time
- Helps in team communication improvement
- Helps team members learn from each other about new techniques
- Helps new team members learn quickly the various project practices
- Improves motivation

2.4.2 Inspections

Inspections can be used throughout the development lifecycle to assess and improve the quality of various products and interim products. An inspection team consists of one or more members who are trained for this task. The inspectors use the checklists of inspection items.

Design and code inspections were first described by Fagan ("*Design and Code inspections to reduce errors in Program Development*" by Fagan M.). In Fagan's experiment, three separate inspections were performed: the first one following design; but before implementation, the second following implementation; but before testing, and the third following unit test. The inspection following unit testing was not considered cost-effective in discovering errors, and therefore, was not recommended.

According to Fagan, an inspection team consists of four persons, who play the roles of moderator, designer, implementer, and tester. Items to be inspected in a design inspection may include completeness of the design with regard to the user and functional requirements, internal completeness and consistency in the definition and usage of terminology, and correctness of the interfaces between modules. Items to be inspected in a code inspection may include subprogram interfaces, decision logic, data referencing computational expressions, I/O statements, comments, data flow, and memory usage.

Actual items to be examined under the category of subprogram interfaces, for example, may include the following questions *("The Art of Software Testing"* by Myers G):

- "Are the numbers of actual parameters and formal parameters in agreement?
- Do the type attributes of actual and formal parameters match?
- Do the dimensional units of actual and formal parameters match?
- Are the numbers, attributes, and ordering of arguments to built-in functions correct?
- Are constants passed as modifiable arguments?
- Are global variable definitions and usage consistent among modules?"

2.5 Static Analysis

In static analysis, the structure of the code is examined, but the code is not executed. This method is particularly useful for discovering questionable code practices and deviation from the coding standards. This is also helpful for detecting structural errors such as uninitialized variables, any mismatches between actual and formal parameters.

Static analysis can be carried out not only manually using walkthroughs or inspections techniques, but also using automated tools such as **static analyzer**. A static analyzer typically constructs a symbol table and a graph of control flow for each subprogram. The symbol table consists of information about each variable- its type and attributes, the statements where the variable is declared, statements where the variable is set to a new value, and statements where the variable is used to provide values.

Figure 2.2 illustrates a code segment and the related control flow graph. Given a control flow graph and a symbol table, a static analyzer can determine data flow information such as uninitialized variables, variables that were declared but never used.

```
READ A;
 IF A<0 then
 B-> B+A;
 ELSE
 B-> A;
 STOP
 END IF;
```

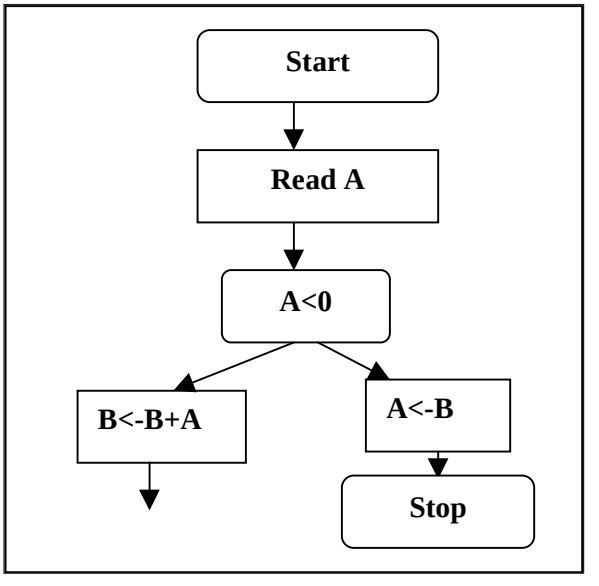


Figure 2.2: **A code segment and a control graph**

There are both practical and theoretical limitations to static analysis. A major practical limitation involves dynamic evaluation of memory references at run time. In higher-level programming languages, array subscripts or pointer values are used. Thus, it is almost practically impossible to distinguish between array elements or members of a link-list using static analysis techniques. Dynamic test cases are typically used to obtain information that is difficult or impossible to obtain by static analysis techniques.

Major theoretical limitations are imposed on static analysis using **decidability theory**. The theory states "given an arbitrary program written in a general purpose programming language, that is, one capable of simulating a Turing machine, it is impossible to examine the program in an *algorithmic* manner and determine if an *arbitrarily chosen* program will be executed when the program operates on arbitrarily chosen input data". **Algorithmic manner** means that it is impossible to write a computer program to perform this task for all possible programs such as loops and conditions. The term **arbitrary** means that there are programs and data, for which the analysis is not possible. Moreover, there is no algorithmic model to identify all programs and data for which the analysis is possible.

Thus, the decidability theory states that it is not possible to write a static analyzer that will examine an arbitrary program and an arbitrary input data, and determine whether the program will execute a particular halt statement, or call a particular subroutine, or execute a particular I/ O statement, or branch to a given label. It is, however, possible to place restrictions on programs so that the undecidable issues become decidable. For example, it is possible to conduct a static analysis for a straight-line program with no branches because it can be guaranteed that all the statements will be executed in the path.

By restricting programs to have only linear path conditions, the problem of automatically deriving test data to drive programs along particular execution path can be solved using static analysis, symbolic execution and linear programming. The term 'linear path' condition implies that there will not be any branching or iterations. The requirements for linear path conditions, of course, place serious restrictions on the programs and programmers.

2.6 The Symbolic Execution

Symbolic execution is a validation technique in which the input variables of a program unit are assigned symbolic values rather than literal values. A program is analyzed by propagating the symbolic values of the inputs into the operands in expressions. This means that the inputs to the program are not numbers, but symbols representing the input data that can take different values. The execution of the program goes on like normal execution. The outputs are symbolic formulas of input values. These formulas can be checked to see if the program will behave as expected. Let us consider a simple function and its symbolic execution:

Example 1

```
function product(a,b,c:integer):integer
var temp1, temp2:integer;
var product: real;
begin
      temp1:= a*b;
      temp2:=b*c;
      product:= (temp1*temp2)/b;
end;
```

The '**integer**' after the function **product** indicates that the function returns an integer.

Now, let us consider that the symbolic inputs to the function are Ai, Bi and Ci. We shall start executing this function with these inputs. The aim is to determine the symbolic values of different variables in the program after the "execution" of each statement. This is shown in Table 2.1:

After statement	A	B	C	Temp1	Temp2	Product
1	Ai	Bi	Ci	?	?	?
4	Ai	Bi	Ci	(Ai)*(Bi)	?	?
5	Ai	Bi	Ci	(Ai)*(Bi)	(Bi)*(Ci)	?
6	Ai	Bi	Ci	(Ai)*(Bi)	(Bi)*(Ci)	(Ai*Bi*Bi*Ci)/Bi

Table 2.1: **Symbolic execution of the function product ('?' symbol means the value is not determined)**

After the 6th statement, the value of the product is (Ai*Bi*Bi*Ci)/Bi; after simplification this expression will be reduced to (Ai*Bi*Ci). In this example, there is only one path in the function and this symbolic equation is equivalent to checking all possible values for a, b and c. Therefore, with only one path and having an acceptable symbolic result of the path, we can claim that the product is correct.

2.6.1 Various path conditions

In the previous program, we studied unconditional statements. While dealing with conditional execution, it is not sufficient to just look at the state of the variables of the program at different statements because a statement will only be executed if the inputs satisfy certain conditions. We shall need a notion of **path condition** to capture such scenarios in symbolic execution. Path condition at a statement gives the conditions, which the inputs must satisfy for the statement to be executed.

A path condition is a Boolean expression using the symbolic inputs, and never contains any program variables. It will be represented in a symbolic execution by a variable '**pc**'. Each symbolic execution begins with the '*pc*' initialized to '*true*'. As and when conditions are encountered for different cases, referring to different paths in the program, the path condition will take different values. For example, the symbolic execution of an IF statement of the form,

if C then S1 else S2

will need two cases to be considered, corresponding to the two possible paths- the first one where C evaluates to 'true' and S1 is executed, and the other where, C evaluates to 'false' and S2 is executed.

For the **first case**, we set pc as pc <- pc & C

For the **second case**, pc can be set to pc <- pc & (~C)

In the second case, (~C) denotes the opposite value of C.

Let us consider an example using the IF statement. The following program determines the maximum of three numbers. The example shows the symbolic execution.

Example 2

```
function max (a,b,c:integer): integer
begin
    if a < = b then
            max:= b
    else
            max:= a;
    if max < c then
            max:=c;
end;
```

As shown in the example 2 above, the symbolic execution forks into two cases, each case having a different path condition. For each path, the value returned is consistent with the specifications of the program. For example, when the inputs satisfy the condition ci > bi > ai, the value of ci is the maximum. Hence, the value of 'ci' is returned in the symbolic execution.

Stmt	**pc**	**max**
1.	true	?
Case (a>b)		
2.	(ai>bi)	?
3.	___	ai
Case (max>c)		
4. (ai>bi) & (ai<ci) ci		
Return this value to max		
Case (max <c)		
4. (ai>bi) & (ai<ci) ai		
Return this value of max		
Case (max>c)		

As discussed before, first, the program path **pc** is set to **true.** The **max** value cannot be determined.

In the second case, the max value cannot be determined, even if pc is set to (ai>bi).

In the third case, no explicit pc is set (**else** statement with no explicit condition). It will return ai as max value only if the condition max > c.

In the fourth case, there are two possibilities:

If the case is max<c, then ci will be returned as the max value.

If the case is max<c, and the pc condition is set to (ai>bi) & (ai<ci), the max value returned is ci.

In the next case, for pc-> (ai>bi) and (ai<ci), the maximum value returned is ai for the case max>c.

Different paths followed during the symbolic execution can be displayed as an **execution tree,** given in Figure 2.3. A node in this tree represents the execution of a tree, while an arrow represents the transition from one state to another. For each IF statement where both the paths are followed, there are two circles from the node corresponding to the IF statement, one labeled with **T**, which is 'true', and the other with **F**, which is 'false', for the '**T**' and '**F**' paths. At each branching, the path condition is also generally displayed in the tree.

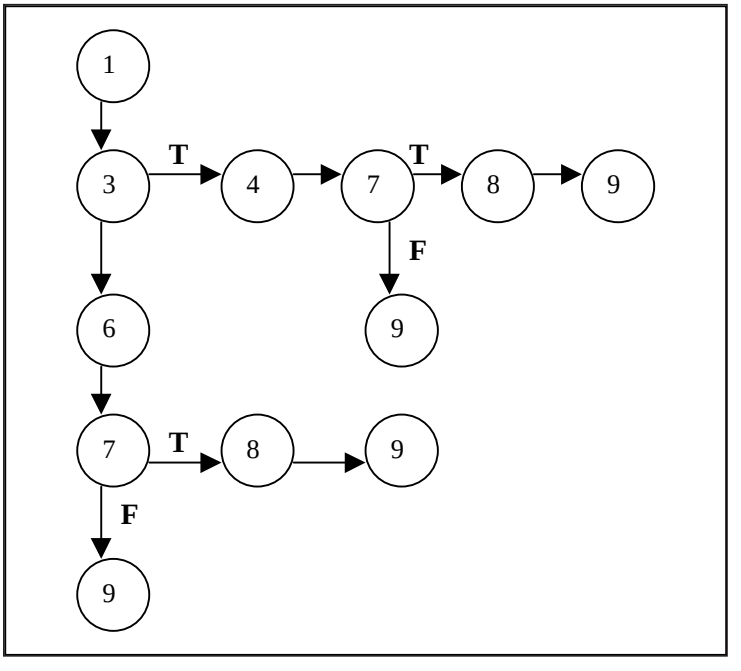


Figure 2.3: **The execution tree for the function 'max'**

The execution tree of a program has some interesting properties. Each leaf in the tree represents a path that will be followed for some input values. In each terminal leaf, there exist some actual numerical inputs such that the sequence of statements in the path is from the root of the tree to the leaf. If the symbolic output at each leaf in the tree is correct then it is equivalent to saying that the program is correct. Hence, if we consider all the paths, the correctness of the program can be established through the symbolic execution.

2.7 Carrying out various software tests

Software testing is the process of examining a product to verify that it satisfies specified requirements or to identify differences between the expected and the actual results. Figure 2.4 shows various levels of tests as well as the stage of the SDLC where a particular test is carried out.

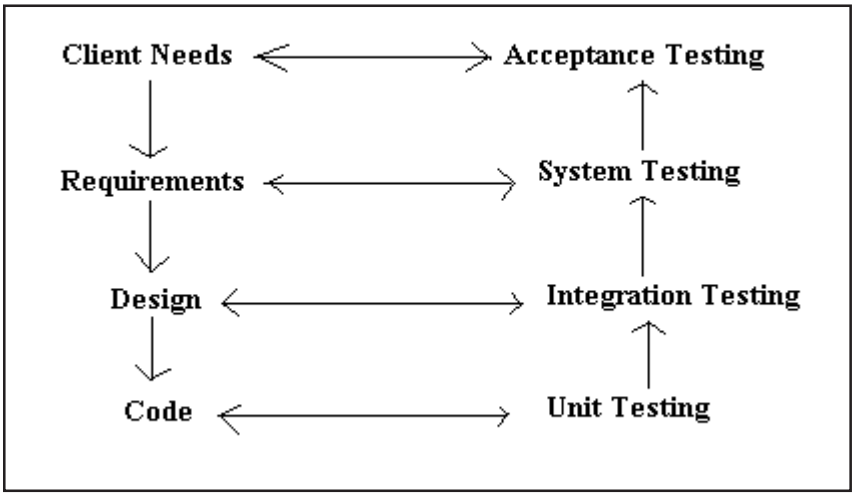


Figure 2.4: **Levels of testing**

Various levels of testing are discussed in the next sessions.

SUMMARY

- Testing is one of most important activities in verification and validation.
- The strategy for testing must be in place before testing starts.
- A test plan is a prescription of activities that will be performed to demonstrate that a software product meets its requirements.
- The software technical review process is a part of validations.
- The review process includes techniques such as walkthroughs, inspections and reviews.
- In static analysis, the structure of the code is examined, but the code is not executed.
- Symbolic execution is a validation technique in which the input variables of a program unit are assigned symbolic values rather than literal values.
- Software testing is the process of examining a product to verify that it satisfies specified requirements or to identify differences between expected and actual results.

CHECK YOUR PROGRESS

1. A test plan is a prescription of activities that will be performed to demonstrate that a software product meets its requirements. **[True/False]**

2. _____ includes strategy, schedule and responsible individuals.

3. The _____criteria is required to achieve a specified rate of error exposure and to achieve a specified percent of logical path coverage.

4. Software technical review includes techniques such as ____, ____ and _____.

5. In static analysis, the structure of the code is examined, but the code is not executed. **[True/False]**

6. A static analyzer typically constructs a symbol table and a graph of control flow for each subprogram. **[True/False]**

7. A path condition is a Boolean expression using the symbolic inputs, and always contains any Program variables. **[True/False]**

Chapter

3 Significance of Testing

Objectives

At the end of this chapter, you will be able to:

- *Explain the importance of software testing*

3.1 Introduction

The quality and reliability of software is often seen as the weak link in industry's attempts to develop new products and services. The last decade has seen the issue of software quality and reliability addressed through a growing adoption of design methodologies and supporting CASE tools, to the extent that most software designers have had some training and experience in the use of formalized software design methods.

Unfortunately, the same cannot be said of software testing. Many developments applying such design methodologies are still failing to bring the quality and reliability of software under control. It is not unusual for 50% of software maintenance costs to be attributed to fixing bugs left by the initial software development; bugs which effective software testing should have eliminated.

3.2 Understand Unique Capability of Testing

Testing will increase our confidence in the correct operation of valuable functions, and decrease the odds of a catastrophic system failure.

Testing produces information reliably grounded in the observed behavior of a system.

Testing unique capability (especially distinct from other "quality" efforts) is producing information reliably grounded in observed system behavior. Reviews, inspections and development are based on the anticipation of what the system might do. Testing is the only software quality activity that directly observes system behavior and reports what it does. Testing is valuable; anytime we can benefit from direct observation, free of speculation.

Speculation is an integral part of development. In reality, all development is a kind of disciplined speculation. Analysis answers the question, "What might be a valuable behavior, if we could create it?". Design answers the question, "What might we do to support this behavior?". Implementation answers, "What will realize the design?". Planning answers, "How can we arrange activities to get what we want?". If this speculation were perfectly reliable, however, we would never need to test and we would never have a defect. Development is inferences stacked on inferences. It is surprising that we get as much of it right as we do.

Organizationally, Testing is often grouped with Development. However, they are not the same process as the outputs are different.

- Development produces an artifact that automates functions. It is funded because it does something.
- Testing produces reliable information about an artifact. It is funded because we learn something.

In terms of systems design, grouping the functions of testing and development because they employ similar skills, tools, or timing, corresponds to weak binding. Executives understand processes in terms of functional binding based on the processes' unique output. Identifying the output of a business process—what it is good for—traces naturally to justification. It is easier to understand, and to explain.

Testing produces value indirectly through information about valuable system behaviors.

We do not use a test; we use a system. Testing is justified by the information it produces and influences the value of the system we will use.

- Testing reduces the odds of misbehavior for the things tested. In this sense, testing functions like a credit check. The check does not generate value. It influences the value realized from a purchase and increases the odds of getting paid.
- Failed tests suggest that the tested features will be repaired and eventually work when they would not otherwise.
- Passed tests suggest that the tested features will work. These can be intended features, designed into a system, or unintended features included through serendipity (particularly common with volume tests). A system designed to support a volume of N may well successfully support N' > N.
- Testing can produce information about what the system might do. The rich interaction of testing a system uncovers many high-quality potential features.
- Testing information can be used to refine the development process, through root cause analysis to classify defects in terms of which process failed. It indicates where to apply attention to development.

Testing value equals the sum of all of the changes in our confidence about valuable system behaviors.

The people who fund testing do not generally work with formulas involving sums and deltas, but that is exactly how they think about business processes. To them, testing value is their increased confidence in all the things the system is supposed to do. This formula is the way to quantify testing value with potential sponsors: "With this investment in testing, we're going to get more confident about this list of valuable assertions about the system." At an aggregate level, where processes operate, this formula is true.

For presentation, remember that executive decision-makers appreciate summary tradeoff data. "Of 1,000 functions in the code that was tested, two failed in operations during the first month. Of 1,000 functions in the untested code, seventeen failed during the same period." In fact, the decision of whether to test a chunk of

code is now a choice between two different frequencies of reported failure. Organizationally, successful testers always represent the testing process in terms of the aggregate reasoning of management decisions.

Testing Integrates with the Rest of the Business

Testing must interact with development and support organizations to get a system to test, and to get an environment to test in. Therefore, before producing any results, testing must interact with two parts of the organization.

In addition, the executives want to know who is going to use what we will produce. Each result from testing must have a consumer in order to be valuable: either Development (where they are used to repair the system), or users (who can avoid problems), or decision-makers (who decide to ship or not). Bugs and fixes without consumers are not very valuable. Reports of passed tests have to go to system users and sponsors, who use the data to make decisions about the system.

If we want to be approved, have a customer for the information we are using as the basis for our proposal. Even better, ask our customers for it. This dependence on customers is more obvious with information about process improvement and new features. Providing this information if Development does not want, it will only annoy the developers.

When satisfying potential sponsors, a successful testing proposal should include process integration, identified customers, an existing interest in the information we will produce, and alignment of this information with the organization's strategy.

Bugs affect the customer as well. Every bug should be discussed in detail and assessed with the question "Can we live with it". We are not reviewing test results (bugs, tests passed, potential features, all of it) with our customers, with Development, and with process improvement groups, we do not have the customers who make us successful.

Testing isn't a moral issue; it's a proposal.

It is to justify an investment in testing. The "correct" investment in testing is completely situational.

For a low-value, low-risk system, no testing at all may be the correct plan of action. In addition, if we are doing highly disciplined development, inspections may render most testing irrelevant. If the organization is not ready to accept suggestions for new functions, producing these suggestions will not help very much.

For example, it has become routine for companies to make large investments in test automation, which they are not ready to exploit.

This is the approach that will convince sponsors:

- Identify a willing, even eager, customer for each kind of information we propose to produce

- Compare the cost of producing with the value expected
- Propose only what we can do
- Work out the related costs, schedule, and so on
- Do it in chunks

Remember that there is not one true investment in testing.

Implementing the change with testing.

By identifying how testing integrates with the rest of the business, we have identified how the sponsors monitor their part of the implementation. They will ask our customers—the people identified as needing and using the information we will produce—what information they are getting, and what they are doing with it.

Performing testing is a technical activity. Implementing testing, on the other hand, is a long-term commitment to adding (or changing) a business process, and then integrating that change with the rest of the business. It has to be tracked, just like any other project that encompasses both infrastructure and process changes. Make sure our plan incorporates basic project management of the testing implementation, including:

- Organizing people: jobs, administration, facilities, time and cost reporting, and so on
- Adding communication with development, customers, sponsors, and more
- Implementing a test execution infrastructure with robust configuration management
- Implementing an infrastructure for test planning: test tracking and reporting results
- Configuring Development to support Testing as an additional customer
- Enhancing software engineering processes to accommodate the volume and robustness needed to support testing

Monitoring Testing Performance

Business managers manage to performance measures, or the company ends up in trouble. They talk about cost of sale, cost of lost customer, cost to resolve a complaint. For monitoring performance, we have to make the connection between testing investment and similar operational measures.

The two fundamental measures of testing are the confidence it produces in tested assertions, and the scope of the assertions that can be tested. Performance measures describe how well this capability is

supplied: cost, precision, and timing are a few examples. We do not have to get mathematical about it, but we do have to name what is going to get better.

- We will get more confident in the things we have tested.
- We will increase the scope of what we know about the system.
- In combination with Development's management of defects, Testing and Development together will produce systems with lower defect rates. Testing contribution is defects found versus prevented.
- In combination with Development's management of candidate features, Testing and development together will provide features that are more valuable in our systems; we will leave less value on the table. Testing contribution is one of the high-quality candidate features.
- Testing reliable information about the system(s) produced will be an important input to ongoing software process improvement efforts in development.

Now that we have laid our plan, the only process left to do is delivering.

Source-stickyminds.com (originally published in STQE magazine)

- It is not unusual for 50% of software maintenance costs to be attributed to fixing bugs left by the initial software development; bugs that effective software testing should have eliminated.
- Testing unique capability is producing information reliably grounded in observed system behavior.
- Testing is justified by the information it produces and influences the value of the system we will use.
- Organizationally, successful testers always represent the testing process in terms of the

SUMMARY

aggregate reasoning of management decisions.

- Testing must interact with development and support organizations to get a system to test, and to get an environment to test in.
- The “correct” investment in testing is completely situational.
- Implementing testing, is a long-term commitment to adding (or changing) a business process, and then integrating that change with the rest of the business.
- The two fundamental measures of testing are the confidence it produces in tested assertions, and the scope of the assertions that can be tested.

CHECK YOUR PROGRESS

1. Testing is the only software quality activity that directly observes system behavior and reports what it does. **[True/False]**

2. _____ tests suggest that the tested features will work.

3. Every bug should be discussed in detail. **[True/False]**

4. For a low-value, low-risk system, testing is a must. **[True/False]**

5. Performance measures describe how well this capability is supplied: cost, precision, and timing. **[True/False]**

6. Testing implementation includes organizing people: jobs, administration, facilities, time and cost reporting. **[True/False]**

ASK to LEARN

Onlinevarsity
your e-way to learning

Questions
in your
mind?

are here to HELP

Post your queries in ASK to LEARN @

www.onlinevarsity.com

Chapter

4 Software Testing

Objectives

At the end of this chapter, you will be able to:

- *Explain Software Testing*
- *Describe Various Methods of Software Testing*
- *Describe Various Testing Strategies*

4.1 Introduction

Software testing is a critical component of the software engineering process. It is an element of software quality assurance and can be described as a process of running a program in such a manner to detect errors (if any). This process, while seen by some as tedious, tiresome and unnecessary, plays a vital role in software development. It is necessary to extensively test the system to ensure its reliability.

There are many published definitions of software testing, however, all these definitions boil down to essentially the same thing: software testing is the process of executing software in a controlled manner to answer the question “Does the software behave as specified?”. Software testing should not be confused with debugging.

4.2 Principles of Software Testing

The process of software testing involves creating test cases to “break the system” but before these can be designed, a few principles have to be observed:

- Testing should be based on user requirements. This is in order to uncover any defects that might cause the program or system to fail to meet the client’s requirements.
- Testing time and resources are limited, so avoid redundant tests.
- It is impossible to test everything. Exhaustive tests of all possible scenarios are impossible, because of the many different variables affecting the system and the number of paths a program flow might take.
- Use effective resources to test. This represents use of the most suitable tools, procedures and individuals to conduct the tests. The test team should use tools that they are confident

and familiar with. Testing procedures should be clearly defined. Testing personnel may be a technical group of people independent of the developers.

- Test planning should be done early. This is because test planning can begin without coding and as soon as the client requirements are set.

- Testing should begin at the module. The focus of testing should be first concentrated on the smallest programming units and then expand to other parts of the system.

4.3 Software Testing Methods

There are many ways to conduct software testing, but the most common methods rely on the following steps.

4.3.1 Test Case Design

Test cases should be designed in such a way as to uncover quickly and easily as many errors as possible. They should “exercise” the program by using and producing inputs and outputs that are both correct and incorrect. Variables should be tested using all possible values (for small ranges) or typical and out-of-bound values (for larger ranges). They should also be tested using valid and invalid types and conditions. Arithmetical and logical comparisons should be examined as well, again using both correct and incorrect parameters.

The objective is to test all modules and then the whole system as completely as possible using a reasonably wide range of conditions.

4.3.2 White-Box Testing

White box method relies on intimate knowledge of the code and a procedural design to derive the test cases. It is most widely utilized in unit testing to determine all possible paths within a module, to execute all loops and to test all logical expressions. Using white-box testing, the software engineer can:

(1) Guarantee that all independent paths within a module have been exercised at least once.

(2) Examine all logical decisions on their true and false sides.

(3) Execute all loops and test their operation at their limits.

(4) Exercise internal data structures to assure their validity (Pressman, 1997).

This form of testing concentrates on the procedural detail. However, there is no automated tool or testing system for this testing method. Therefore, even for relatively small systems, exhaustive white-box testing is impossible because of all the possible path permutations.

4.3.3 Basis Path Testing

Basis path testing is a white-box technique. It allows the design and definition of a basis set of execution paths. The test cases created from the basis set, allow the program to be executed in such a way as to examine each possible path through the program by executing each statement at least once (Pressman, 1997). When determining different program paths, the engineer needs a representation of the logical flow of control. The control structure can be illustrated by a flow graph. A flow graph can be used to represent any procedural design.

Next, a metric can be used to determine the number of independent paths. It is called cyclomatic complexity and it provides the number of test cases that have to be designed. This insures coverage of all program statements (Pressman, 1997).

4.3.4 Control Structure Testing

Since the basis path testing alone is insufficient, other techniques should be utilized.

Condition testing can be utilized to design test cases, which examine the logical conditions in a program. It focuses on all conditions in the program and includes testing of both relational expressions and arithmetic expressions. This can be accomplished using branch testing and/or domain testing methods. Branch testing executes both true and false branches of a condition. Domain testing utilizes values on the left-hand side of the relation by making them greater than, equal to and less then the right-hand side value. This method test both values and the relation operators in the expression.

Data flow testing method is effective for error protection because it is based on the relationship between statements in the program according to the definition and uses of variables. Loop testing method concentrates on validity of the loop structures.

4.3.5 Black-box Testing

Black box, on the other hand, focuses on the overall functionality of the software. Hence, it is the chosen method for designing test cases used in functional testing. This method allows the functional testing to uncover faults like incorrect or missing functions, errors in any of the interfaces, errors in data structures or databases and errors related to performance and program initialization or termination.

It is important to understand the relationships between different modules in the system model to perform a successful black-box test. Next, all necessary ways of testing all object relationships need to be defined. In this case, a graph representing all the objects can be constructed. A node represents each object and then links between the nodes show the direct node-to-node relationship. An arrow on the link shows the direction of the relationship. Each node and link is further described by node weight or link weight respectively. This method is called graph-based testing (Pressman, 1997).

Equivalence partitioning, on the other hand, divides input conditions into equivalence classes such as Boolean, value, range or set of values. These classes represent a set of valid or invalid states for input conditions (Pressman, 1997).

Boundary values analysis (BVA), as the term suggests, concentrates on designing test cases that examine the upper and lower limits of the equivalence class. These test cases are not based solely on input conditions as in equivalence partitioning above but on output conditions as well (Pressman, 1997).

4.4 Software Testing Strategies

When conducting a proper and thorough set of tests, the types of testing mentioned below should be performed.

4.4.1 Unit Testing

Module or unit testing is the lowest level of testing and involves the testing of a software module or a unit. The goal of unit testing is to ensure that the component being tested conforms to the specifications and is ready to be integrated with other components of the product.

This testing comprises of a set of tests performed by an individual programmer before the integration of the unit into a larger system. The situation is illustrated as follows:

Coding & debugging -> unit testing -> integration testing

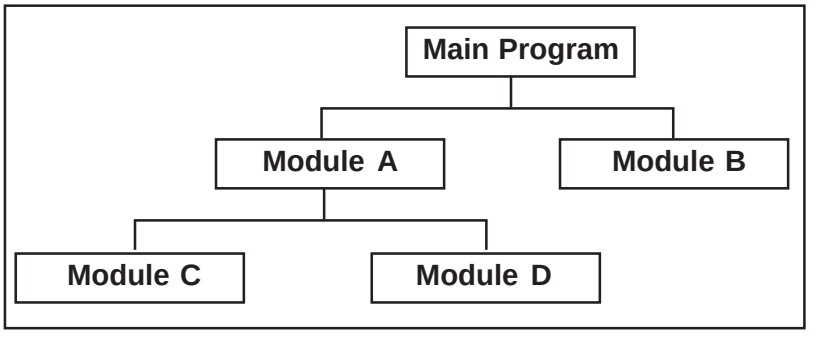


Figure 4.1: Individual pieces of code are built up and tested separately, and then integrated and tested again

Four categories of testing are carried out on a program unit or module. These categories are as discussed below:

- **Functional tests**: Involve examining the code with nominal input values for which the expected results are known. Also, involve boundary values (refers to minimum values), maximum

values, values within the range of functional boundaries, and special values such as logically related inputs.

- **Performance tests**: Determine the amount of execution time spent in various parts of the unit, as well as program throughput, response time, and the device utilization by the unit or module.

- **Stress tests**: Designed to intentionally break the unit. This test checks whether passing a large amount of data breaks the system or not.

- **Structure tests**: Concerned with exercising the internal logic of a program and traversing particular execution paths. Sometimes, the above three tests are collectively referred to as the *black box* testing, whereas, structure tests are sometimes referred to as the *white box* testing. The major activities during structure tests are deciding which path to examine, determining the test coverage criteria to be used, executing the test cases, and measuring the test coverage achieved when test cases are examined.

Several tools have been developed for the testing purpose. One example is the SOFTOOL package. This tool provides both static analysis and dynamic tracing capabilities.

4.4.2 Debugging

Debugging is the process of isolating and correcting the causes of known errors. Commonly used debugging methods are *inductive*, *deductive* and *backtracking*. *Debugging by induction* involves the following steps (*Source*: *Program debugging* by Brown and Sampson and *The Art of Software Testing* by Myers):

- Collecting the available information
- Looking for patterns
- Forming one hypothesis more than one hypotheses
- Proving or disproving each hypothesis
- Implementing the appropriate corrections
- Verifying the corrections.

Debugging by deduction proceeds in the following way:

- List all possible causes for the observed failure
- Use the available information to eliminate various hypotheses
- Elaborate the remaining hypotheses

- Prove or disprove each hypothesis
- Determine the correctness
- Verify the corrections

Debugging by backtracking involves working backward in the source code from the point where the error was observed in an attempt to identify the exact point where the error occurred. It might be necessary to run more test cases to collect more information.

Common debugging techniques utilize **diagnostic output statements**, **snapshot dumps** and **instruction-dependent breakpoints**. Modern debugging techniques use **assertion-controlled breakpoints and execution histories**.

- **Diagnostic output statements** can be embedded in the source code as specially formatted commented statements. The diagnostic output from these statements provides snapshots of selected components of the program state from where a programmer tries to infer the program behavior. The program state includes the values associated with all currently accessible symbols and any additional information such as the program stack.
- A **snapshot dump** is a machine-level representation of the partial or total program state at a particular point in the execution sequence. A structured snapshot dump is a source-level representation of the partial or total state of the program.
- A **trace facility** lists changes in the selected state components. In its simplest form, a trace will print all changes in control flow with the change of the selected variables.
- **Assertions** are logical predicates written at the source code level to describe relationships among components of the current program state and relationships between program states, for example, an assertion **I< 10**. An assertion violation can change the program sequence or trap control to the programmer's terminal. Assertion violations that return the control to the programmer's terminal are called *conditional breakpoints*. Conditional breakpoints are state-dependent, whereas unconditional breakpoints are *instruction-dependent*. Conditional breakpoints allow a programmer to focus on the nature of the error itself.

  In the debugging mode, an assertion facility frees a programmer from the customary trial and error guessing needed to locate the source of an error. The programmer does not have to test the dumps and traces to understand the correct program behavior.
- An **execution history** is actually a record of execution events collected from an executing program. The history is typically stored in a database for the post-mortem examination after the execution of the program. A **traceback** facility uses the execution history to trace the control flow and the data flow both in the forward and the backward directions in the execution time. This allows analysis of how a particular calculation was influenced by the previous events. One such example is the Extendable Debugging and Monitoring System (EXDAMS). (Source: "*EXDAMS: Extendable Debugging & Monitoring System*" by Balzer).

4.4.3 System testing

This type of testing is the process of testing the integrated hardware and software system to verify that the system meets its specified requirements. Practical priorities must be established to complete the task effectively. One of the priorities can be that the system testing must stress on the system capabilities. This implies that the system tests concentrate more on ensuring the use and interaction of functions rather than testing the details of their implementation. There are several *key principles* that are to be considered while testing the system. These principles are as given below:

- System tests should be developed and performed by a group independent of the people who developed the code.
- System test plans must be developed and inspected with the same rigor as other elements of the project.
- System test principles must be planned and tracked similarly to other elements of the projects.
- System tests must be repeatable.

4.4.4 Integration testing

This type of testing consists of the systematic combination and execution of product components. Multiple levels of integration testing are possible with a combination of various hardware and software components at different levels. The goal of integration testing is to ensure that the interfaces between the components are correct and that the product components are combined to execute the functionality of the product correctly.

Bottom-up integration is usually used as the integration testing strategy. This consists of unit testing, followed up by testing of the subsystem, followed by the testing of the entire system. Modules are tested in isolation from one another in an environment called **testing harness**. A testing harness consists of driver programs and data necessary to examine the modules. Test harnesses provide data environments and calling sequences for the routines and subsystems that are being tested in isolation. Test harness preparation typically can amount to 50 percent or more of the coding and debugging effort for a software product.

Unit testing should be as exhaustive as possible to ensure that each representative case handled by each module has been tested.

A **subsystem** consists of several modules that communicate with each other through well-defined interfaces. The primary goal of the **sub-system testing** is to verify the operation of the interfaces among the modules of the subsystem. Both control and data interfaces must be tested.

System testing is concerned with subtleties in the interfaces, decision logic, control flow, recovery procedures, throughput, capacity, and the timing characteristics of the entire system.

Automated tools for carrying out the integration testing include module drivers, test data generators, environment simulators, and a library management facility to allow the easy configuration of system. Module drivers allow the specification of test cases (both inputs and expected results) in a descriptive language. The driver tool then invokes the subroutine using the specified test cases, compares actual results with expected results, and report discrepancies.

Test case generators are of two types including those that generate files of random data values according to some predefined format, and those that generate test data for particular execution paths.

Environment simulators are generally used during integration and acceptance testing to simulate the operated environment in which the software product will function. Simulators are typically used in scenarios where operation in actual environment is impractical such as development of a software product for non-existent machine and in situations where ballistic missiles will be deployed. Example of environment simulators is the **Saturn Flight Program Simulator** for simulating live flight tests.

An automated development library management system consists of a database that consists of all project documentation, source code, object code, test cases, error reports, and so on in the machine-readable form. Examples of automated library management systems include the **Source Code Control System** (SCCS).

4.4.5 Regression Testing

This can be defined as the process of executing previously defined test cases on a modified program to assure that the software changes have not adversely affected the program's functions previously executed. The error-prone nature of software projects demands this type of testing. Some types of errors targeted by regression testing include:

- **Data corruption errors**: These errors are side effects generated due to shared data.
- **Inappropriate control sequencing errors:** These errors are side effects generated due to changes in execution sequences. An appropriate example will be that of removing an item in the queue before placing that item in the queue.
- **Resource Contention:** Examples are potential bottlenecks and deadlocks in the system.
- **Performance deficiencies:** These include timing and storage utilization errors.

An important regression testing is to place a higher priority for testing the older capabilities of the product than the new capabilities provided by the changes. This ensures that the capabilities the user has become dependent upon are still intact.

Regression testing strategies are not well defined in the software engineering literature. Most strategies require that some baseline of product tests be rerun. Strategies for testing modifications generally involve some sort of systematic execution of the modifications and related areas. At

module-level, this may involve retesting the module execution paths traversing the modification. At the product level, this may involve retesting functions that execute in the modified area.

4.4.6 Acceptance Testing

This type of testing involves planning and execution of functional tests, performance tests, and stress tests to demonstrate that the implemented system satisfies its requirements. Generally, two sets of acceptable test cases are run which includes those developed by the quality assurance groups and those developed by the customer.

In most of the cases, acceptance tests will incorporate test cases during the unit testing and integration testing. Additional test cases are also considered sometimes to achieve the desired level of functional, performance, and stress testing of the entire system. Generally, various testing tools such as test coverage analyzer, a timing analyzer, and a coding standards checker are employed during testing.

A **test coverage analyzer** records the control paths followed for each test case. A **timing analyzer** reports the time spent in various regions of source code under different test cases. **Static analyzers** and **standards checkers** can be used to inspect code deviations from standards and guidelines. The coding standards can be project-related, customer-related, developer-related or language-related.

Source site : http://www.cs.pdx.edu/~warren/CS510FST/L1.htm#Heading1

SUMMARY

- Testing can be described as a process of running a program in such a manner as to uncover any errors.
- There are many ways to conduct software testing.
- Unit testing is the lowest level of testing and involves the testing of a software module or a unit.
- Debugging is the process of isolating and correcting the causes of known errors.
- Integration testing consists of the systematic combination and execution of product components.
- System testing is the process of testing the integrated hardware and software system to verify that the system meets its specified requirements.
- Regression testing is the process of executing previously defined test cases on a modified program to assure that the software changes have not adversely affected the program's functions, which are previously executed.
- Acceptance testing involves planning and execution of functional tests, performance tests, and stress tests to demonstrate that the implemented system satisfies its requirements.

CHECK YOUR PROGRESS

1. Testing should be based on user requirements. **[True/False]**

2. The focus of testing should be first concentrated on the largest programming units. **[True/False]**

3. White box method relies on intimate knowledge of the _____ and a procedural design to derive the test cases.

4. Black box focuses on the overall _____of the software.

5. Commonly used debugging methods are_____, _____and_____.

6. A subsystem consists of several modules that communicate with each other through well-defined interfaces. **[True/False]**

7. _____ errors are side effects generated due to shared data.

8. System tests should be developed and performed by a group independent of the people who developed the code. **[True/False]**

9. Basis path testing is a black-box technique. **[True/False]**

Onlinevarsity

your e-way to learning

BI g

Balanced Learner-Oriented Guide

for enriched learning available @

www.onlinevarsity.com

Chapter

5 Organizational Approaches for Unit Testing

Objectives

At the end of this chapter, you will be able to:

- *Describe three basic organizational approaches for unit testing*

5.1 Introduction

Unit testing is the testing of individual components (units) of the software. Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

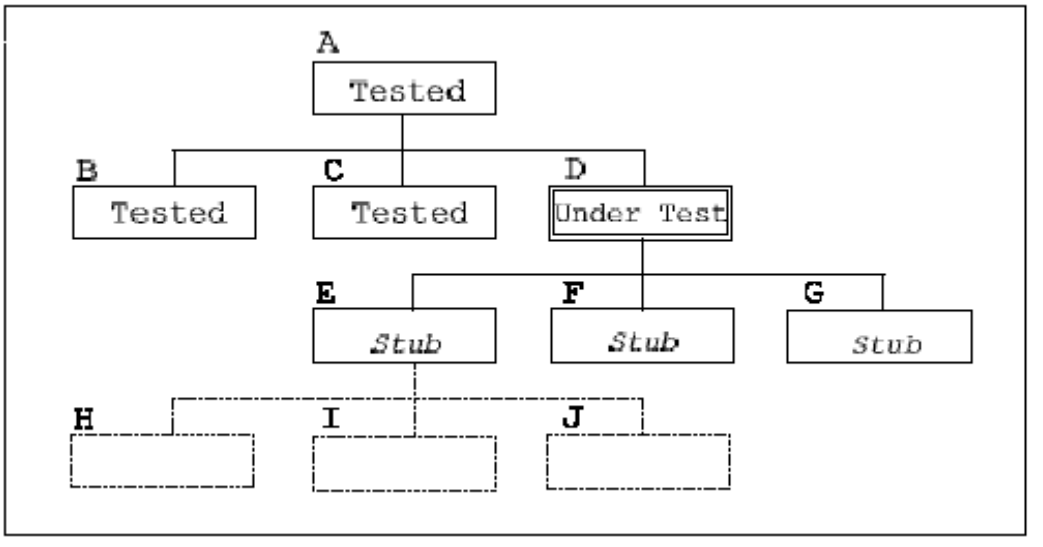
When developing a strategy for unit testing, there are three basic organizational approaches that can be taken. These are **top down**, **bottom up** and **isolation**. In this session, we will learn about these three approaches with their advantages and disadvantages.

The concepts of **test drivers** and **stubs** are used throughout this session. A **test driver** is the software, which executes software in order to test it, providing a framework for setting input parameters, executing the unit, and reading the output parameters. A **stub** is an imitation of a unit, used in place of the real unit to facilitate testing.

5.2 Top Down Testing

In **top down** testing, individual units are tested by using them from the units that call them, but in isolation from the units called. The unit at the top of a hierarchy is tested first, with all called units replaced by stubs. Testing continues by replacing the stubs with the actual called units, with lower level unit being stubbed. This process is repeated until the lowest level units have been tested. Top down testing requires test stubs, but not test drivers.

Figure 5.1 illustrates the test stubs and tested units needed to test unit D, assuming units A, B and C have already been tested in the top down approach. A unit test plan for the program shown in Figure 5.1, using a strategy based on the top down organizational approach, can be read as follows:



Step (1)

Test unit A, using stubs for units B, C and D.

Step (2)

Test unit B, by calling it from tested unit A, using stubs for units C and D.

Step (3)

Test unit C, by calling it from tested unit A, using tested units B and a stub for unit D.\

Step (4)

Test unit D, by calling it from tested unit A, using tested unit B and C, and stubs for units

E, F and G. (Shown in Figure 5.1).

Step (5)

Test unit E, by calling it from tested unit D, which is called from tested unit A, using tested units B and C, and stubs for units F, G, H, I and J.

Step (6)

Test unit F, by calling it from tested unit D, which is called from tested unit A, using tested units B, C and E, and stubs for units G, H, I and J.

Step (7)

Test unit G, by calling it from tested unit D, which is called from tested unit A, using tested units B, C, E and F, and stubs for units H, I and J.

Step (8)

Test unit H, by calling it from tested unit E, which is called from tested unit D, which is called from tested unit A, using tested units B, C, E, F and G, and stubs for units I and J.

Step (9)

Test unit I, by calling it from tested unit E, which is called from tested unit D, which is called from tested unit A, using tested units B, C, E, F, G and H, and a stub for units J.

Step (10)

Test unit J, by calling it from tested unit E, which is called from tested unit D, which is called from tested unit A, using tested units B, C, E, F, G, H and I.

5.2.1 Advantages

Top down unit testing provides an early integration of units before the software integration phase. In fact, top down unit testing is really a combined unit test and software integration strategy. The detailed design of units is top down and top down unit testing implements tests in the sequence the units are designed. Hence, by overlapping unit testing with the detailed design and code phases of the software, lifecycle shortens the Development time.

In a conventionally structured design, where units at the top of the hierarchy provide high-level functions, with units at the bottom of the hierarchy implementing details, top down unit testing will provide an early integration of 'visible' functionality. This gives a very requirements oriented approach to unit testing.

Redundant functionality in lower level units will be identified by top down unit testing because there will be no route to test it. (However, there can be some difficulty in distinguishing between redundant functionality and untested functionality).

5.2.2 Disadvantages

Stubs control top down unit testing, with test cases often spread across many stubs. With each unit tested, testing becomes more complicated, and consequently more expensive to develop and maintain. As testing progresses down the unit hierarchy, it also becomes more difficult to achieve the good structural coverage, which is essential for high integrity and safety critical applications, and which is required by many standards. Difficulty in achieving structural coverage can also lead to confusion between genuinely redundant functionality and untested functionality. Testing some low level functionality, especially error handling code, can be totally impractical.

Changes to a unit often impact the testing of sibling units and units below it in the hierarchy. For example, consider a change to unit D. Obviously, the unit test for unit D would have to change and be repeated. In addition, unit tests for units E, F, G, H, I and J, which use the tested unit D, would also have to be repeated. These tests may also have to change themselves, as a consequence of

the change to unit D, even though units E, F, G, H, I and J had not actually changed. This leads to a high cost associated with retesting when changes are made, and a high maintenance and overall lifecycle cost. The design of test cases for top down unit testing requires structural knowledge when the unit under test calls other units. The sequence in which units can be tested is constrained by the hierarchy of units, with lower units having to wait for higher units to be tested, forcing a 'long and thin' unit test phase. (However, this can overlap substantially with the detailed design and code phases of the software lifecycle).

The relationships between units in the example program in Figure 5.1 is much simpler than would be encountered in a real program, where units can be referenced from more than one other unit in the hierarchy. A unit being referenced from more than other unit compounds, all the disadvantages of a top down approach to unit testing.

A top down strategy will cost more than an isolation-based strategy, due to complexity of testing units below the top of the unit hierarchy, and the high impact of changes. The top down organizational approach is not a good choice for unit testing. However, a top down approach to the integration of units, where the units have already been tested in isolation, can be viable.

5.3 Bottom up Testing

In **bottom up** unit testing, units are tested in isolation from the units, which call them, but using the actual units called as part of the test. The lowest level units are tested first, and then used to facilitate the testing of higher-level units. Other units are then tested using previously tested called units. The process is repeated until the unit at the top of the hierarchy has been tested. Bottom up testing requires test drivers, but does not require test stubs.

Figure 5.2 illustrates the test driver and tested units needed to test unit D, assuming that units E, F, G, H, J and I have already been tested in a bottom up approach.

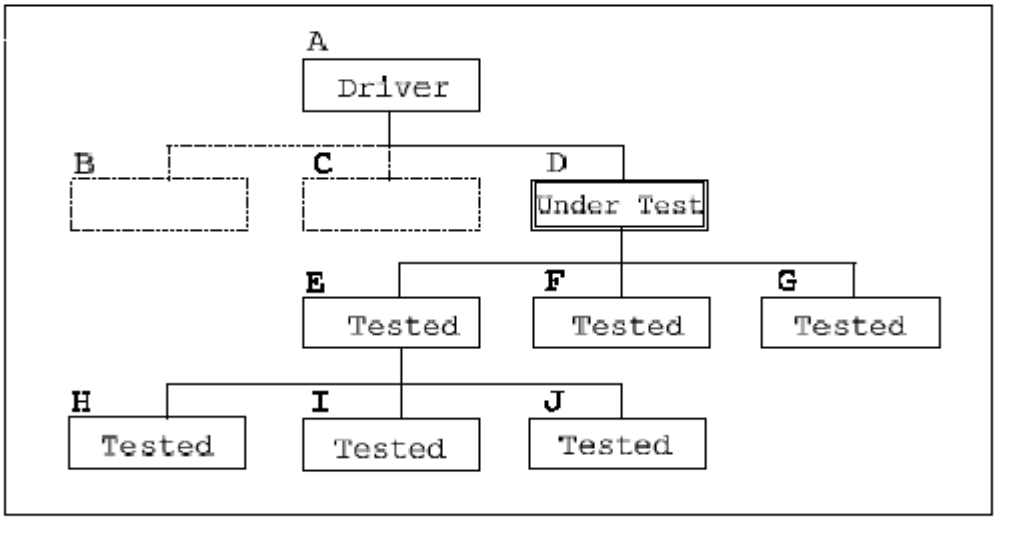


Figure 5.2 - Bottom Up Testing

A unit test plan for the program shown in Figure 5.2, using a strategy based on the bottom up organizational approach, can be read as follows:

Step (1)

(Note that the sequence of tests within this step is unimportant, all tests within step 1 can be executed in parallel.)

Test unit H, using a driver to call it in place of unit E;

Test unit I, using a driver to call it in place of unit E;

Test unit J, using a driver to call it in place of unit E;

Test unit F, using a driver to call it in place of unit D;

Test unit G, using a driver to call it in place of unit D;

Test unit B, using a driver to call it in place of unit A;

Test unit C, using a driver to call it in place of unit A.

Step (2)

Test unit E, using a driver to call it in place of unit D and tested units H, I and J.

Step (3)

Test unit D, using a driver to call it in place of unit A and tested units E, F, G, H, I and J.

(Refer Figure 5.2).

Step (4)

Test unit A, using tested units B, C, D, E, F, G, H, I and J.

5.3.1 Advantages

Like top down unit testing, bottom up unit testing provides an early integration of units before the software integration phase. Bottom up unit testing is a combined unit test and software integration strategy. Solely the test driver controls all test cases, with no stubs required. This can make unit tests near the bottom of the unit hierarchy relatively simple. (However, higher-level unit tests can be very complicated).

Test cases for bottom up testing may be designed solely from functional design information, requiring no structural design information (although structural design information may be useful in achieving full coverage). This makes the bottom up approach to unit testing useful when the detailed design documentation lacks structural detail.

Bottom up unit testing provides an early integration of low-level functionality, with higher-level functionality being added in layers as unit testing progresses up the unit hierarchy. This makes bottom up unit testing readily compatible with the testing of objects.

5.3.2 Disadvantages

As testing progresses up the unit hierarchy, bottom up unit testing becomes more complicated, and consequently more expensive to develop and maintain. As testing progresses up the unit hierarchy, it also becomes more difficult to achieve good structural coverage.

Changes to a unit often impact the testing of units above it in the hierarchy. For example, consider a change to unit H. Obviously, the unit test for unit H would have to change and be repeated. In addition, unit tests for units A, D and E, which uses the tested unit H, would have to be repeated. These tests may also have to change themselves, as a consequence of the change to unit H, even though units A, D and E had not actually changed. This leads to a high cost associated with retesting when changes are made, and a high maintenance and overall lifecycle cost.

The sequence in which units can be tested is constrained by the hierarchy of units, with higher units having to wait for lower units to be tested, forcing a 'long and thin' unit test phase. The first units to be tested are the last units to be designed, so unit testing cannot overlap with the detailed design phase of the software lifecycle.

The bottom up organizational approach can be a reasonable choice for unit testing, particularly when objects and reuse are considered. However, the bottom up approach is biased towards functional testing, rather than structural testing. This can present difficulties in achieving the high levels of structural coverage essential for high integrity and safety critical applications, and which are required by many standards.

The bottom up approach to unit testing conflicts with the tight timescales required of many software developments. Overall, a bottom up strategy will cost more than an Isolation based strategy, due to complexity of testing units above the bottom level in the unit hierarchy and the high impact of changes.

5.4 Isolation Testing

Isolation testing tests each unit in isolation from the units, which call it, and the units it calls. Units can be tested in any sequence because no unit test requires any other unit to have been tested. Each unit test requires a test driver. Stubs replace all called units. Figure 5.3 illustrate the test driver and tested stubs needed to test unit D.

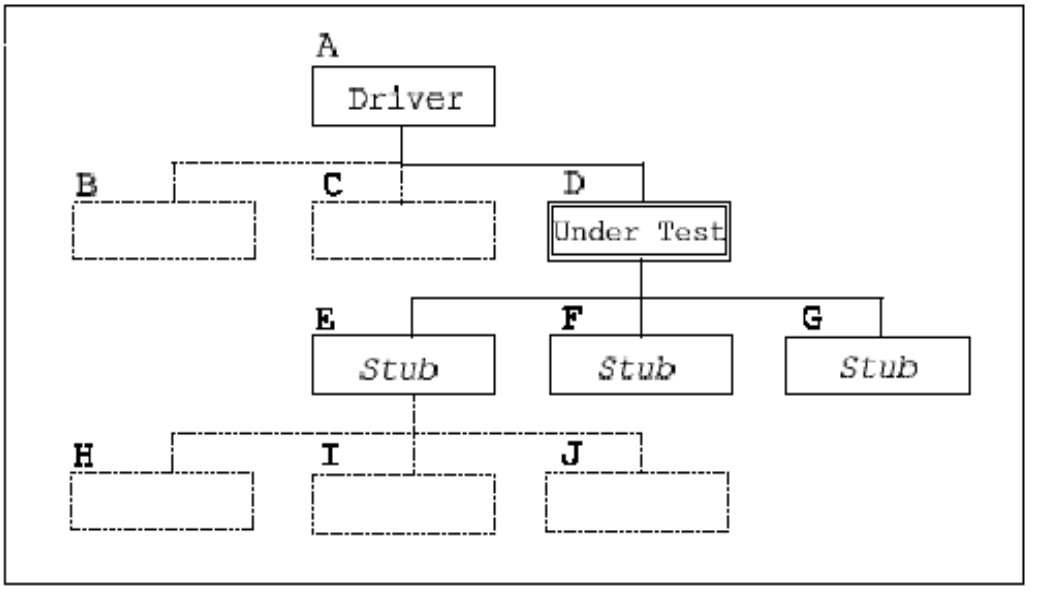


Figure 5.3 - Isolation Testing

A unit test plan for the program shown in Figure 5.3, using a strategy based on the isolation organizational approach, contains only one step, as follows:

Step (1)

(Note that there is only one step to the test plan. The sequence of tests is unimportant; all tests can be executed in parallel.)

Test unit A, using a driver to start the test and stubs in place of units B, C and D;

Test unit B, using a driver to call it in place of unit A;

Test unit C, using a driver to call it in place of unit A;

Test unit D, using a driver to call it in place of unit A and stubs in place of units E, F and G,

Test unit E, using a driver to call it in place of unit D and stubs in place of units H, I and J;

Test unit F, using a driver to call it in place of unit D;

Test unit G, using a driver to call it in place of unit D;

Test unit H, using a driver to call it in place of unit E;

Test unit I, using a driver to call it in place of unit E;

Test unit J, using a driver to call it in place of unit E.

5.4.1 Advantages

It is easier to test an isolated unit thoroughly, where the unit test is removed from the complexity of other units. Isolation testing is the easiest way to achieve good structural coverage, and the difficulty of achieving good structural coverage does not vary with the position of a unit in the unit hierarchy.

As only one unit is being tested at a time, the test drivers tend to be simpler than for bottom up testing, while the stubs tend to be simpler than for top down testing. With an isolation approach to unit testing, there are no dependencies between the unit tests, so the unit test phase can overlap the detailed design and code phases of the software life cycle. Any number of units can be tested in parallel, to give a 'short and fat' unit test phase. This is a useful way of using an increase in team size to shorten the overall time of a software development.

A further advantage of the removal of interdependency between unit tests is that changes to a unit only require changes to the unit test for that unit, with no impact on other unit tests. This results in a lower cost than the bottom up or top down organizational approaches, especially when changes are made.

An isolation approach provides a distinct separation of unit testing from integration testing, allowing developers to focus on unit testing during the unit test phase of the software lifecycle, and on integration testing during the integration phase of the software lifecycle.

Isolation testing is the only pure approach to unit testing, both top down testing and bottom up testing result in a hybrid of the unit test and integration phases. Unlike the top down and bottom up approaches, a unit being referenced from more than another unit does not affect the isolation approach to unit testing.

5.4.2 Disadvantages

The main disadvantage of an isolation approach to unit testing is that it does not provide any early integration of units. Integration has to wait for the integration phase of the software lifecycle.

An isolation approach to unit testing requires structural design information and the use of both stubs and drivers. This can lead to higher costs than bottom up testing for units near the bottom of the unit hierarchy. However, this will be compensated by simplified testing for units higher in the unit hierarchy, together with lower costs each time a unit is changed.

5.4.3 Overall

An isolation approach to unit testing is the best overall choice. When supplemented with an appropriate integration strategy, it enables shorter development timescales and provides the lowest cost, both during development and for the overall lifecycle. Following unit testing in isolation, tested units can be integrated in a top down or bottom up sequence, or any convenient groupings and combinations of groupings. However, a bottom up integration is the most compatible strategy with current trends in object-oriented and object-biased designs.

An isolation approach to unit testing is the best way of achieving the high levels of structural coverage essential for high integrity and safety critical applications, and which are required by many standards. With all the difficult work of achieving good structural coverage achieved by unit testing, integration testing can concentrate on overall functionality and the interactions between units.

5.5 Conclusion

In practice, it is unlikely that any single approach to unit testing can be used exclusively. Typically, an isolation approach to unit testing is modified with some bottom up testing, in which the called units are a mixture of stubs and already tested real units. For example, it makes more sense for a mathematical function to be used directly, provided that it has already been tested and is unlikely to change.

The recommended strategy is to:

- Base our unit test strategy on the isolation approach, and then integrate groups of tested units bottom up.
- Compromise by incorporating some bottom up where it is convenient (for example, using real operators, mathematical functions, string manipulation and the like), but remember the potential impact of changes.

This will result in the lowest cost; both to develop unit tests, and to repeat and maintain tests following changes to units, whilst also facilitating the thorough test coverage necessary to achieve reliable software. Remember that unit testing is about testing units, and that integration testing is about testing the interaction between tested units.

SUMMARY

- Unit testing is the testing of individual components (units) of the software.
- In top down testing, individual units are tested by using them from the units that call them, but in isolation from the units called.
- Top down unit testing provides an early integration of units before the software integration phase.
- The top down organizational approach is not a good choice for unit testing.
- In bottom up unit testing, units are tested in isolation from the units, which call them, but using the actual units called as part of the test.
- Bottom up approach to unit testing is useful when the detailed design documentation lacks structural detail.
- Bottom up strategy will cost more due to complexity of testing units above the bottom level in the unit hierarchy and the high impact of changes.
- Isolation testing tests each unit in isolation from the units, which call it, and the units it calls.
- Isolation testing is the only pure approach to unit testing.
- The main disadvantage of an isolation approach to unit testing is that it does not provide any early integration of units.

CHECK YOUR PROGRESS

1. A _____is an imitation of a unit, used in place of the real unit to facilitate testing.

2. Top down testing requires test stubs, but not test drivers. **[True/False]**

3. Redundant functionality in lower level units will be identified by top down unit testing because there will be no route to test it. **[True/False]**

4. With an isolation approach to unit testing, there are dependencies between the unit tests. **[True/False]**

5. It is not easy to test an isolated unit. **[True/False]**

6. After unit testing in isolation, tested units can be integrated in a top down or bottom up sequence. **[True/False]**

Onlinevarsity

your e-way to learning

Visit

Frequently Asked Questions

@

www.onlinevarsity.com

Chapter

6 Designing Unit Test Cases

Objectives

At the end of this chapter, you will be able to:

- *Develop unit test specifications*
- *Explain test case Design Techniques*

6.1 Introduction

The design of tests is subject to the same basic engineering principles as the design of software. Good test design consists of a number of stages, which progressively elaborate the design of tests:

- Test strategy
- Test planning
- Test specification
- Test procedure

These four stages of test design apply to all levels of testing, from unit testing through to system testing. The design of tests has to be driven by the specification of the software. In unit testing, tests are designed to verify that an individual unit implements all design decisions made in the unit's design specification. A thorough unit test specification should include positive testing, which the unit does what it is supposed to do, and negative testing, which the unit does not do anything that it is not supposed to do.

Producing a test specification including the design of test cases is the level of test design that has the highest degree of creative input. Furthermore, unit test specifications will usually be produced by a large number of staff with a wide range of experience, not just a few experts.

6.2 Developing Unit Test Specifications

Once a unit has been designed, the next development step is to design the unit tests. It is important to note that it is more rigorous to design the tests before the code is written. If the code was written first, it would be too tempting to test the software against what it is observed to do (which is not really testing at all), rather than against what it is specified to do. A unit test specification comprises a sequence of unit test cases. Each unit test case should include four essential elements:

- A statement of the initial state of the unit, the starting point of the test case (this is only applicable where a unit maintains state between calls)
- The inputs to the unit including the value of any external data read by the unit
- What the test case actually tests, in terms of the functionality of the unit and the analysis used in the design of the test case (for example, which decisions within the unit are tested)
- The expected outcome of the test case (the expected outcome of a test case should always be defined in the test specification, prior to test execution)

The following subsections provide general process for developing a unit test specification as a set of individual unit test cases. For each step of the process, suitable test case design techniques are suggested. (Note that these are only suggestions)

6.2.1 Step 1 - Make it Run

The purpose of the first test case in any unit test specification should be to execute the unit under test in the simplest way possible. When the tests are actually executed, knowing that at least the first unit test will execute, is a good confidence boost. If it will not execute then it is preferable to have something as simple as a starting point for debugging.

Suitable techniques:

- Specification derived tests
- Equivalence partitioning

6.2.2 Step 2 - Positive Testing

Test cases should be designed to show that the unit under test does what it is supposed to do. The test designer should walk through the relevant specifications; each test case should test one or more statements of specification. Where more than one specification is involved, it is best to make the sequence of test cases corresponding to the sequence of statements in the primary specification for the unit.

Suitable techniques:

- Specification derived tests
- Equivalence partitioning
- State-transition testing

6.2.3 Step 3 - Negative Testing

Existing test cases should be enhanced and further test cases should be designed to show that the software does not do anything that it is not specified to do. This step depends primarily upon error guessing, relying upon the experience of the test designer to anticipate problem areas.

Suitable techniques:

- Error guessing
- Boundary value analysis
- Internal boundary value testing
- State-transition testing

6.2.4 Step 4 - Special Considerations

Test cases should be designed to address issues such as performance, safety requirements and security requirements. Particularly in the cases of safety and security, it can be convenient to give test cases special emphasis to facilitate security analysis or safety analysis and certification. Test cases already designed that addresses security issues or safety hazards should be identified in the unit test specification. Further, test cases should then be added to the unit test specification to ensure that all security issues and safety hazards applicable to the unit will be fully addressed.

Suitable techniques:

- Specification derived tests

6.2.5 Step 5 - Coverage Tests

The test coverage likely to be achieved by the designed test cases should be visualized. Further, test cases can then be added to the unit test specification to achieve specific test coverage objectives. Once coverage tests have been designed, the test procedure can be developed and the tests executed.

Suitable techniques:

- Branch testing
- Condition testing
- Data definition-use testing
- State-transition testing

6.2.6 Step 6 -Test Execution

A test specification designed using the above five steps should in most cases provide a thorough test for a unit. At this point, the test specification can be used to develop an actual test procedure, and the test procedure used to execute the tests. Execution of the test procedure will identify errors in the unit, which can be corrected, and the unit re-tested. Dynamic analysis during execution of the test procedure will yield a measure of test coverage, indicating whether coverage objectives have been achieved. There is, therefore, a further coverage completion step in the process of designing test specifications.

6.2.7 Step 7 - Coverage Completion

Depending upon an organization's standards for the specification of a unit, there may be no structural specification of processing within a unit other than the code itself. Possibly, there are also likely to have been human errors made in the development of a test specification. Consequently, there may be complex decision conditions, loops and branches within the code for which coverage targets may not have been met when tests were executed. Wherever coverage objectives are not achieved, analysis must be conducted to determine the reasons behind them.

Failure to achieve a coverage objective may be due to:

- Infeasible paths or conditions - The corrective action should be to annotate the test specification to provide a detailed justification of why the path or condition is not tested.

- Unreachable or redundant code - The corrective action will probably be to delete the offending code. It is easy to make mistakes in this analysis, particularly where defensive programming techniques have been used. If there is any doubt, defensive programming should not be deleted.

- Insufficient test cases - Test cases should be refined and further test cases added to a test specification to fill the gaps in test coverage. Ideally, the coverage completion step should be conducted without looking at the actual code. However, in practice some sight of the code may be necessary to achieve coverage targets. It is vital that all test designers should recognize that use of the coverage completion step should be minimized.

The most effective testing will come from analysis and specification, not from experimentation and over dependence upon the coverage completion step.

Suitable techniques:

- Branch testing
- Condition testing

- Data definition-use testing
- State-transition testing

6.2.8 General Guidance

Note that the first five steps in producing a test specification can be achieved:

- Solely from design documentation
- Without looking at the actual code
- Before developing the actual test procedure

It is usually a good idea to avoid long sequences of test cases, which depend upon the outcome of the preceding test cases. An error identified by a test case early in the sequence could cause secondary errors and reduce the amount of real testing achieved when the tests are executed.

The process of designing test cases including executing them as “thought experiments”, often identifies bugs before the software has even been built. It is common to find more bugs when designing tests than when executing tests. Throughout unit test design, the primary input should be the specification documents for the unit under test. While use of actual code as an input to the test design process may be necessary in some circumstances, test designers must take care that they are not testing the code against itself. A test specification developed from the code will only prove that the code does what the code does, not that it does what it is supposed to do.

6.3 Test Case Design Techniques

Test case design techniques can be broadly split into two main categories. Black box techniques use the interface to a unit and a description of functionality, but do not need to know how the inside of a unit is built. White box techniques make use of information about how the inside of a unit works. There are also some other techniques, which do not fit into either of the above categories. Error guessingzz falls into this category.

Black box (functional)	White box (structural)	Other
Specification derived tests	Branch testing	Error guessing
Equivalence partitioning	Condition testing	
Boundary value analysis	Data definition use testing	
State transition testing	Internal boundary value testing	

Table 6.1 – Categories of test case design techniques

6.3.1 Specification Derived Tests

As the name suggests, test cases are designed by walking through the relevant specifications. Each test case should test one or more statements of specification. It is often practical to make the sequence of test cases correspond to the sequence of statements in the specification for the unit under test. For example, consider the specification for a function to calculate the square root of a real number, shown below:

Input - real number

Output - real number

When given an input of 0 or greater, the positive square root of the input shall be returned.

When given an input of less than 0, the error message “Square root error - illegal negative input” shall be displayed and a value of 0 returned. The library routine Print_Line shall be used to display the error message.

There are three statements in this specification, which can be addressed by two test cases. Note that the use of Print_Line conveys structural information in the specification.

Test Case 1: Input 4, Return 2

- Exercises the first statement in the specification (“When given an input of 0 or greater, the positive square root of the input shall be returned.”).

Test Case 2: Input -10, Return 0, Output “Square root error - illegal negative input” using Print_Line.

- Exercises the second and third statements in the specification

(“When given an input of less than 0, the error message “Square root error - illegal negative input” shall be displayed and a value of 0 returned. The library routine Print_Line shall be used to display the error message.”).

Specification derived test cases can provide an excellent correspondence to the sequence of statements in the specification for the unit under test, enhancing the readability and maintainability of the test specification. However, specification derived testing is a positive test case design technique. Consequently, specification derived test cases have to be supplemented by negative test cases to provide a thorough unit test specification. A variation of specification derived testing is to apply a similar technique to a security analysis, safety analysis, software hazard analysis, or other document, which provides supplementary information to the unit’s specification.

6.3.2 Equivalence Partitioning

Equivalence partitioning is a much more formalized method of test case design. It is based upon splitting the inputs and outputs of the software under test into a number of partitions, where the behavior of the software is equivalent for any value within a particular partition. Data that forms partitions is not just routine parameters. Partitions can also be present in data accessed by the software, in time, in input and output sequence, and in state. Equivalence partitioning assumes that all values within any individual partition are equivalent for test purposes. Test cases should therefore be designed to test one value in each partition. Reconsider the square root function used in the previous example. The square root function has two input partitions and two output partitions as tabled below.

Input partitions		Output partitions	
i	<0	a	> = 0
ii	>=0	b	Error

Table 6.2 : Partitions for square root

These four partitions can be tested with two test cases:

Test Case 1: Input 4, Return 2

- Exercises the >=0 input partition (ii)
- Exercises the >=0 output partition (a)

Test Case 2: Input -10, Return 0, Output “Square root error - illegal negative input”

using Print_Line.

- Exercises the <0 input partition (i)
- Exercises the “error” output partition (b)

Considering a function like square root, we can see that equivalence partitioning is quite simple. We can have one test case for a positive number and a real result; and a second test case for a negative number and an error result. However, as software becomes more complex, the identification of partitions and the inter-dependencies between partitions becomes much more difficult, making it less convenient to use this technique to design test cases. Equivalence partitioning is still a positive test case design technique and needs to be supplemented by negative tests.

6.3.3 Boundary Value Analysis

Boundary value analysis uses the same analysis of partitions as equivalence partitioning.

However, boundary value analysis assumes that errors are most likely to exist at the boundaries between partitions. Boundary value analysis consequently incorporates a degree of negative testing into the test design, by anticipating that errors will occur at or near the partition boundaries. Test cases are designed to exercise the software on and at either side of boundary values. Consider the two input partitions in the square root example, as illustrated by Figure 6.1.

```
Input
Partitions (i)                                          (ii)
-                                  0                                  +
|——————————————————————————|————————————————————————————————————|
1                                2 3 4 5
Boundaries and test cases
```

Figure 6.1 - Input Partition Boundaries in Square Root

The zero or greater partition has a boundary at 0 and a boundary at the most positive real number. The less than zero partition shares the boundary at 0 and has another boundary at the most negative real number. The output has a boundary at 0, below which it cannot go.

Test Case 1: Input {the most negative real number}, Return 0, Output “Square root error - illegal negative input” using Print_Line - Exercises the lower boundary of partition (i).

Test Case 2: Input {just less than 0}, Return 0, Output “Square root error - illegal negative input” using Print_Line - Exercises the upper boundary of partition (i).

Test Case 3: Input 0, Return 0 - Exercises just outside the upper boundary of partition (i), the lower boundary of partition (ii) and the lower boundary of partition (a).

Test Case 4: Input {just greater than 0}, Return {the positive square root of the input} - Exercises just inside the lower boundary of partition (ii).

Test Case 5: Input {the most positive real number}, Return {the positive square root of the input} - Exercises the upper boundary of partition (ii) and the supper boundary of partition (a).

As for equivalence partitioning, it can become impractical to use boundary value analysis thoroughly for more complex software. Boundary value analysis can also be meaningless for non-scalar data such as enumeration values. In the example, partition (b) does not really have boundaries. For purists, boundary value analysis requires knowledge of the underlying representation of the

numbers. A more pragmatic approach is to use any small values above and below each boundary, and suitably big positive and negative numbers.

6.3.4 State-Transition Testing

State transition testing is particularly useful where either the software has been designed as a state machine or the software implements a requirement that has been modeled as a state machine. Test cases are designed to test the transitions between states by creating the events, which lead to transitions.

6.3.5 Branch Testing

In branch testing, test cases are designed to exercise control flow branches or decision points in a unit. This is usually aimed at achieving a target level of Decision Coverage. Given a functional specification for a unit, a “black box” form of branch testing is to “guess” where branches may be coded and to design test cases to follow the branches. However, branch testing is really a “white box” or structural test case design technique. Given a structural specification for a unit, specifying the control flow within the unit, test cases can be designed to exercise branches. Such a structural unit specification will typically include a flowchart or PDL.

Returning to the square root example, a test designer could assume that there would be a branch between the processing of valid and invalid inputs, leading to the following test cases:

Test Case 1: Input 4, Return 2 -Exercises the valid input-processing branch

Test Case 2: Input -10, Return 0, Output “Square root error - illegal negative input” using Print_Line. - Exercises the invalid input processing branch

However, there could be many different structural implementations of the square root function. The following structural specifications are all valid implementations of the square root function, but the above test cases would only achieve decision coverage of the first and third versions of the specification.

Specification 1

```
If input<0 THEN
CALL Print_Line “Square root error - illegal negative input”
RETURN 0
ELSE
Use maths co-processor to calculate the answer
RETURN the answer
END_IF
```

Specification 2

```
If input<0 THEN
CALL Print_Line “Square root error - illegal negative input”
RETURN 0
ELSE
IF input=0 THEN
RETURN 0
ELSE
Use maths co-processor to calculate the answer
RETURN the answer
END_IF
END_IF
```

Specification 3

```
Use maths co-processor to calculate the answer
Examine co-processor status registers
If status=error THEN
CALL Print_Line “Square root error - illegal negative input”
RETURN 0
ELSE
RETURN the answer
END_IF
```

Specification 4

```
If input<0 THEN
CALL Print_Line “Square root error - illegal negative input”
RETURN 0
ELSE_IF input=0 THEN
RETURN 0
ELSE
Calculate first approximation
LOOP
Calculate error
EXIT_LOOP WHEN error<desired accuracy
Adjust approximation
END_LOOP
RETURN the answer
END_IF
```

It can be seen that branch testing works best with a structural specification for the unit. A structural unit specification will enable branch test cases to be designed to achieve decision coverage, but a purely functional unit specification could lead to coverage gaps.

Be careful while concentrating upon branches because a test designer could loose sight of the overall functionality of a unit. It is important to always remember that it is the over all functionality of a unit that is important, and that branch testing is a means to an end, not an end in itself. Another consideration is that branch testing is based solely on the outcome of decisions. It makes no allowances for the complexity of the logic, which leads to a decision.

6.3.6 Condition Testing

There are a range of test case design techniques, which fall under the general title of condition testing, all of which endeavor to mitigate the weaknesses of branch testing when complex logical conditions are encountered. The objective of condition testing is to design test cases to show that the individual components of logical conditions and combinations of the individual components are correct.

Test cases are designed to test the individual elements of logical expressions, both within branch conditions and within other expressions in a unit. As for branch testing, condition testing could be used as a “black box” technique, where the test designer makes intelligent guesses about the implementation of a functional specification for a unit. However, condition testing is more suited to “white box” test design from a structural specification for a unit.

When illustrating condition testing, consider the example specification for the square root function, which uses successive approximation (Specification 4). Suppose that the designer for the unit made a decision to limit the algorithm to a maximum of 10 iterations because after 10 iterations the answer would be as close as it would ever get. The PDL specification for the unit could specify an exit condition like that given below:

```
:
:
EXIT_LOOP WHEN (error<desired accuracy) or (iterations=10)
:
:
:
```

If the coverage objective is Modified Condition Decision Coverage, test cases have to prove that both error<desired accuracy and iterations=10 can independently affect the outcome of the decision.

Test Case 1: 10 iterations, error>desired accuracy for all iterations.

- Both parts of the condition are false for the first 9 iterations. On the tenth iteration, the first part of the condition is false and the second part becomes true, showing that the iterations=10 part of the condition can independently affect its outcome.

Test Case 2: 2 iterations, error>=desired accuracy for the first iteration, and error<desired accuracy for the second iteration.

- Both parts of the condition are false for the first iteration.

On the second iteration, the first part of the condition becomes true and the second part remains false, showing that the error<desired accuracy part of the condition can independently affect its outcome. Condition testing works best when a structural specification for the unit is available. It provides a thorough test of complex conditions, an area of frequent programming and design error and an area, which is not addressed by branch testing. As for branch testing, it is important for test designers to beware that concentrating on conditions could distract a test designer from the overall functionality of a unit.

6.3.7 Data Definition-Use Testing

Data definition-use testing designs test cases to test pairs of data definitions and uses. A data definition is anywhere that the value of a data item is set, and a data use is anywhere that a data item is read or used. The objective is to create test cases, which will drive execution through paths between specific definitions and uses.

Like decision testing and condition testing, data definition-use testing can be used in combination with a functional specification for a unit, but is better suited to use with a structural specification for a unit.

Consider one of the earlier PDL specifications for the square root function, which sent every input to the math co-processor and used the co-processor status to determine the validity of the result (Specification 3). The first step is to list the pairs of definitions and uses. In this specification, there are a number of definition-use pairs, as shown in table below.

	Definition	Use
1	Input to routine	By the math co-processor
2	Co-processor status	Test for status-error
3	Error message	By print line
4	Return 0	By the calling unit
5	Answer by the co-processor	Return the answer
6	Return the answer	By the calling unit

Table 6.3 : Pairs of definitions and uses

These pairs of definitions and uses can then be used to design test cases. Two test cases are required to test all six of these definition-use pairs:

Test Case 1: Input 4, Return 2

- Tests definition-use pairs 1, 2, 5, and 6

Test Case 2: Input -10, Return 0, Output “Square root error - illegal negative input” using Print_Line.

- Tests definition-use pairs 1, 2, 3, 4

The analysis needed to develop test cases using this design technique can also be useful or identifying problems before the tests are even executed; for example, identification of situations where data is used without having been defined. This is the sort of data flow analysis that some static analysis tool can take help. The analyses of data definition-use pairs, which can become very complex, even for relatively simple units. Consider what the definition-use pairs would be for the successive approximation version of square root!

It is possible to split data definition-use tests into two categories of uses, which affect control and flow (predicate uses); and uses, which are purely computational.

6.3.8 Internal Boundary Value Testing

In many cases, partitions and their boundaries can be identified from a functional specification for a unit, as described under equivalence partitioning and boundary value analysis above. However, a unit may also have internal boundary values, which can only be identified from a structural specification. Consider a fragment of the successive approximation version of the square root unit specification, as shown below:

```
:
Calculate first approximation
LOOP
Calculate error
EXIT_LOOP WHEN error<desired accuracy
Adjust approximation
END_LOOP
RETURN the answer
:
```

The calculated error can be in one of the two partitions about the desired accuracy, a feature of the structural design for the unit, which is not apparent from a purely functional specification. An analysis of internal boundary values yields three conditions for which test cases need to be designed.

Test Case 1: Error just greater than the desired accuracy

Test Case 2: Error equal to the desired accuracy

Test Case 3: Error just less than the desired accuracy

Internal boundary value testing can help to bring out some elusive bugs. For example, suppose "<=" had been coded instead of the specified "<". Nevertheless, internal boundary value testing is a luxury to be applied only as a final supplement to other test case design techniques.

6.3.9 Error Guessing

Error guessing is based mostly upon experience, with some assistance from other techniques such as boundary value analysis. Based on experience, the test designer guesses the types of errors that could occur in a particular type of software and designs test cases to uncover them. For example, if any type of resource is allocated dynamically, a good place to look for errors is in the de-allocation of resources. Are all resources correctly de-allocated, or are some lost as the software executes?

Error guessing by an experienced engineer is probably the single most effective method of designing tests, which uncover bugs. A well-placed error guess can show a bug, which could easily be missed by many of the other test case design techniques.

Conversely, in the wrong hands error guessing can be a waste of time. When making the maximum use of available experience and adding some structure to this test case design technique, it is a good idea to build a checklist of different types of errors. This checklist can then be used to help "guess" where errors may occur within a unit. The checklist should be maintained with the benefit of experience gained in earlier unit tests, helping to improve the overall effectiveness of error guessing.

6.4 Conclusion

Experience has shown that a conscientious approach to unit testing will detect many bugs at a stage of the software development where they can be corrected economically. A rigorous approach to unit testing requires the following:

- The design of units is documented in a specification before coding begins
- Unit tests are designed from the specification for the unit, also preferably before coding begins
- The expected outcomes of unit test cases are specified in the unit test specification

The process for developing unit test specifications presented in this paper is generic, in that it can be applied to any level of testing. Nevertheless, there will be circumstances where it has to be tailored to specific situations. Tailoring of the process and the use of test case design techniques should be documented in the overall test strategy.

Although the square root example used to illustrate the test case design techniques is trivial, it does serve to show the principles behind the techniques. It is unlikely that any single test case design techniques will lead to a particularly thorough test specification. When used to complement each other through each stage of the test specification development process, the synergy of techniques can be much more effective. Nevertheless, test designers should not let any of these techniques obstruct the application of experience and common sense.

SUMMARY

- Good test design consists of a number of stages.
- For unit testing, tests are designed to verify that an individual unit implements all design decisions made in the unit's design specification.
- A unit test specification comprises a sequence of unit test cases.
- Test case design techniques can be broadly split into two main categories of Black box and White box.
- Error guessing is based mostly upon experience of the test designer.

CHECK YOUR PROGRESS

1. Test cases should be designed to show that the unit under test does what it is supposed to do. **[True/False]**

2. Failure to achieve a coverage objective may be due to infeasible paths or conditions. **[True/False]**

3. Test cases already designed, which addresses security issues or safety hazards, should not be identified in the unit test specification. **[True/False]**

4. _____is based upon splitting the inputs and outputs of the software under testinto a number of partitions.

5. Condition testing is more suited to _____ test design from a structural specification for a unit.

6. The design of units is documented in a specification before coding begins. **[True/False]**

7. Boundary value analysis assumes that errors are most likely to exist at the boundaries between partitions. **[True/False]**

Onlinevarsity

your e-way to learning

Get

WORD WISE

Visit

Glossary@

www.onlinevarsity.com